Introduction to Machine Learning 2: Adding Background

SE G QPI QM P

Training Objective

SIEMENS
Ingenuity for life

By the end of this course, you will …

- Have an understanding / basic mathematical background of decision tree and neural networks taught in ML1
- Be enabled to better interpret Machine Learning results using this background
- Know enough about ML to explore other algorithms on your own
- Discuss application opportunities of Machine Learning in our business

This training is an experiment

- New teaching approach, digging deeper after targeting “everybody”
- Focus is not to program it yourself in python
- Helpline (Kay)

We are all learning: We don’t have very much experience yet, where Machine Learning is actually improving our business.

How This Training Works

SIEMENS
Ingenuity for life

- Experiment, we are all learning
- ML1 is prerequisite (enough to read through slides)
- 2x10 Min Break
- Recording: high demand, few training seats
- Based on interaction
  - Discussions, Exercises + Debriefs
  - Harder in Circuit
  - If I should not call on you, please tell us now

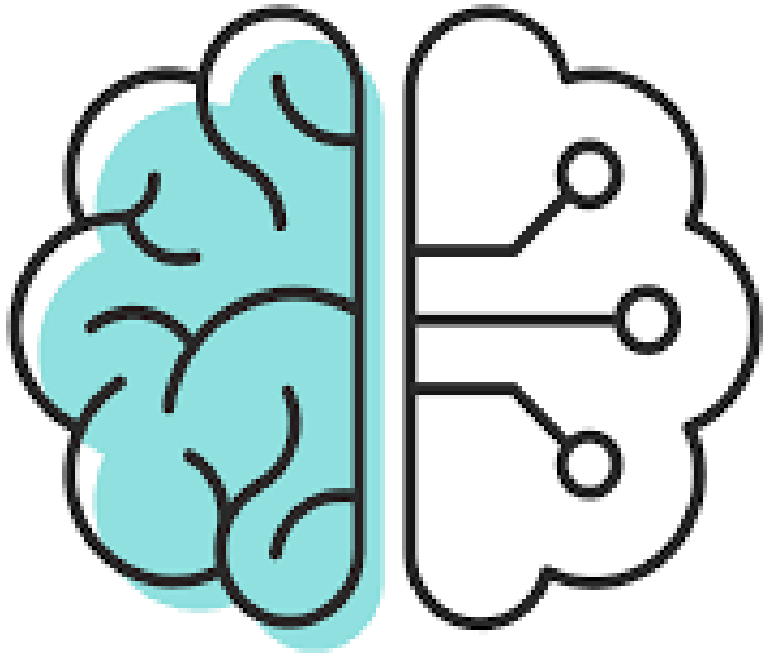
Please Give Us Feedback!

SIEMENS
Ingenuity for life

- **Feedback is very important for us. This course is free, but we need to know if this is beneficial for you.**
- **Based on your feedback, we will continue to offer this, develop additional courses – or not.**

- Please visit at the end of this course the following website: https://forms.office.com/Pages/ResponsePage.aspx?id=PqlLJW8f80iQ5uJ2ZmS0d-5LdjYRShZEhN3YmsD7St9UMFZZTFFVQjFSUUxTMzhQNTM3S1hSM0ExVS4u

Agenda



- Review ML1
- Model Evaluation:
  - Regression Quality Metrics
  - Classification Quality Metrics
  - The Bias-Variance Trade-Off
- Model Types:
  - Decision Trees (Ensemble: Random Forrest)
  - Neural Networks
    - Basic Steps of Training a Neural Network
    - Neural Network Exercise
- Application of Machine Learning in Our Business

Overview

SIEMENS
Ingenuity for life

	Target values	Loss function	Error metric	Act function last layer (for NN)	Act func hidden layers (For NN)
Regression	Continuous	MSE, R^2, MAE,…	MSE, R^2, MAE,…	Linear/ ReLu	ReLu
Classification	Binary	Binary cross entropy	F1 score, precision, recall, ROC-curve	Sigmoid	ReLu
	multiple classes	categorical cross entropy	F1 score, precision, recall, ROC-curve	Softmax	ReLu

	Decision Trees	Artificial Neural Nets
Tabular data	Ensemble methods: Random Forest, Boosting	Starting from simple architectures
Text	-	Transformers (encoding, decoding, attention)
Images	-	CNNs

Matthias Groh, Your Instructor

SIEMENS
Ingenuity for life

- Department “Improvement Projects” of SE Generation Quality
- Data Science consultant
- Six Sigma Master Black Belt

Previous engagements:

- Senior Lean Expert focusing on administration processes @ Siemens Energy Fossil Products, Berlin, Germany
- Director Six Sigma @ IIL, New York, NY
- Six Sigma Black Belt @ Deutsche Bahn, Frankfurt, Germany

Professional degrees in:

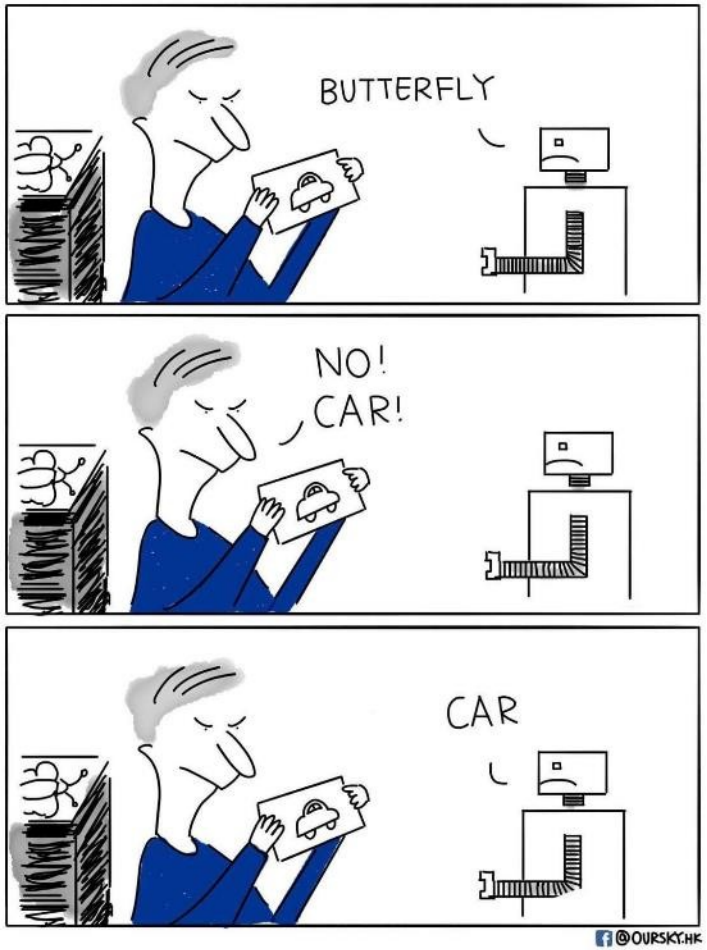
- Precision Mechanics
- Mathematics
- Psychology

Additional qualifications:

- Certified Group Facilitator and Trainer (ABF)
- Certified Six Sigma Master Black Belt (IIL)
- Certified Project Management Professional (PMI)
- Certified Senior Lean Expert (LMI)

If you need help, please contact our support mail box: dataScienceSupport.energy@siemens.com
To contact me personally, email me at groh.matthias@siemens-energy.com

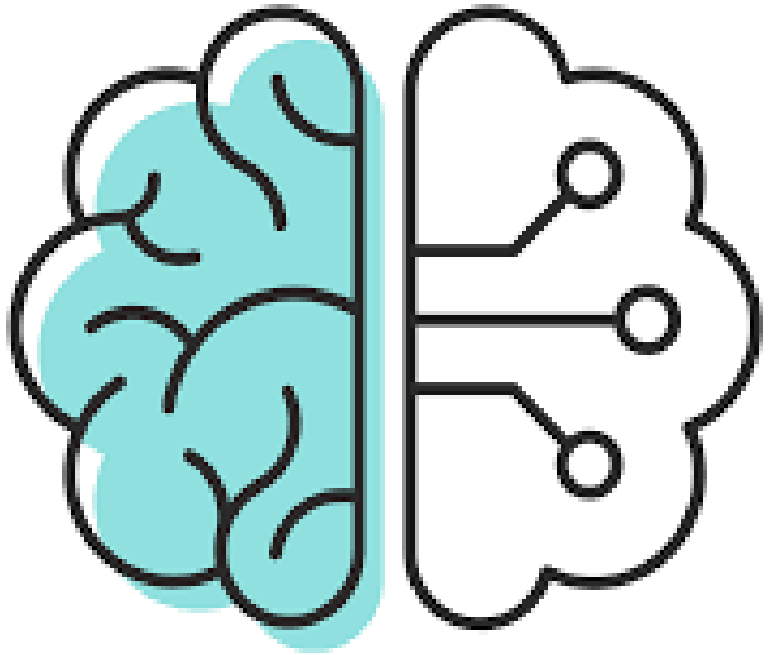
Please Quickly Introduce Yourself



Who is new (has not attended ML 1 class)?

- Your name and function / organization
- Your background regarding Machine Learning

Agenda



- Review ML1
- Model Evaluation:
  - Regression Quality Metrics
  - Classification Quality Metrics
  - The Bias-Variance Trade-Off
- Model Types:
  - Decision Trees (Ensemble: Random Forrest)
  - Neural Networks
    - Basic Steps of Training a Neural Network
    - Neural Network Exercise
- Application of Machine Learning in Our Business

Learning Types

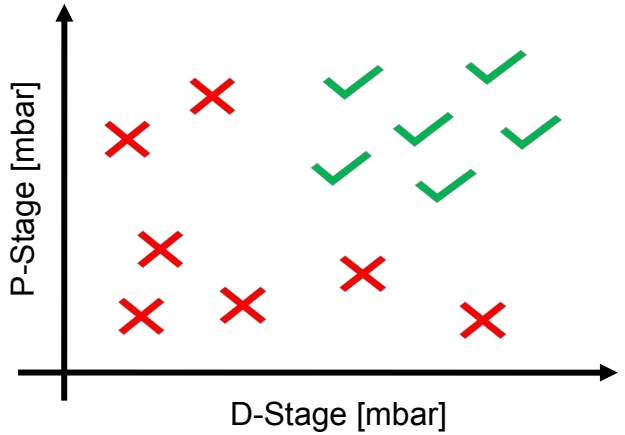
SIEMENS
Ingenuity for life

Supervised Learning

"Features" (Xs) and "Targets" (Ys)

Example: Start Reliability

Y: Successful ignition ✓ / Failure ignition ✗



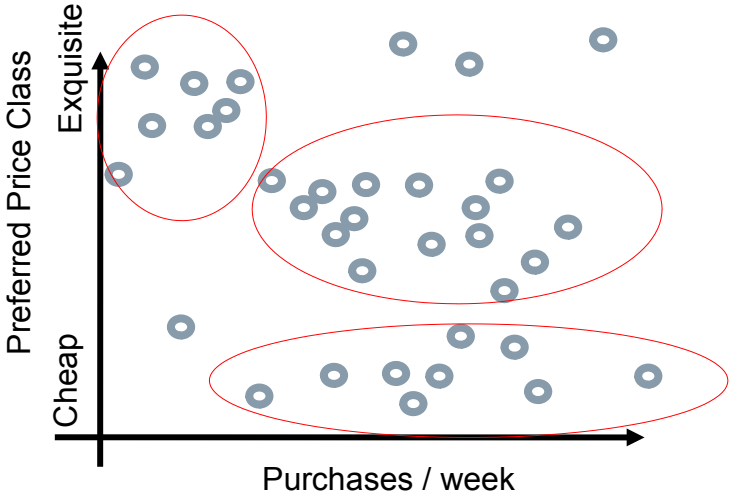
X and Y are known

Only X are known

Unsupervised Learning

Only "Features" (Xs)

Example: Amazon Customer analysis



Result: Model to predict Y: Y = f (X1, … , Xn)
i.e. for assuring next start is reliable

Result: Clusters to group customers in,
i.e. for targeting advertisement

For our business „supervised learning" is probably the most frequent relevant learning type

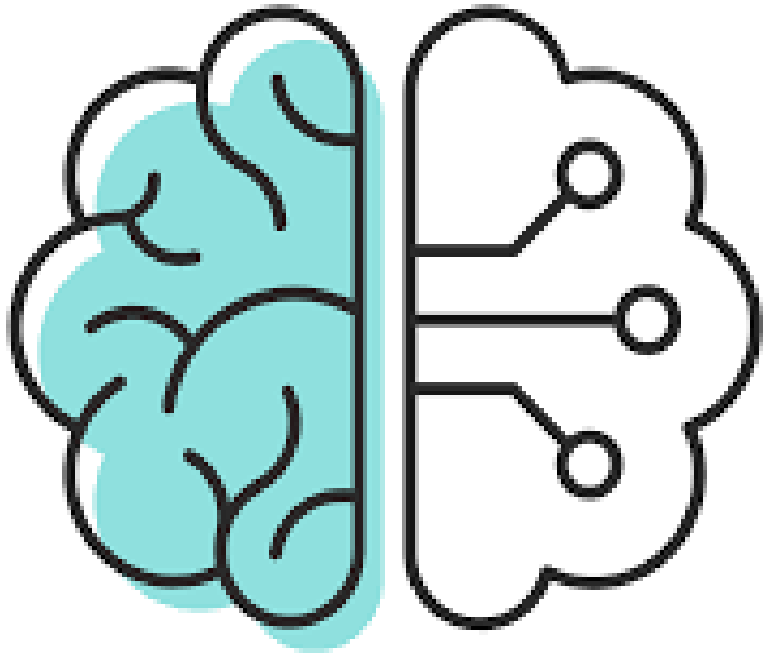
Having a „Supervised Learning“ Approach, We Should Distinguish between the Different Data Types of the Target Variable

SIEMENS
Ingenuity for life

The target variable is	**Attributive**	**Numerical**
	 success failure	40,45 39,97 40,85 38,93 39,12 40,07
The learning approach is called:	**„Classification“:** Selecting the right group for a new element	„**Regression**“: Estimates a numerical value for a new element

Most learning algorithms have a version for each, Classification and Regression

Agenda

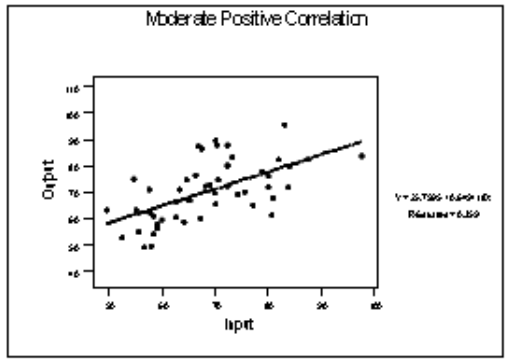


- Review ML1
- Model Evaluation:
  - Regression Quality Metrics
  - Classification Quality Metrics
  - The Bias-Variance Trade-Off
- Model Types:
  - Decision Trees (Ensemble: Random Forrest)
  - Neural Networks
    - Basic Steps of Training a Neural Network
    - Neural Network Exercise
- Application of Machine Learning in Our Business

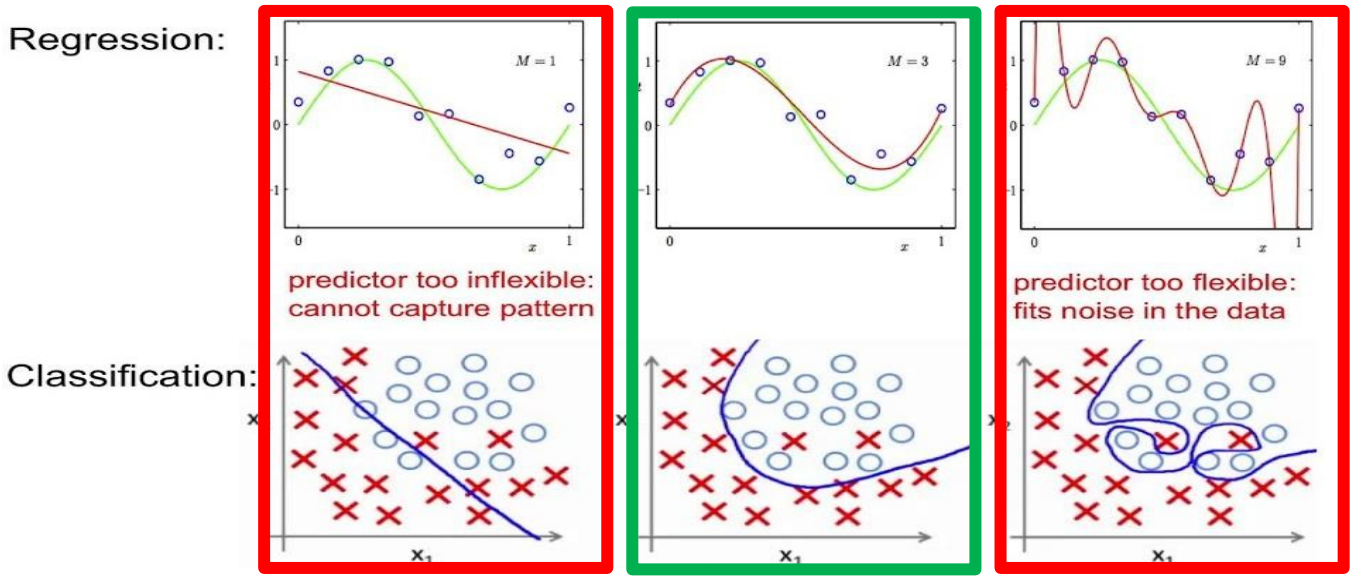
2 Questions to be answered by a Model

SIEMENS
Ingenuity for life

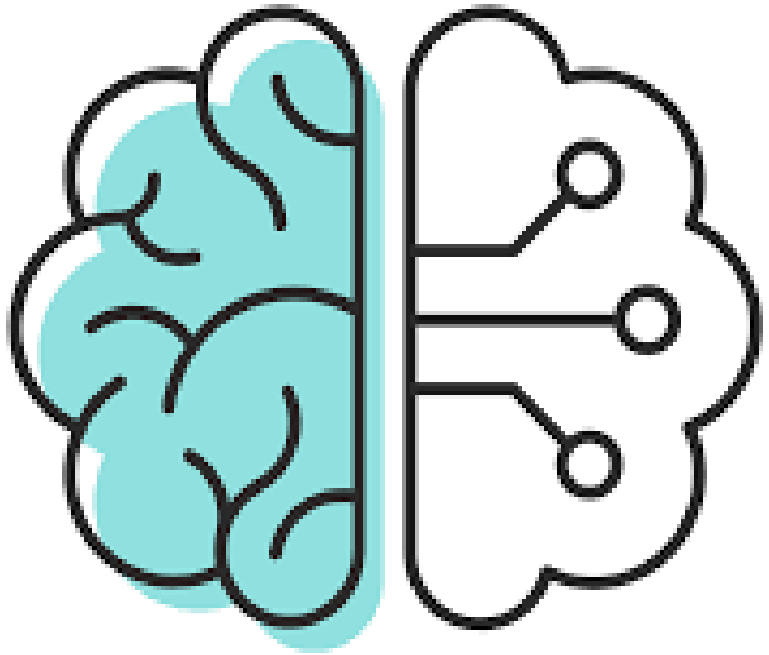
- Does the model represent the given data in the best possible manner? (loss function, error metrics)



- Is the Model general enough to still be valid for unknown data?
(bias – variance tradeoff)



Agenda



- Review ML1
- Model Evaluation:
  - Regression Quality Metrics
  - Classification Quality Metrics
  - The Bias-Variance Trade-Off
- Model Types:
  - Decision Trees (Ensemble: Random Forrest)
  - Neural Networks
    - Basic Steps of Training a Neural Network
    - Neural Network Exercise
- Application of Machine Learning in Our Business

MSE
Mean Squared Error as Error Metric and Loss Function

SIEMENS
Ingenuity for life

Simple linear model: y = ax+c

Error = Estimated value - True value

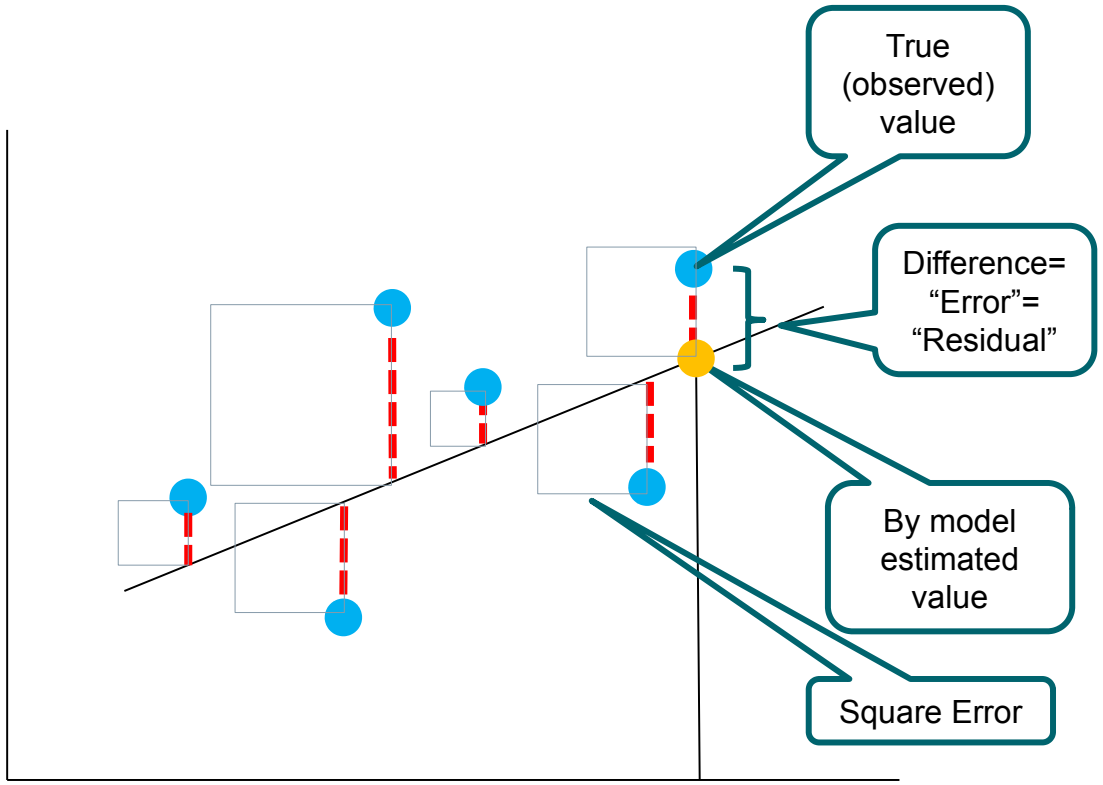
Mean squared error (MSE) measures the average of the squares of the errors

: Predicted Value

$$MSE = \frac{\sum_{i=1}^{n} (\hat{Y}_i - Y_i)^2}{n}$$

: Observed Value

n: Sample Size



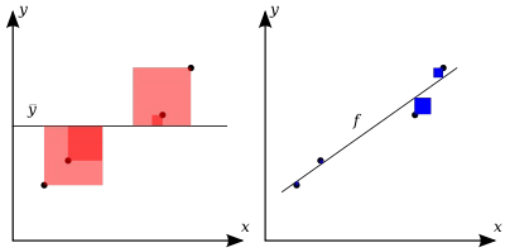
Think of the MSE as an squared average error the model makes while predicting.

R-Square/ “Determination Coefficient”

SIEMENS
Ingenuity for life

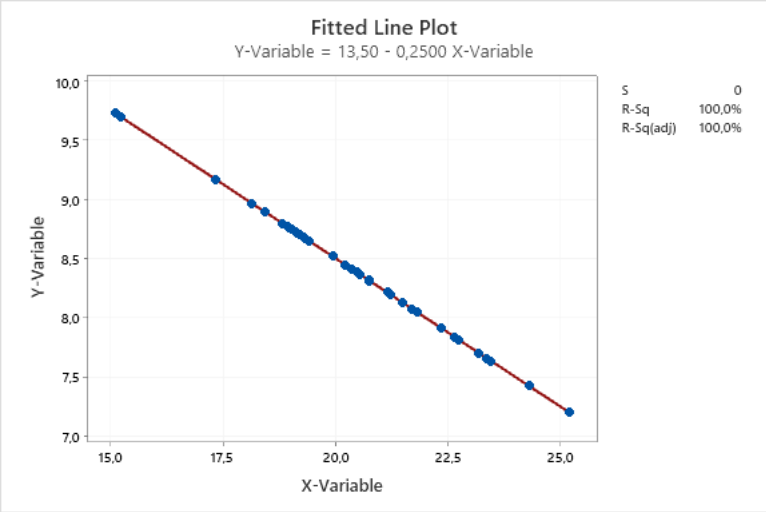
Shows how much % of the variation of the target variable Y can be explained by the variation of the input variables X1, …, Xn

$$r^2 = 1 - \frac{\text{SS Error}}{\text{SS Total}} = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$



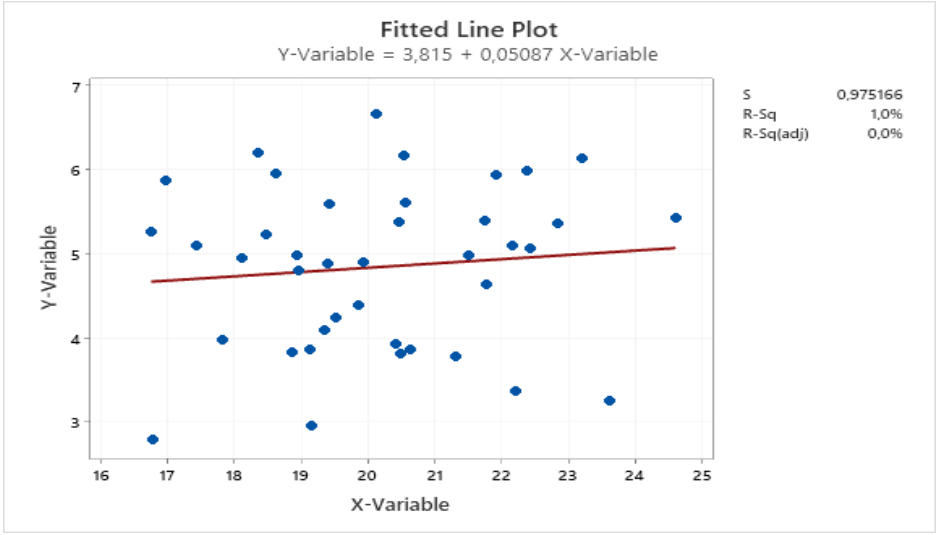
R-Sq = 100%:

- Y can be 100% explained with X1-Xn
- Modell matches observation perfectly



R-Sq = 0:

- no correlation btw. X1-Xn and Y
- Model explains randomness, no correlation btw. X1, …, Xn and Y



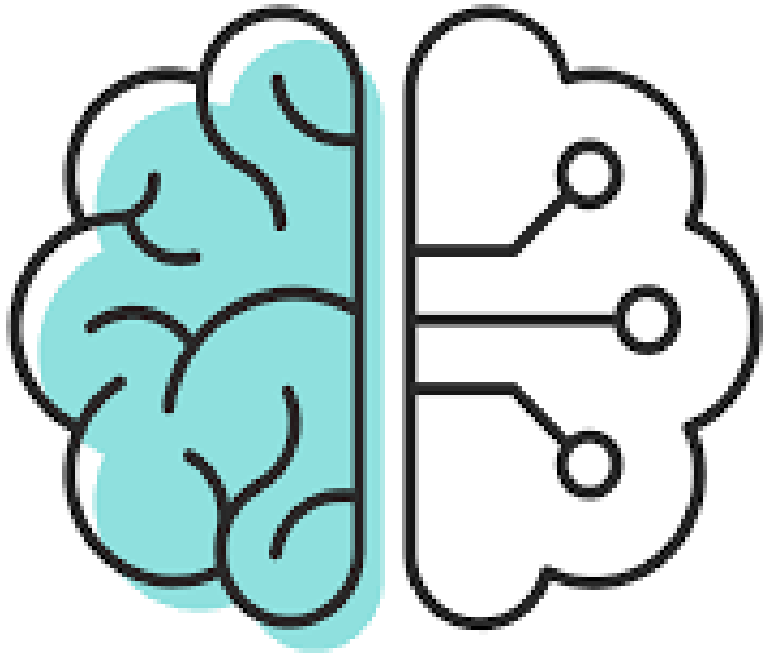
Overview: what we achieved so far

SIEMENS
Ingenuity for life

	Target values	Loss function	Error metric	Act function last layer	Act func hidden layers
Regression	Continuous	MSE, R^2, MAE,…	MSE, R^2, MAE,…		
Classification	Binary				
	multiple classes				

	Decision Trees	Artificial Neural Nets
Tabular data		
Text		
Images		

Agenda

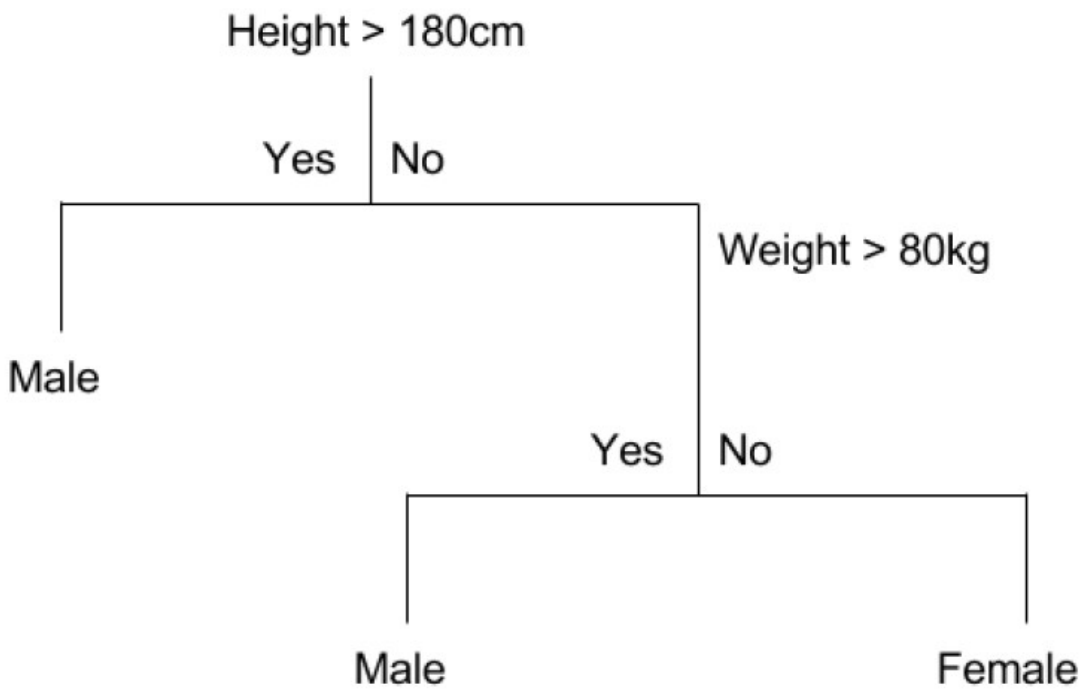


- Review ML1
- Model Evaluation:
  - Regression Quality Metrics
  - Classification Loss Function and Error Metrics
  - The Bias-Variance Trade-Off
- Model Types:
  - Decision Trees (Ensemble: Random Forrest)
  - Neural Networks
    - Basic Steps of Training a Neural Network
    - Neural Network Exercise
- Application of Machine Learning in Our Business

Classification by Decision Trees (Review ML1)

SIEMENS
Ingenuity for life

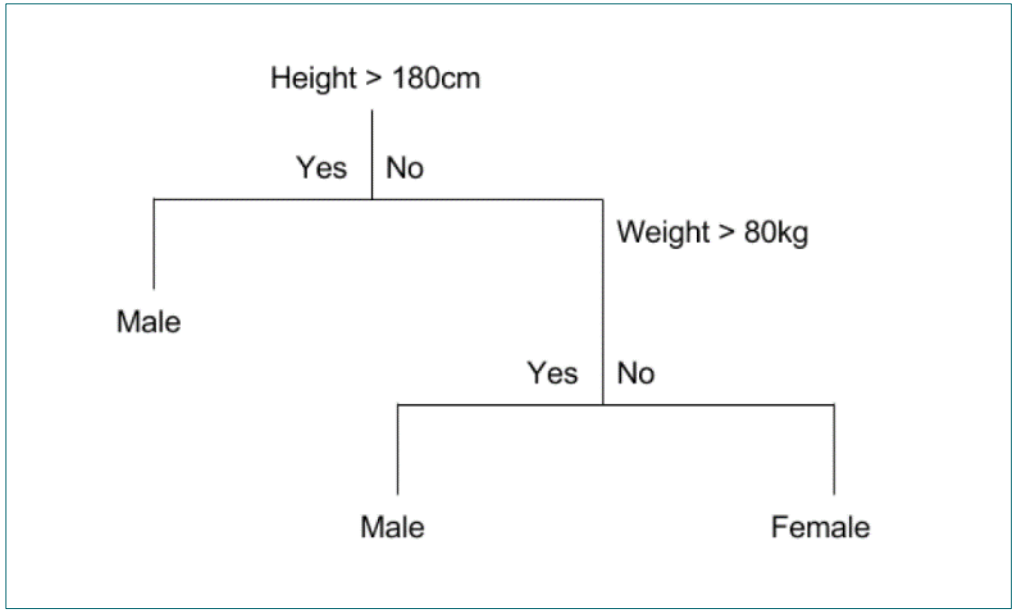
New wording CART (Classification and Regression Tree)
A crude example:



Main Idea How a Decision Tree Is Build (Review ML1): Which Loss Function to be minimized?

SIEMENS
Ingenuity for life

1. Start at the top: first node
2. Evaluate:
   - all input variables
   - all their possible split points
   - Which would have the least uncertainty
3. Split accordingly and generate two leaves
4. Repeat 2 + 3 with both leaves
5. Stop when:
   - Uncertainty can't be further reduced
   - Some predefined metrics are reached:
     - Maximal depth (=consecutive branches) ☾ will play with this
     - Maximal # of leaves



Loss Function: Entropy Measures Uncertainty of a Dataset

SIEMENS
Ingenuity for life

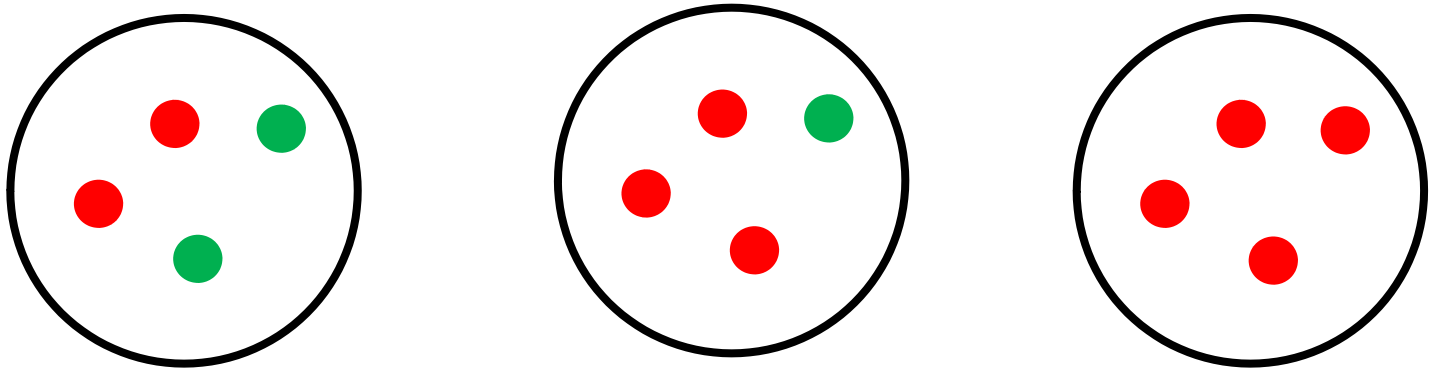
How to calculate the uncertainty of a dataset?

- Common method is the entropy (measure of uncertainty):

- Background:

  - Event X with n possible outcomes and probabilities

Example: 3 datasets with 4 elements



Formula fulfils 4 criteria:

- Uniform distributions have maximum uncertainty (same amount for each class)
- Uncertainty is additive for independent events
- Adding outcome with zero probability has no effect
- Continuous measure

Loss Function: Entropy Measures Uncertainty of a Dataset

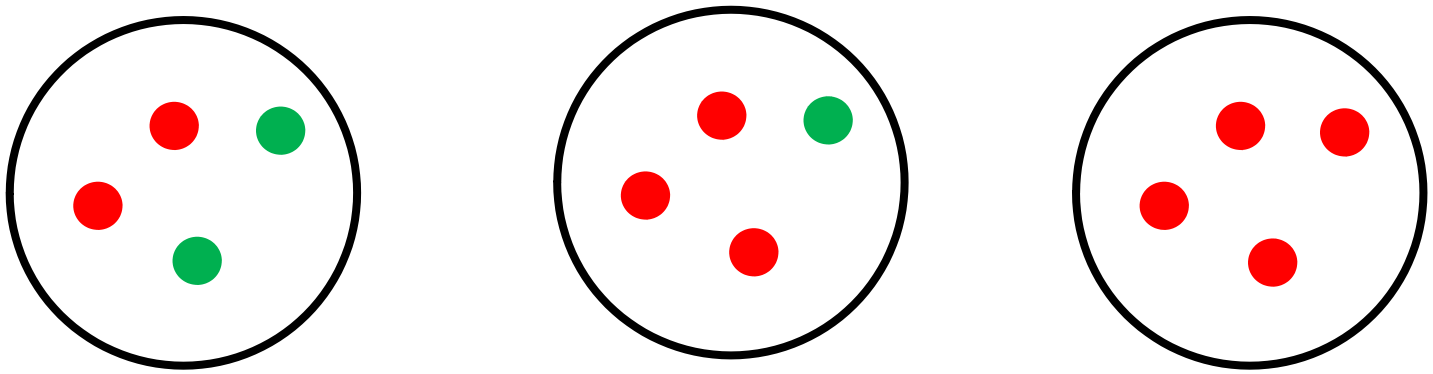
SIEMENS
Ingenuity for life

How to calculate the uncertainty of a dataset?

- Common method is the entropy (measure of uncertainty):

- Event X with n possible outcomes and probabilities

Example: 3 datasets with 4 elements



Loss Function: Entropy Measures Uncertainty of a Dataset

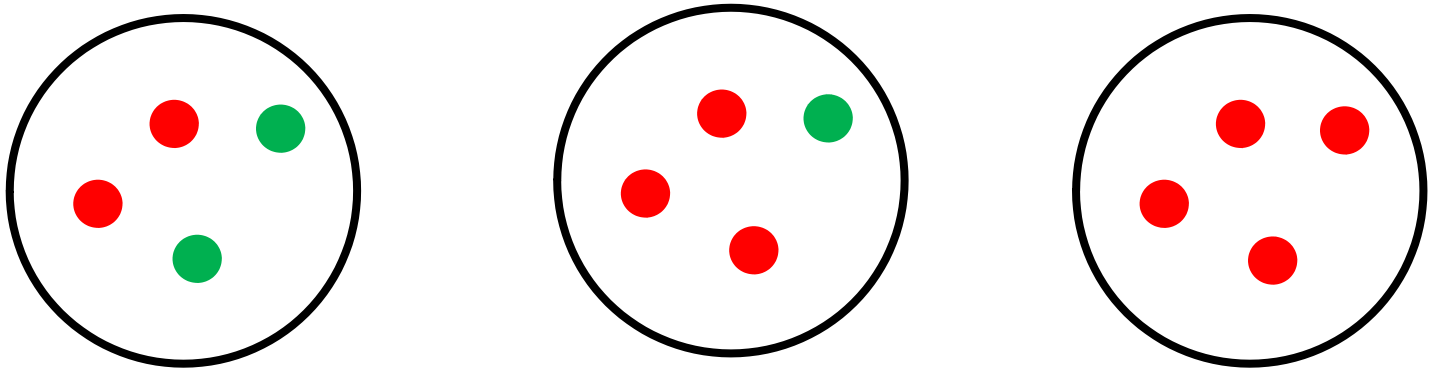
SIEMENS
Ingenuity for life

How to calculate the uncertainty of a dataset?

- Common method is the entropy (measure of uncertainty):

- Event X with n possible outcomes with probabilities , N classes

Example: 3 datasets with 4 elements



$$Entropy = 1$$

Loss Function: Entropy Measures Uncertainty of a Dataset

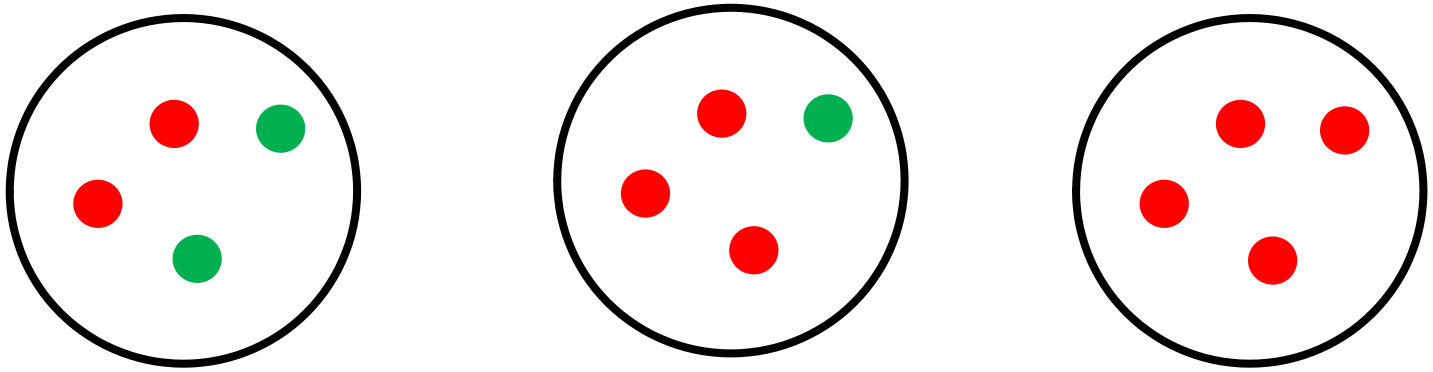
SIEMENS
Ingenuity for life

How to calculate the uncertainty of a dataset?

- Common method is the entropy (measure of uncertainty):

- Event X with n possible outcomes with probabilities , N classes

Example: 3 datasets with 4 elements



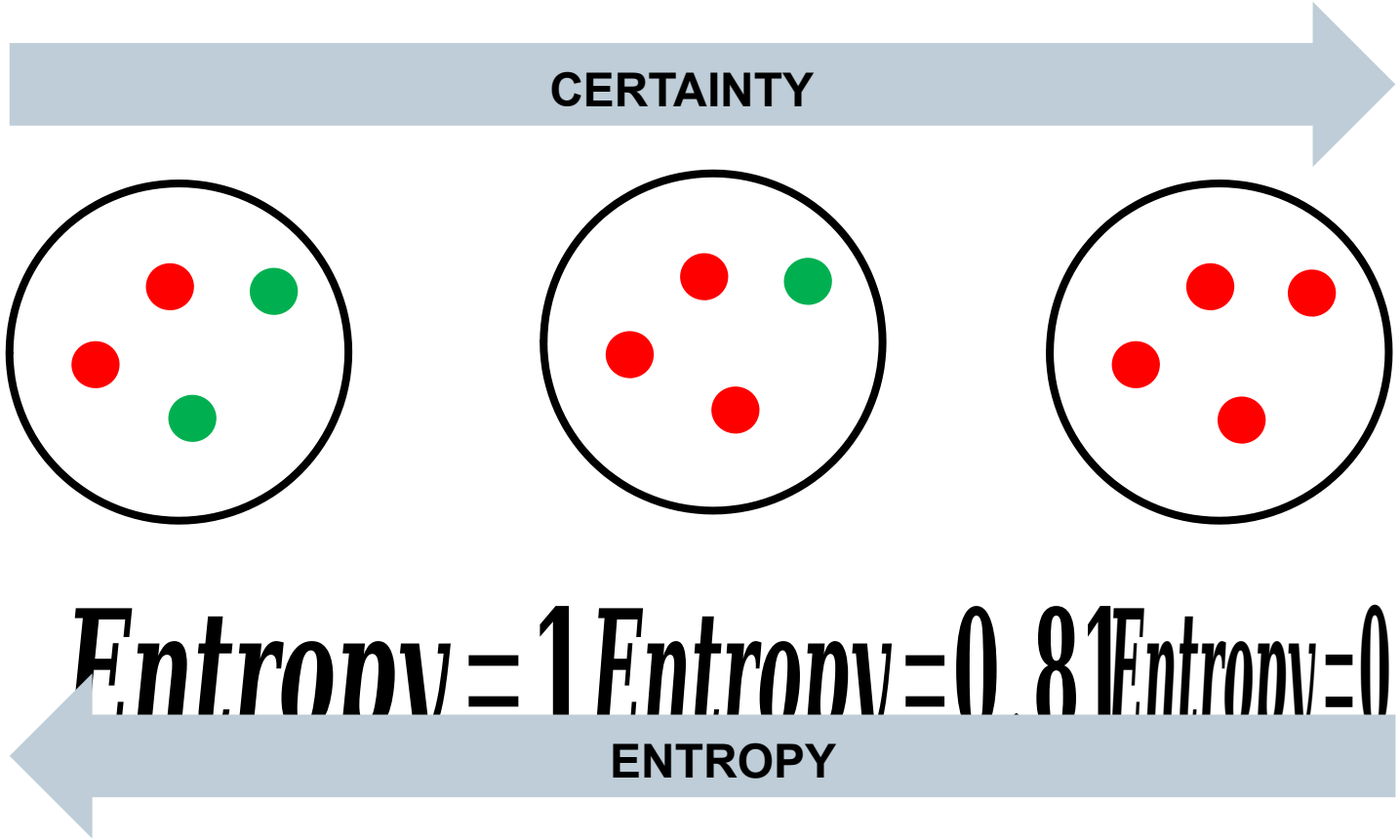
$$Entropy = 1 \quad Entropy = 0.81$$

Loss Function: Entropy Measures Uncertainty of a Dataset

SIEMENS
Ingenuity for life

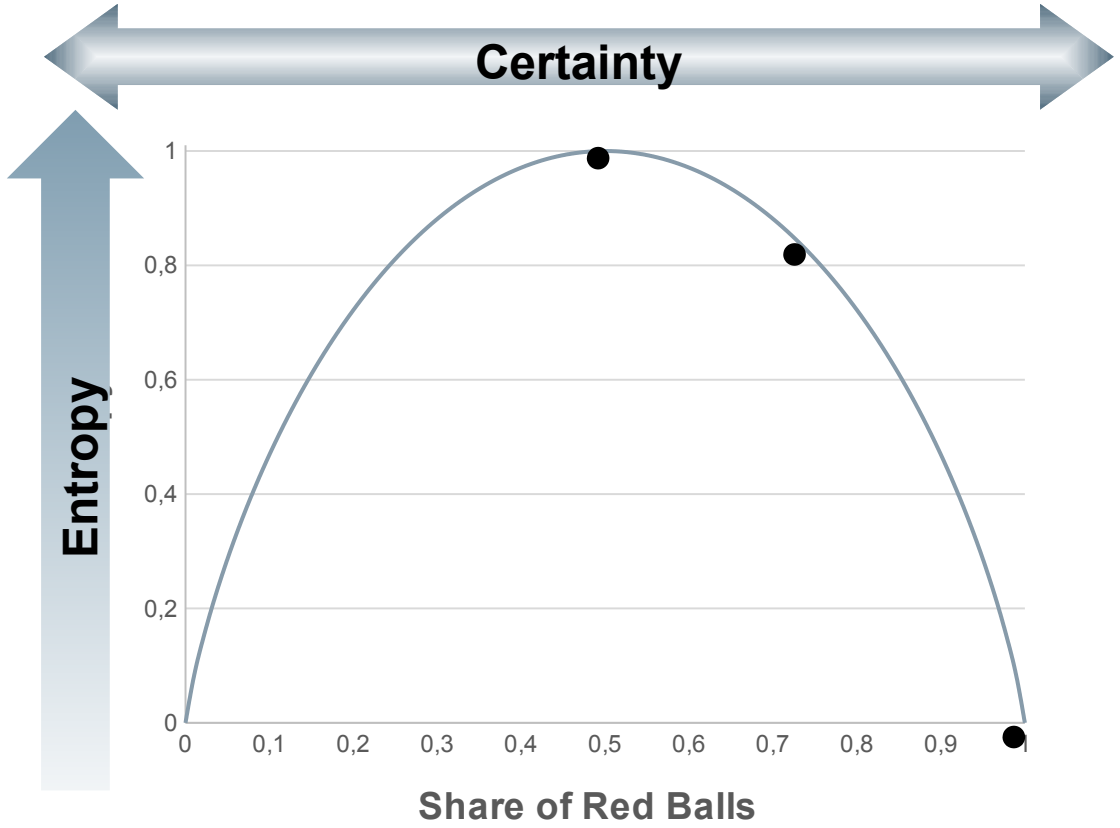
How to calculate the uncertainty of a dataset?

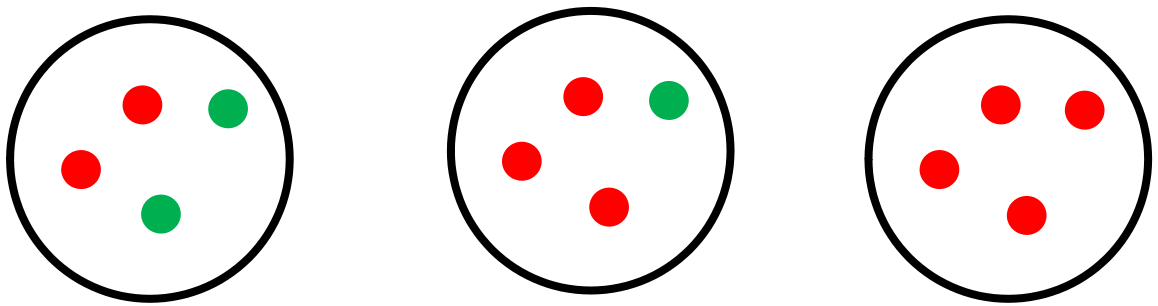
- Common method is the entropy:



With Decreasing Entropy the Certainty Increases.

Loss Function: Entropy Measures Uncertainty of a Dataset



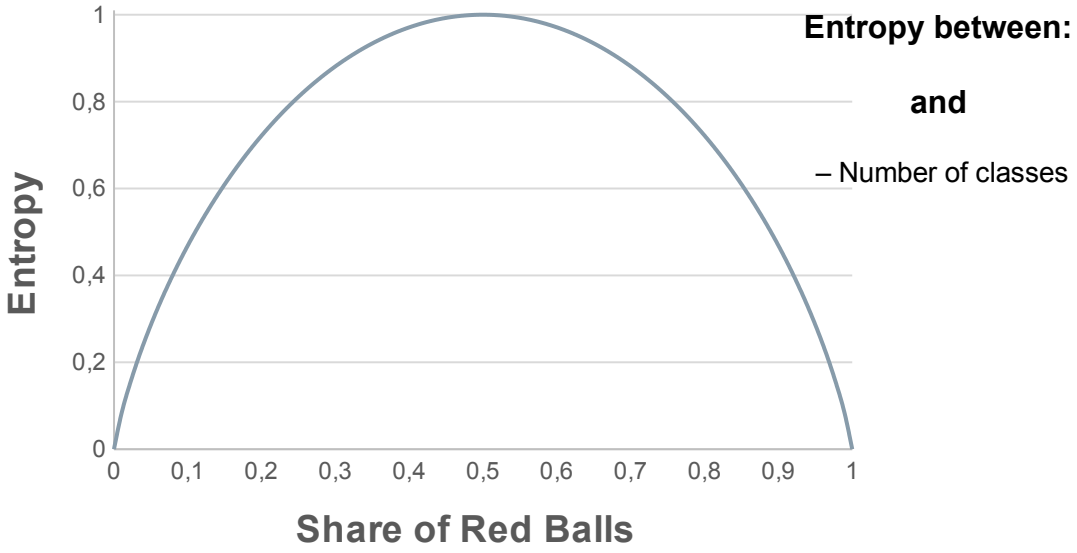


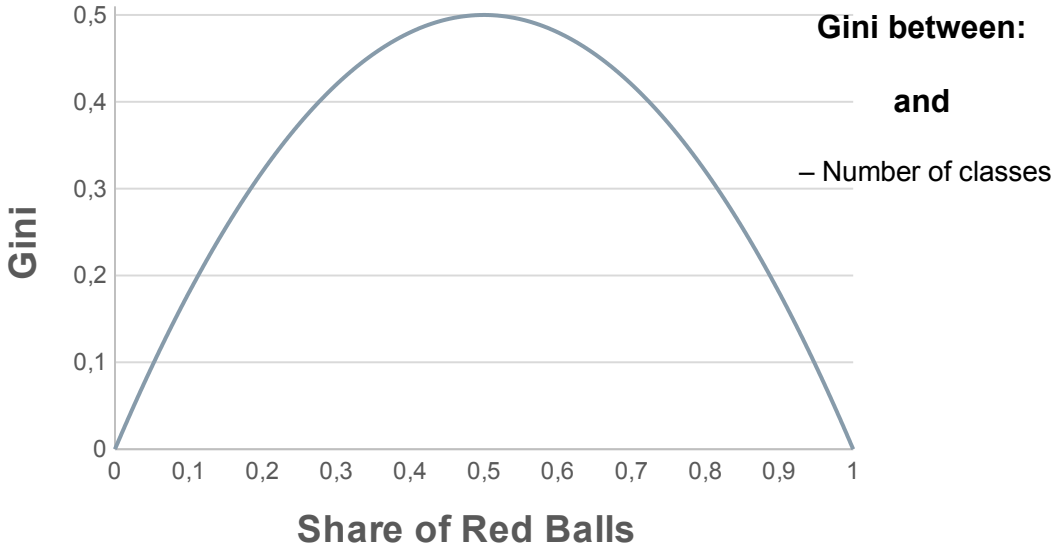
With Decreasing Entropy the Certainty Increases.

Gini-Index vs. Entropy to Measure Impurity of a Dataset

SIEMENS
Ingenuity for life

- Another way to calculate the purity of a dataset is the Gini-Index:
- The results are pretty much the same (if you apply a scaling factor)
- However, Gini needs less calculation time





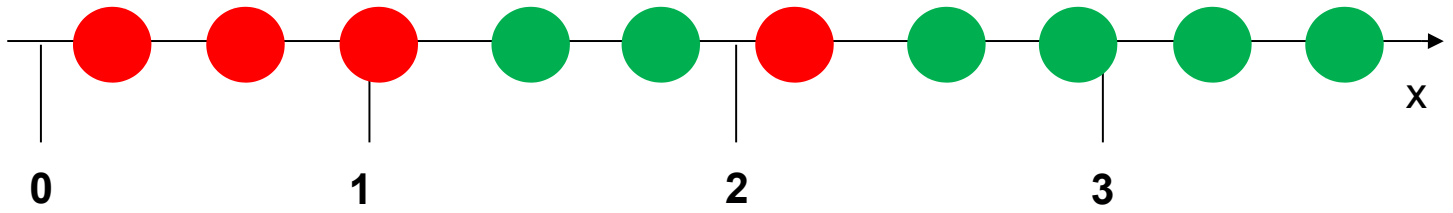
No Clear Preference Which Index to Use.

How to use the Entropy as a Loss Function? Information Gain (=Uncertainty reduction) as a Criteria for a Good Split

SIEMENS
Ingenuity for life

The bigger the gained information the better the split ☾ we want to maximize the gained information

Given are these balls measured on a x-scale.

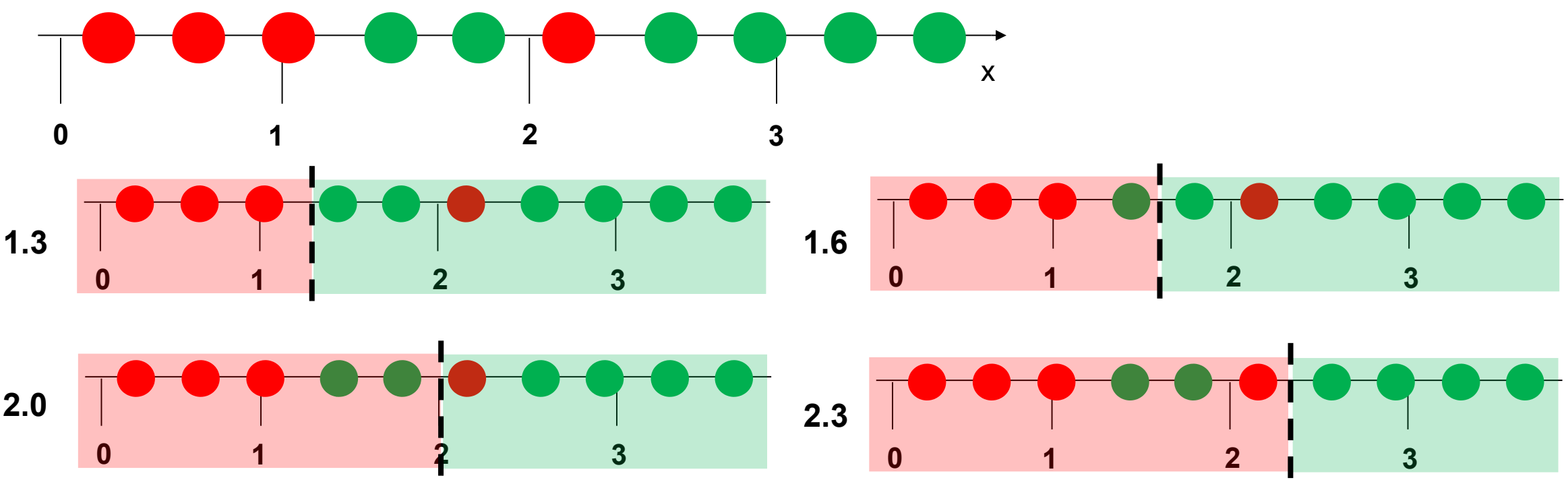


Information Gain as a Criteria for a Good Split

SIEMENS
Ingenuity for life

The bigger the gained information the better the split ☾ we want to maximize the gained information

What do you think is the best way to split these balls in two groups?

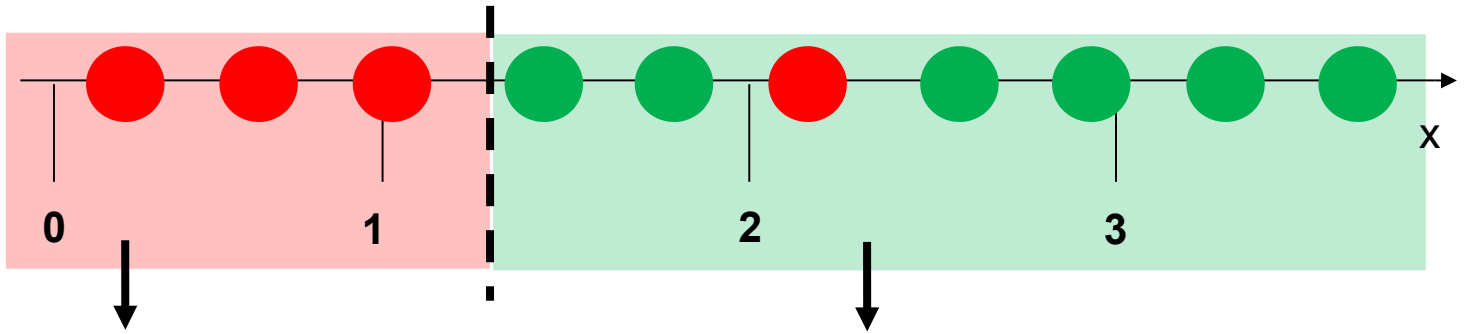


Information Gain as a Criteria for a Good Split

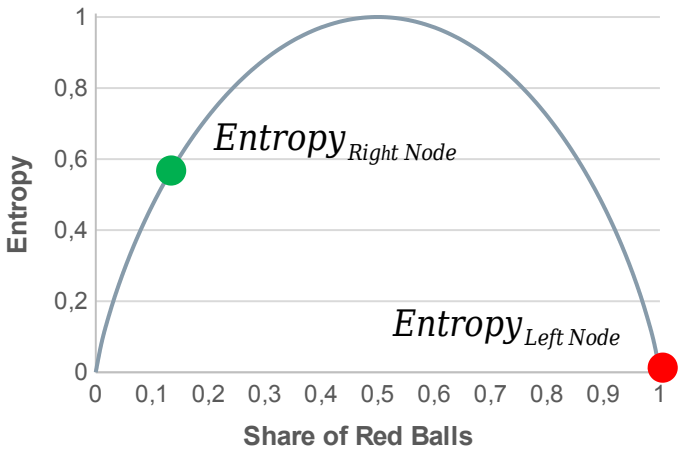
SIEMENS
Ingenuity for life

The bigger the gained information the better the split ☾ we want to maximize the gained information

What‘s the best way to split these balls in two groups?



Split at 1,3:

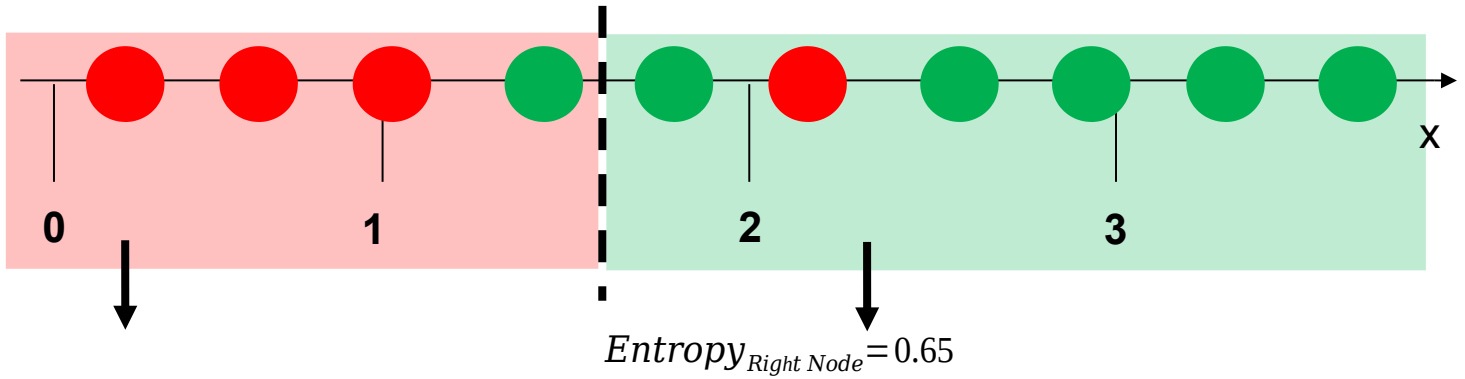


Information Gain as a Criteria for a Good Split

SIEMENS
Ingenuity for life

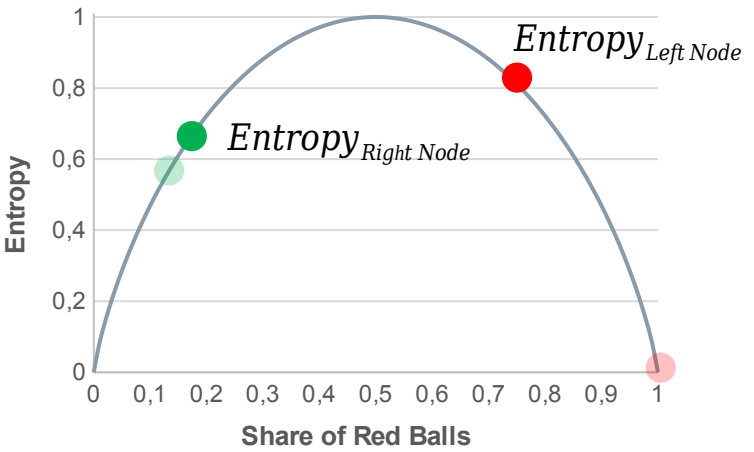
The bigger the gained information the better the split ☾ we want to maximize the gained information

What‘s the best way to split these balls in two groups?



Split at 1.3:

Split at 1.6:

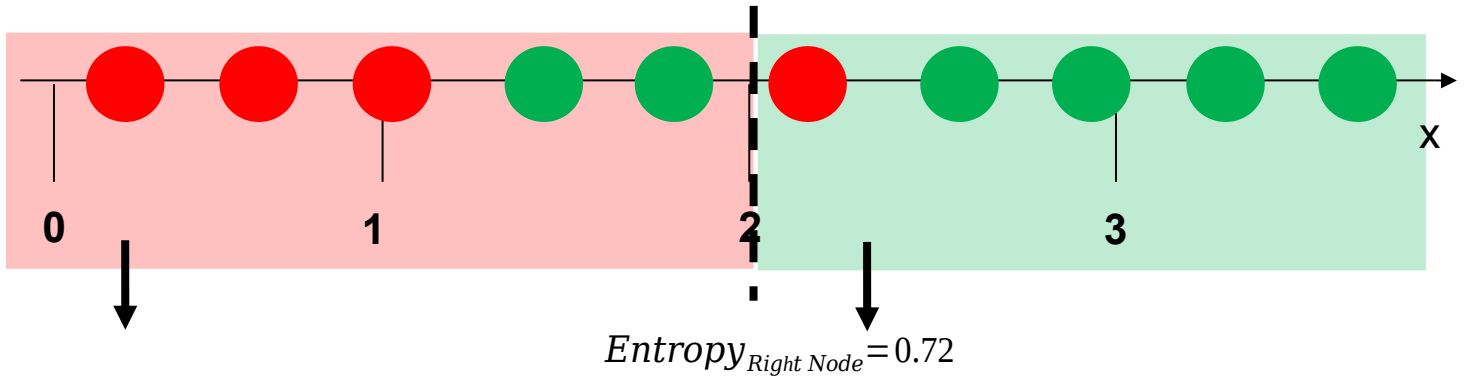


Information Gain as a Criteria for a Good Split

SIEMENS
Ingenuity for life

The bigger the gained information the better the split ☾ we want to maximize the gained information

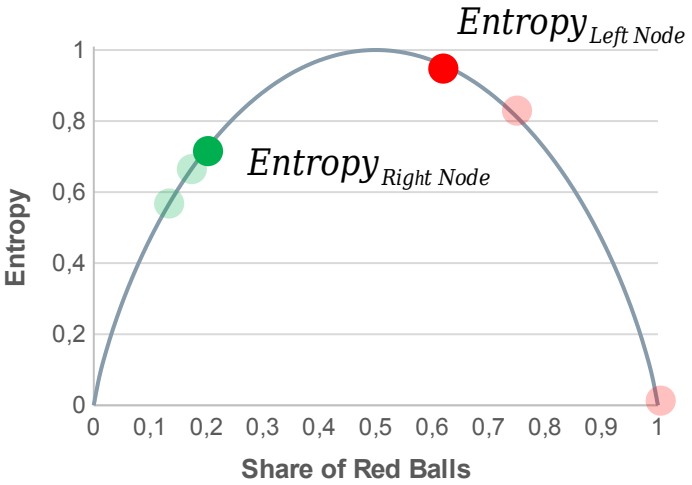
What‘s the best way to split these balls in two groups?



Split at 1.3:

Split at 1.6:

Split at 2.0:

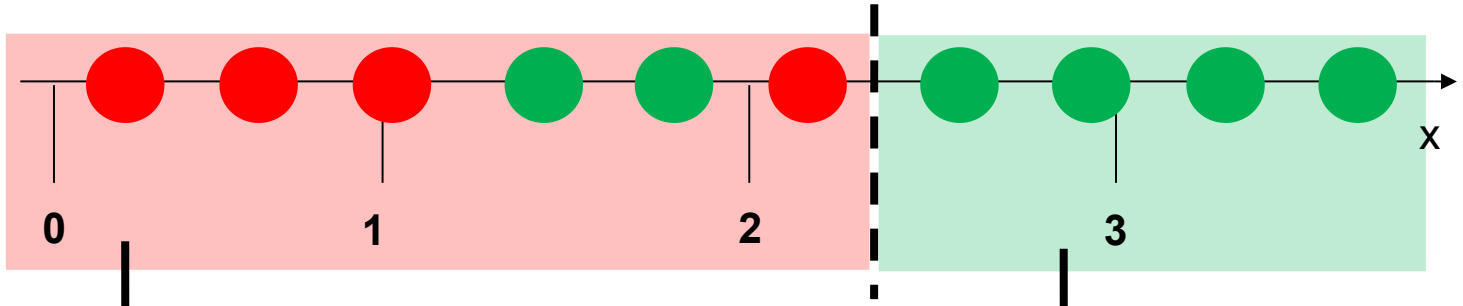


Information Gain as a Criteria for a Good Split

SIEMENS
Ingenuity for life

The bigger the gained information the better the split ☾ we want to maximize the gained information

What‘s the best way to split these balls in two groups?



$Entropy_{Left\ Node} = 0.91$

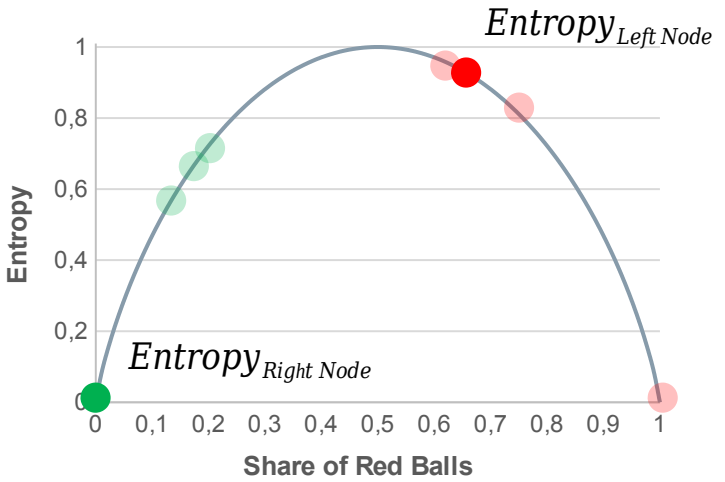
$Entropy_{Right\ Node} = 0$

Split at 1.3:

Split at 1.6:

Split at 2.0:

Split at 2.3:

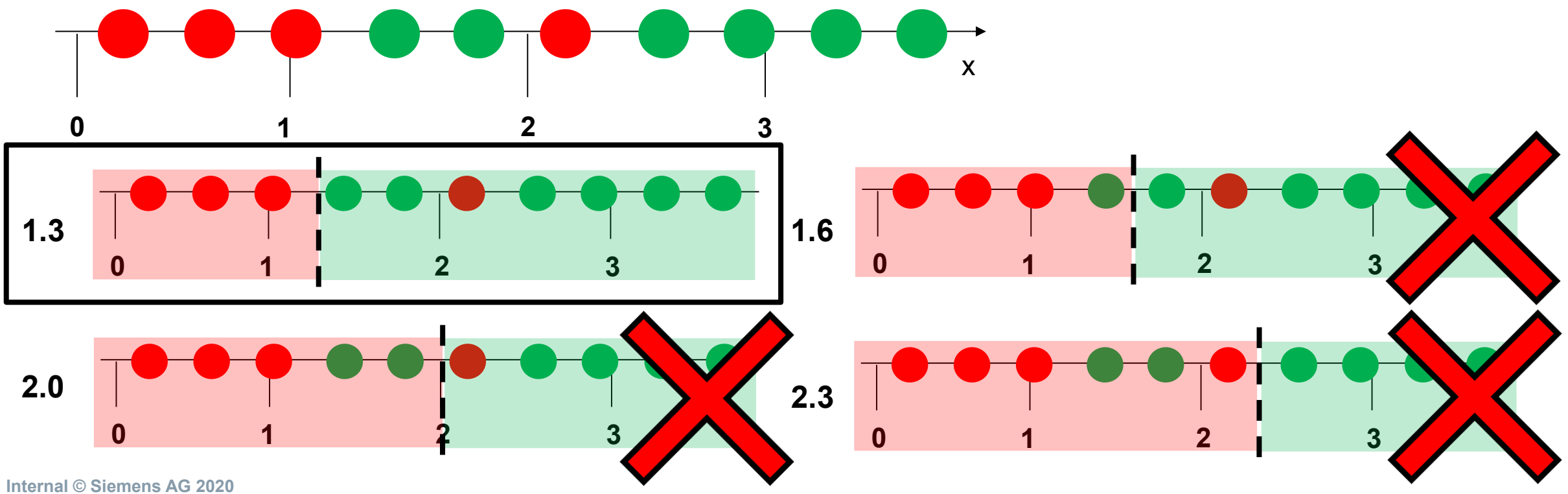


Information Gain as a Criteria for a Good Split (Loss Function)

SIEMENS
Ingenuity for life

The bigger the gained information the better the split ☾ we want to maximize the gained information

What‘s the best way to split these balls in two groups?



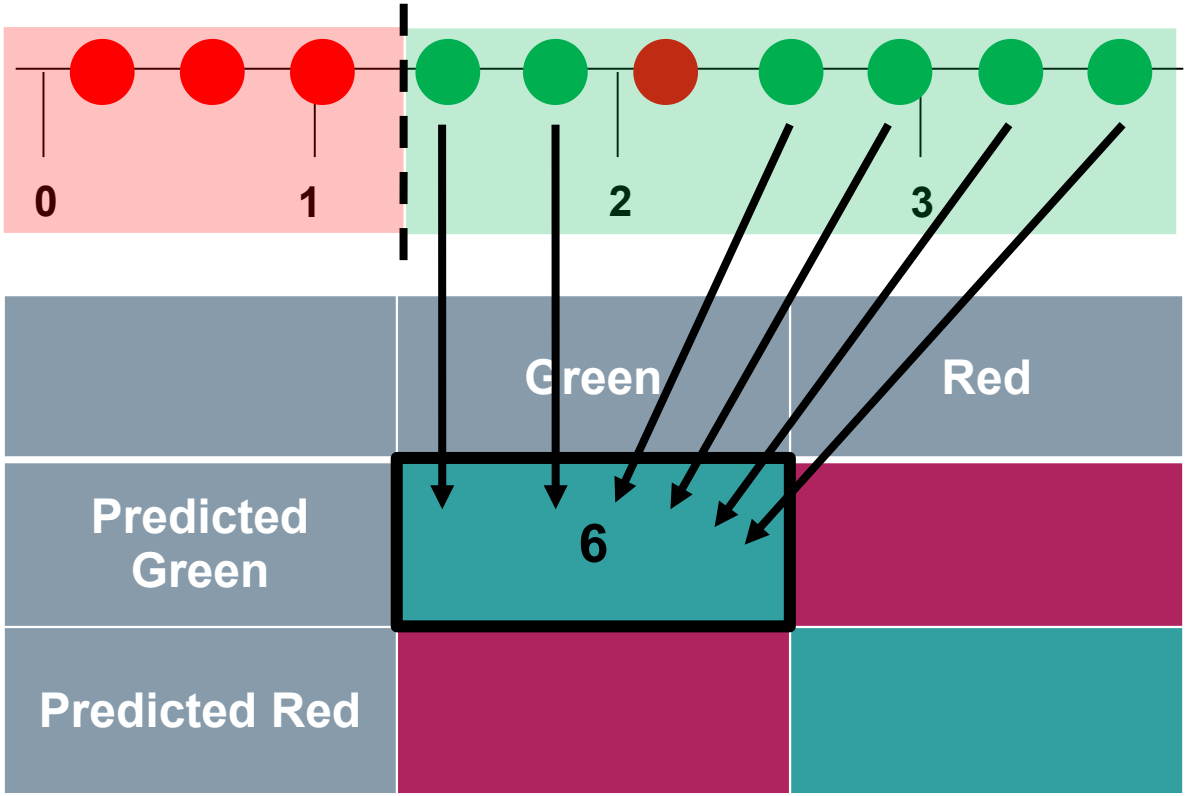
Towards an Error Metric for Classification: Confusion Matrix

SIEMENS
Ingenuity for life

	Reality	
	Positive	**Negative**
Predicted Positive	True Positive	**False Positive**
Predicted Negative	**False Negative**	True Negative

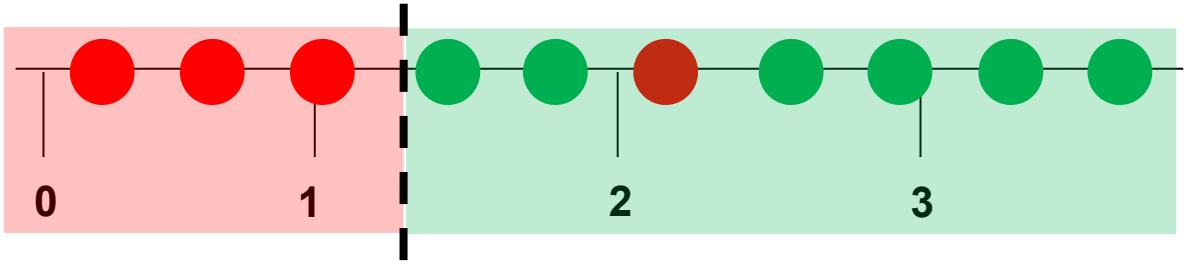
Towards an Error Metric for Classification: Confusion Matrix

SIEMENS
Ingenuity for life



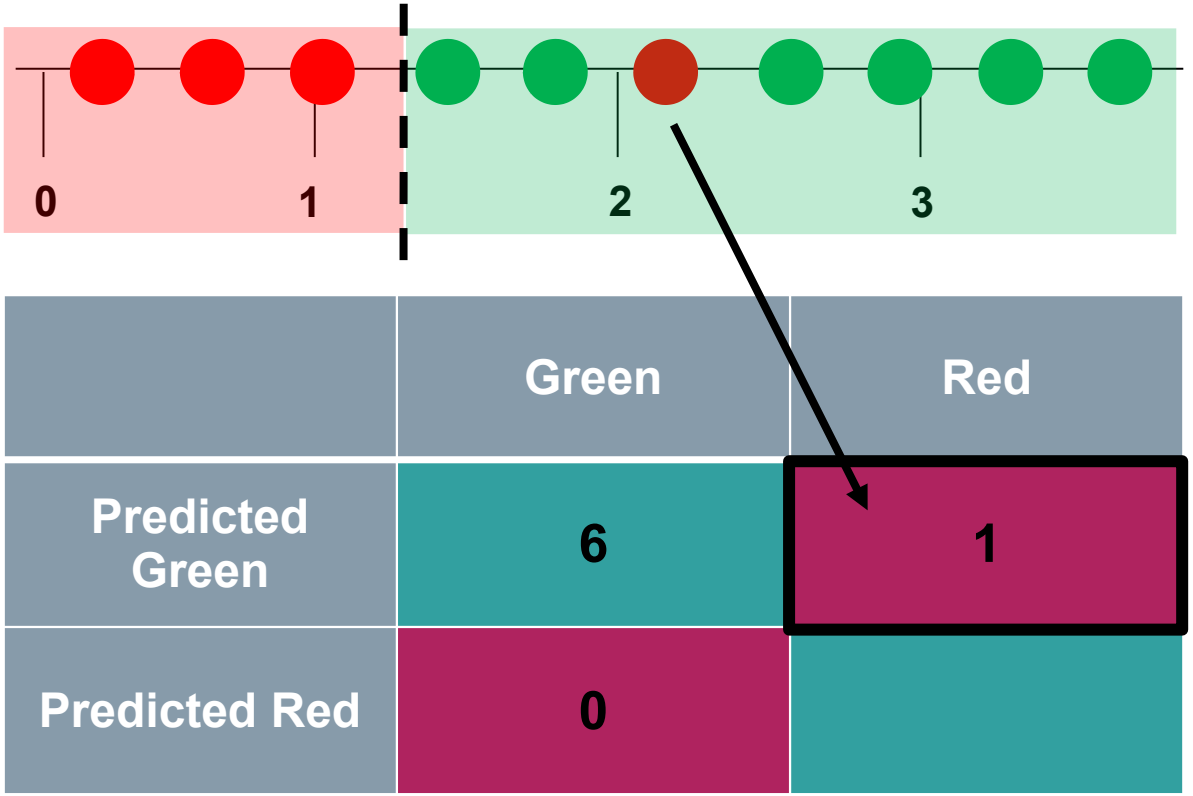
Towards an Error Metric for Classification: Confusion Matrix

SIEMENS
Ingenuity for life

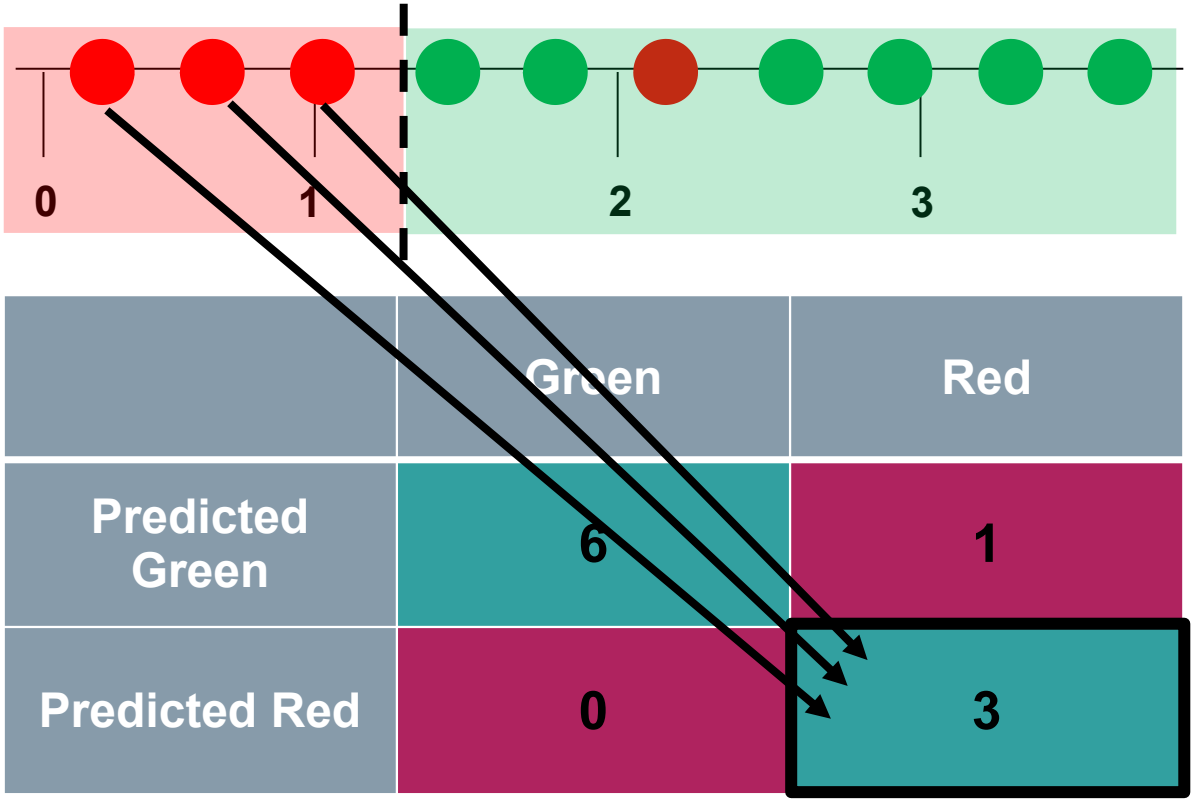


	Green	Red
Predicted Green	6	
Predicted Red	0	

Towards an Error Metric for Classification: Confusion Matrix

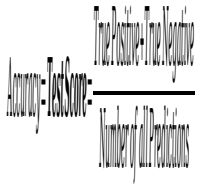


Towards an Error Metric for Classification: Confusion Matrix

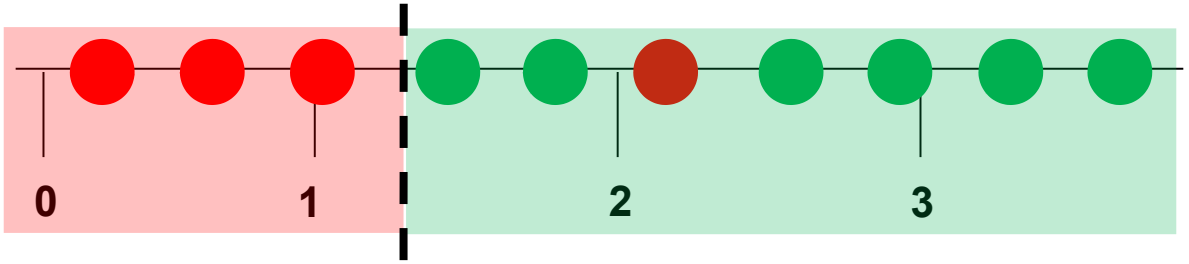


Error Metric: Accuracy

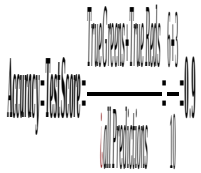
	Reality	
	Positive	**Negative**
Predicted Positive	True Positive	**False Positive**
Predicted Negative	**False Negative**	True Negative



Error Metric: Accuracy



	Green	Red
Predicted Green	6	1
Predicted Red	0	3



Problems with Accuracy

SIEMENS
Ingenuity for life

Example:

Classifier: medical data, predict if person has disease or not; highly imbalanced classes

Result: high accuracy, BUT :

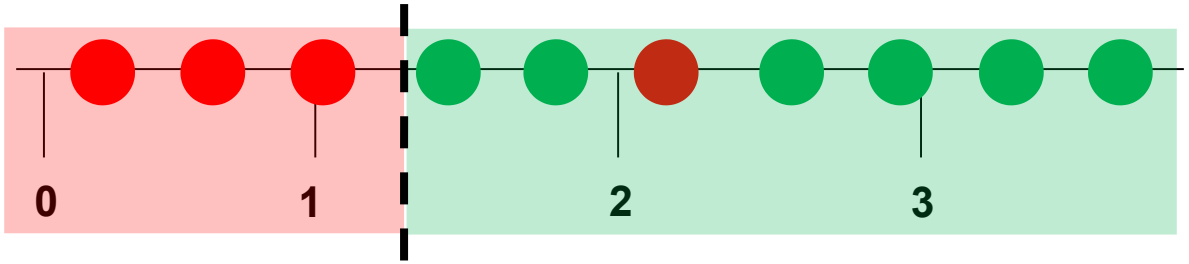
- Always correctly diagnosed disease, but it gets half of the healthy people wrong. (precision)
- Right for every healthy person, but misses half of the disease cases. (recall)

Accuracy doesn't help. What's important is that no matter if someone is sick or healthy the classifier gets it right.

Let's say you are screening a population for a sickness that occurs only 1 in a 1 000 cases. Would a classifier that is 99.9% accurate seems good ? Well intuitively yes, but the classifier could just predict every person is healthy and get it right 999 out of 1000 times, so it would be 99.9% accurate. Huh. Not good.

Error Metric: Precision

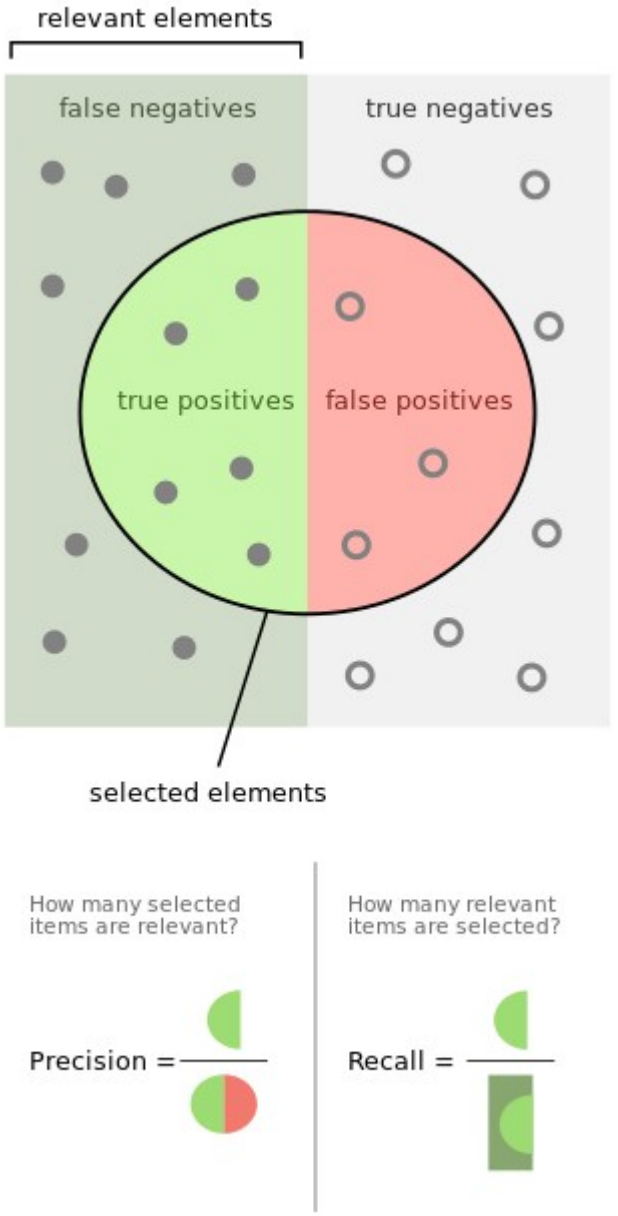
How many positive/green predictions are actual positive/true greens?



	Reality	
	Positive	**Negative**
Predicted Positive	True Positive 6	**False Positive 1**
Predicted Negative	**False Negative 0**	True Negative 3

$$\text{Precision} = \frac{\text{True Positive}}{\text{Number of all Positive Predictions}}$$

$$\text{Precision} = \frac{\text{True Greens}}{\text{all Green Predictions}}$$

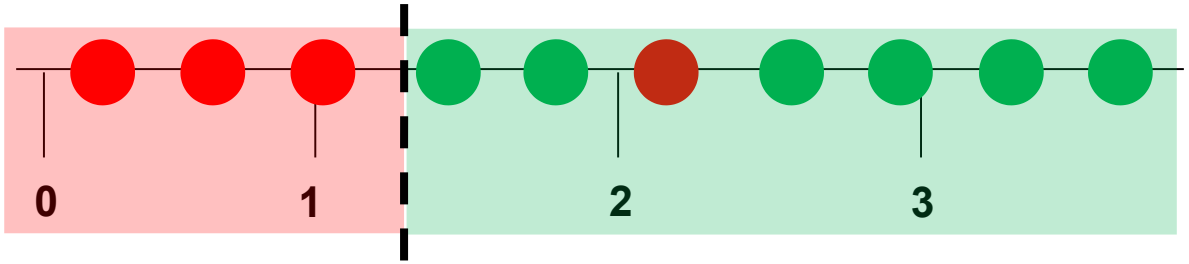


How valid are the search results?

Use precision if predicting right is important (have all patients diagnosed by the classifier really had the disease?)

Error Metric: Recall

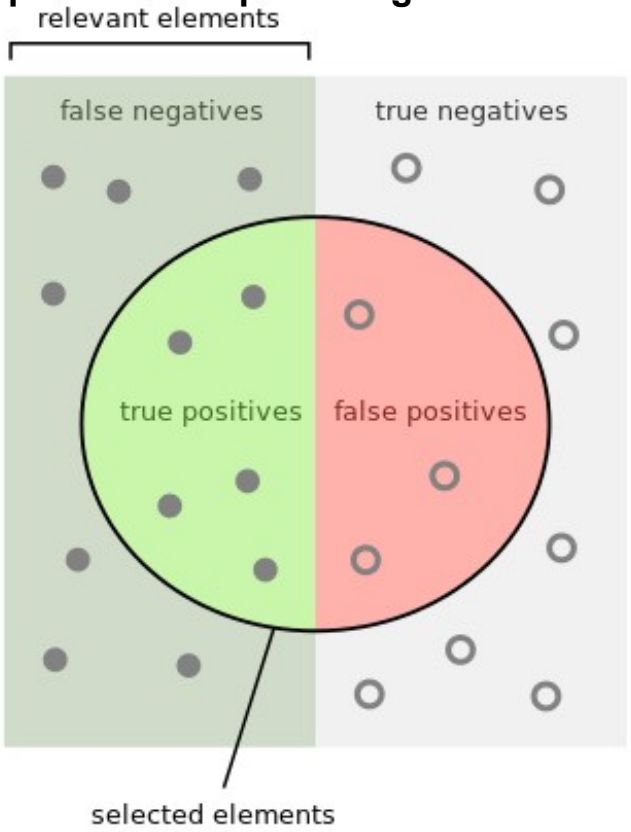
How many positives/greens are predicted as positive/green?

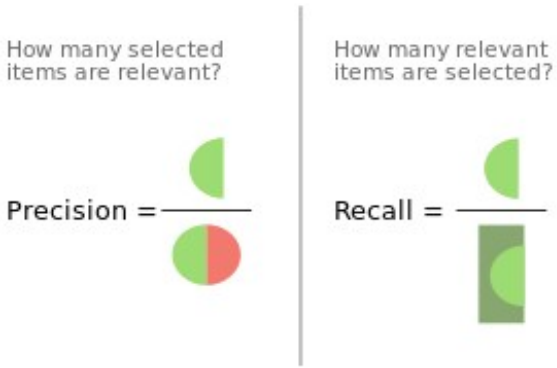


	Reality: Positive	Reality: Negative
Predicted Positive	True Positive 6	**False Positive 1**
Predicted Negative	**False Negative 0**	True Negative 3

$$\text{Recall} = \frac{\text{True Positive}}{\text{Number of all Positive Elements}}$$

$$\text{Recall} = \frac{\text{True Greens}}{\text{all Green Elements}} = \frac{6}{6} = 1$$





SIEMENS
Ingenuity for life

How complete are the results?

Use recall if finding all is important (are all patients with the disease detected?)

Error Metric: F1 Score

	Reality	
	Positive	**Negative**
Predicted Positive	True Positive	**False Positive**
Predicted Negative	**False Negative**	True Negative

$$\text{Recall} = \frac{\text{True Positive}}{\text{Number of all Positive Elements}}$$

How many positive are positive predicted?

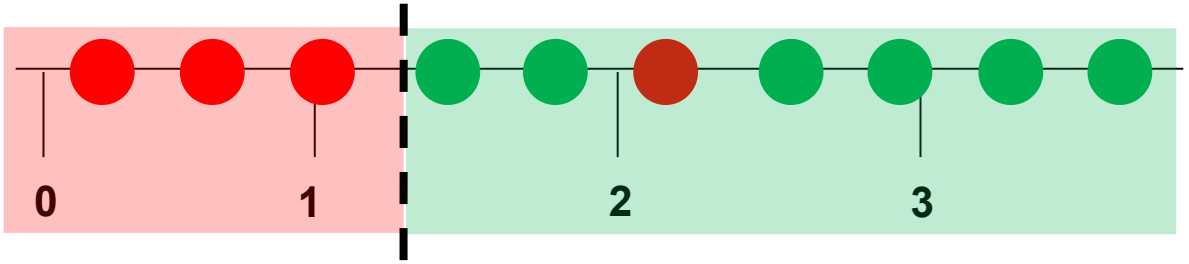
$$\text{Precision} = \frac{\text{True Positive}}{\text{Number of all Positive Predictions}}$$

How many positive predictions are actual positive?

$$F1\,\text{Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Tradeoff of Precision and Recall.
Harmonic mean of precision and recall

Error Metric: F1 Score



	Green	Red
Predicted Green	6	1
Predicted Red	0	3

Tradeoff of precision and recall.

$$F1score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} = 2 \cdot \frac{1 \cdot 0.86}{1 + 0.86} = 0.92$$

Error Metrics: Accuracy, Precision, Recall and F1 Score

	Reality	
	Positive	**Negative**
Predicted Positive	True Positive	**False Positive**
Predicted Negative	**False Negative**	True Negative

$$Precision = \frac{True\ Positive}{Number\ of\ all\ Positive\ Predictions}$$

How many positive predictions are actual positives?

$$Recall = \frac{True\ Positive}{Number\ of\ all\ Positive\ Elements}$$

How many positive are positive predicted?

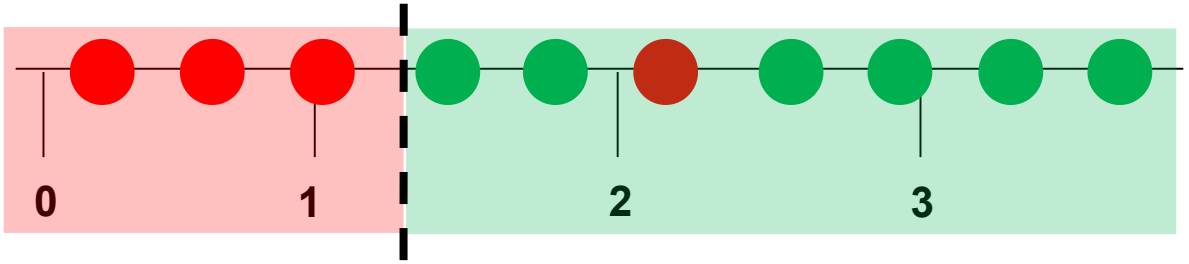
$$F1\ Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Tradeoff of precision and recall.

$$Accuracy = F1\ Score = \frac{True\ Positive + True\ Negative}{Number\ of\ all\ Predictions}$$

https://towardsdatascience.com/should-i-look-at-precision-recall-or-specificity-sensitivity-3946158aace1

Error Metrics: Accuracy, Precision, Recall and F1 Score



	Green	Red
Predicted Green	6	1
Predicted Red	0	3

How many green predictions are actual green?

$$Precision = \frac{TrueGreens}{allGreenPredictions} = \frac{6}{7} = 0.86$$

How many greens are green predicted?

$$Recall = \frac{TrueGreens}{allGreenElements} = \frac{6}{6} = 1$$

Tradeoff of precision and recall.

$$F1Score = 2 * \frac{Precision * Recall}{Precision + Recall} = 2 * \frac{1 * 0.86}{1 + 0.86} = 0.92$$

$$Accuracy = TASScore = \frac{TrueGreens + TrueReds}{allPredictions} = \frac{6+3}{10} = 0.9$$

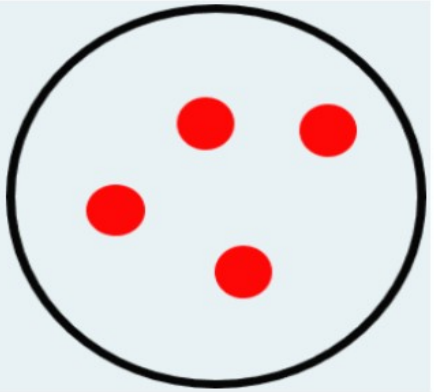
Exercise 1.1: Quiz

SIEMENS
Ingenuity for life

https://forms.office.com/r/yGdHArDsHu

1

What's the entropy of this data set? *



- 0
- 0.5
- 1

2

An gini of zero means *

- The sample set is maximal pure
- The sample set is maximal impure

3

The bigger the gained information *

- the better the split
- the worser the split

4

Precision is *

- the ratio between true "positives" and the number of all "positive" elements
- the ratio between true "positives" and the number of all "positive" predictions
- the ration between true "positives" and "negatives" and the number of all predictions

https://forms.office.com/Pages/DesignPage.aspx#

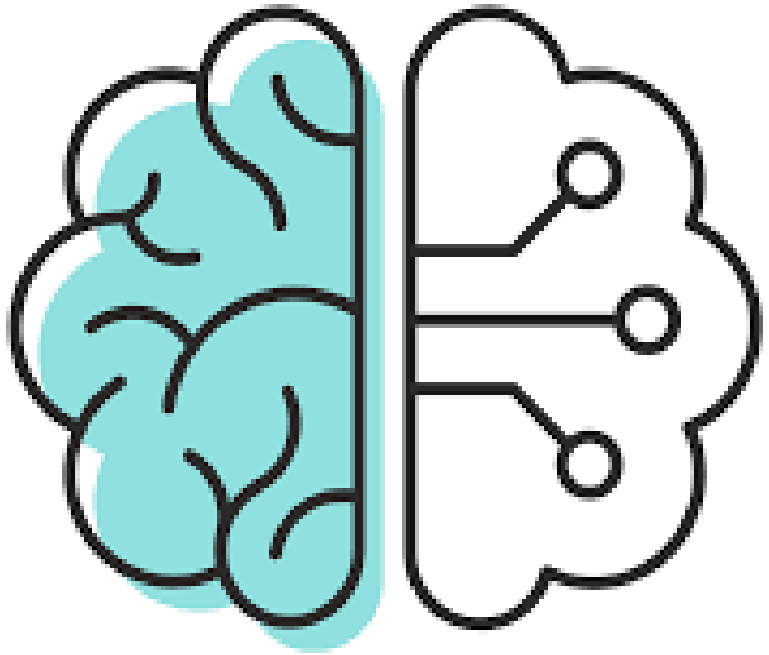
Overview: what we achieved so far

SIEMENS
Ingenuity for life

	Target values	Loss function	Error metric	Act function last layer	Act func hidden layers
Regression	Continuous	MSE, R^2, MAE,…	MSE, R^2, MAE,…		
Classification	Binary	Binary cross entropy	F1 score, precision, recall, ROC-curve		
	multiple classes	categorical cross entropy	F1 score, precision, recall, ROC-curve		

	Decision Trees	Artificial Neural Nets
Tabular data		
Text		
Images		

Agenda



- Review ML1
- Model Evaluation:
  - Regression Quality Metrics
  - Classification Quality Metrics
  - The Bias-Variance Trade-Off
- Model Types:
  - Decision Trees (Ensemble: Random Forrest)
  - Neural Networks
    - Basic Steps of Training a Neural Network
    - Neural Network Exercise
- Application of Machine Learning in Our Business

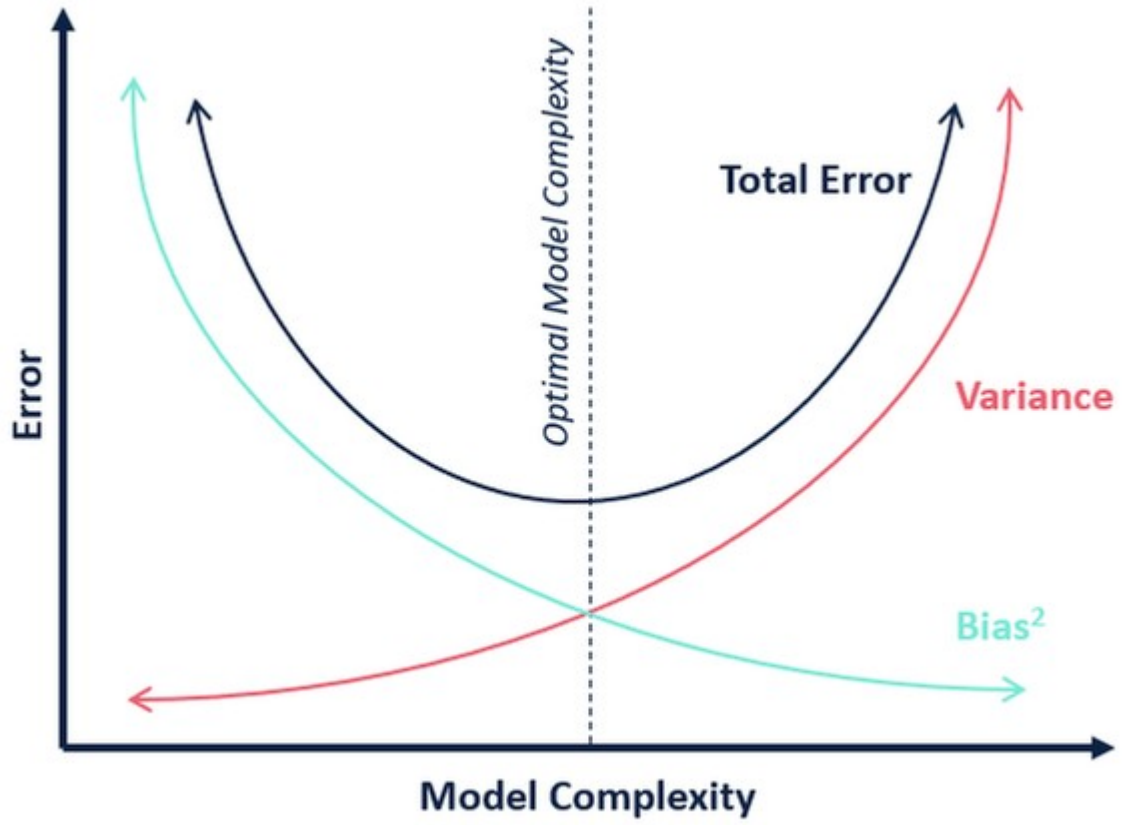
Limitations for generalization of Model (known and unknown data) Overview (New Compared to ML1 in Red)

SIEMENS
Ingenuity for life

	Error Type	Summary	Details	Consequences	How to prevent it
Model Error	**Bias Error/ Underfitting**	Model is too simple	The (unknown) true target function is more complex than the model.	• High forecasting error both in known and unknown data • Model doesn't vary much when adding new data points	• Review with domain experts • Analyze residuals • Increase complexity
	Variance Error/ Overfitting	Modell is too complex	The true target function is simpler than the model. Common reason: model tries to overcome irreducible error/ noise.	• High forecasting error in unknown data, not in known data • Model varies a lot when adding new data points	• Review with domain experts • Split data in training and test data • k-fold cross validation
	Irreducible Error	Data are not sufficient/ Noise	Factors which were not measured have an impact: • not measured input variables • environmental factors • measurement errors • sampling bias, …	• Model varies not a lot when adding new data points	Can not be reduced by change of algorithm! • Review with domain experts • Collect better data

Any built model build is an approximation of a true target function and therefore includes errors.

Bias-Variance Trade-Off

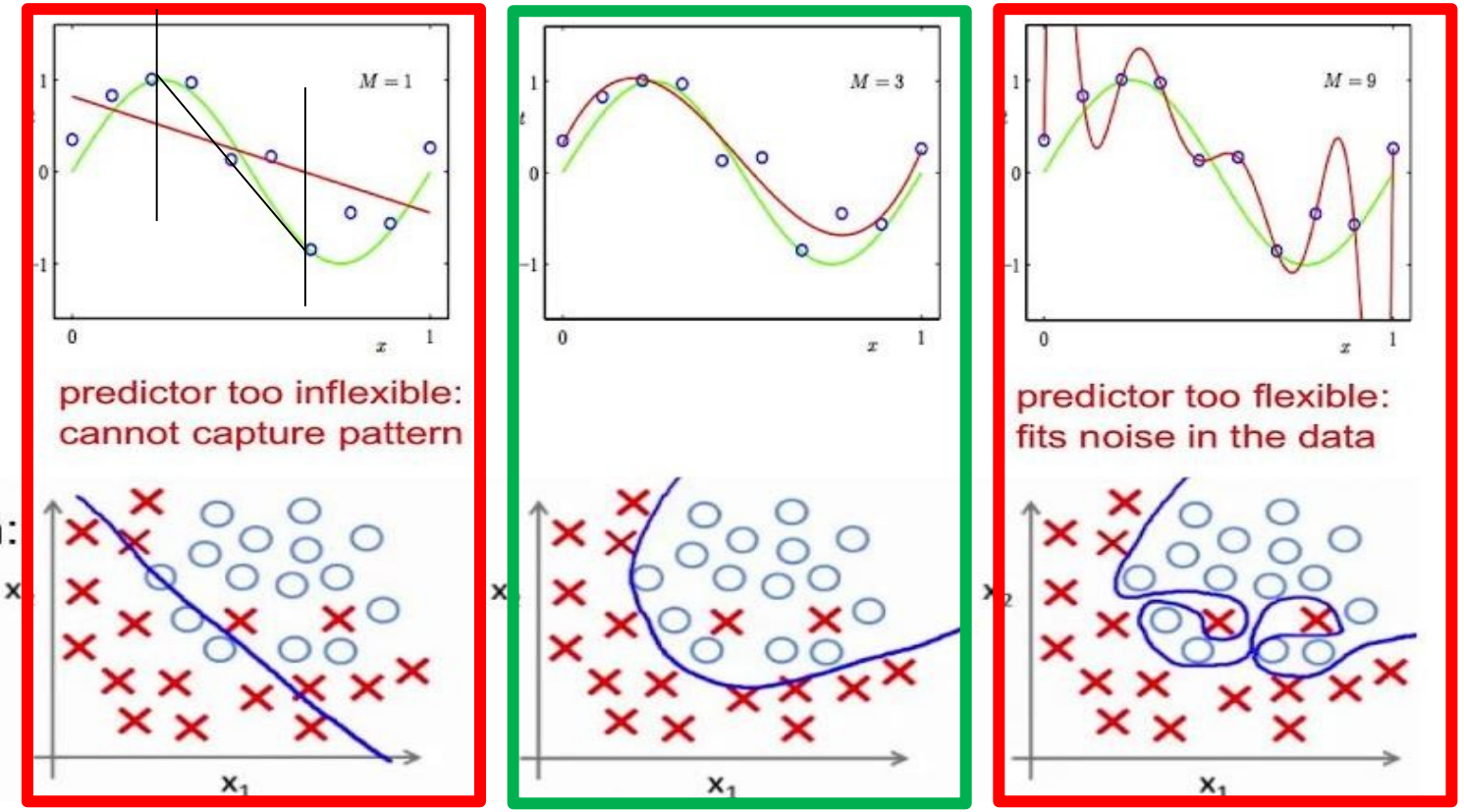


Occam‘s razor: The simplest model/most intuitive method is preferable

Model Selection: Under- and Overfitting for Numerical and Categorical Target Variables

SIEMENS
Ingenuity for life

Regression:



True, unknown target function

Model target function

Classification:

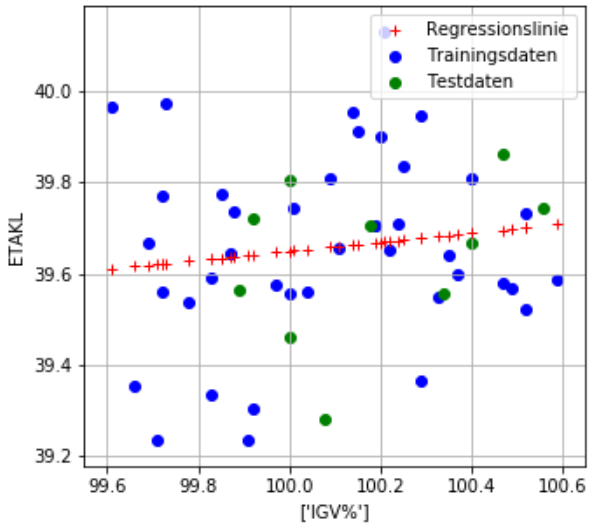
Variance/ Overfitting

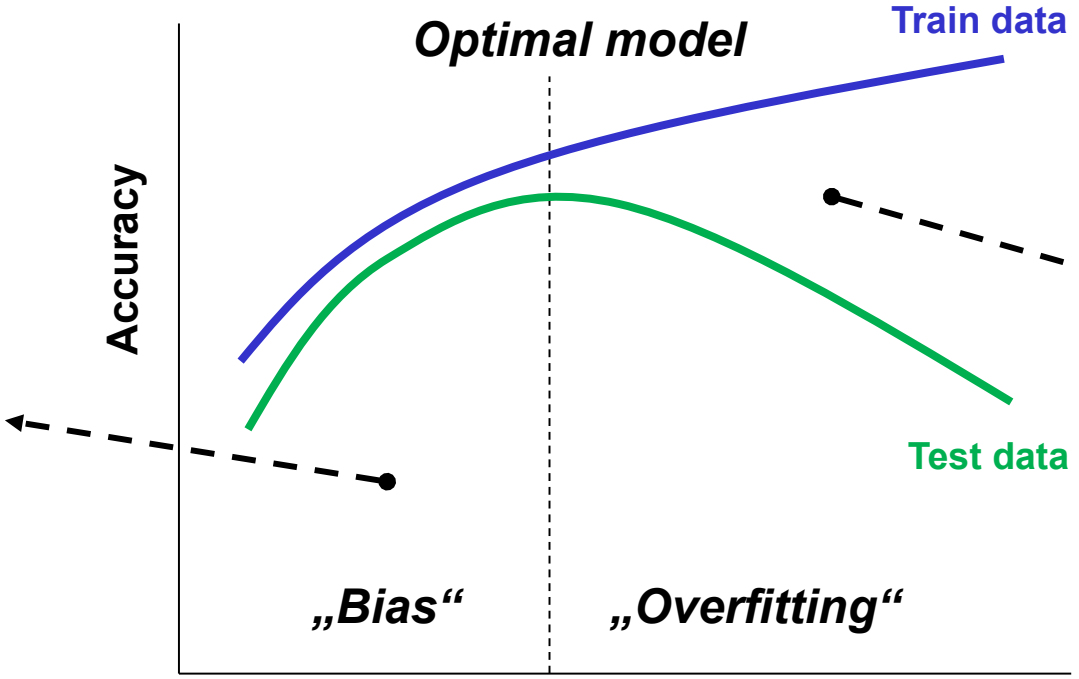
Bias / underfitting

How to Prevent Overfitting 1: Split Dataset into Training and Test Data

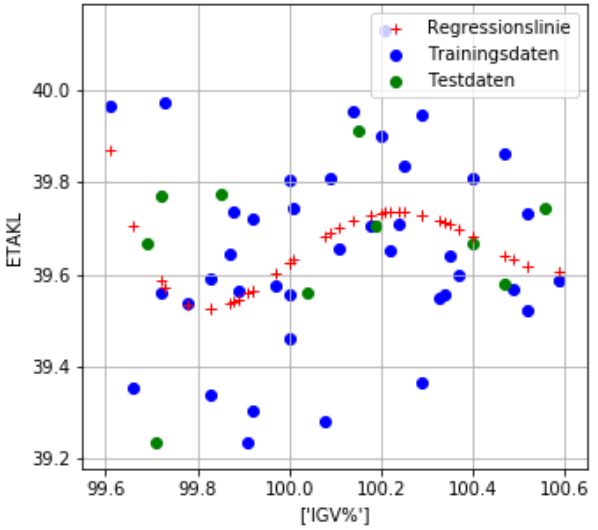
SIEMENS
Ingenuity for life

Polynomial 1st degree





Polynomial 4th degree



Model complexity
E.g. the number of predictors
or the polynomial degree
(both resulting in a larger number of coefficients)

How to Prevent Overfitting 2: k-Fold Cross Validation

SIEMENS
Ingenuity for life

- Used if number of training data is limited
- Parameter k: data is split into k subgroups
  - i.e. k=10 data is split into 10 groups.
- Generally model evaluation is less biased than with simple train/test split

General procedure :

1. Shuffle the dataset randomly.
2. Split the dataset into k groups
3. For each unique group:
   1. Take the group as a hold out or test data set
   2. Take the remaining groups as a training data set
   3. Fit a model on the training set and evaluate it on the test set
   4. Retain the evaluation score and discard the model
4. Summarize the skill of the model using the sample of model evaluation scores

k subgroups -> k models: one subgroup is test dataset, rest is training data set.

Average the results.

Source: https://machinelearningmastery.com/k-fold-cross-validation/

Exercise 1.2: Quiz

SIEMENS
Ingenuity for life

https://forms.office.com/r/d0fArXiVtm

1. Which regression error measure is INDEPENDEND of the measured scale (mm, cm, inch...)? *

- ◯ MSE
- ◯ R-SQ

2. Which is the "best" R-SQ-Value? *

- ◯ 0%
- ◯ 50%
- ◯ 100%

3. When you increase the complexity of a model, can you improve Variance and Bias at the same time? *

- ◯ No
- ◯ Yes
- ◯ Yes, always

4. Does the variance drops when the bias grows? *

- ◯ No
- ◯ Yes

5. Wich statement is correct? *

- ◯ Underfitting means the true target function is simpler than the model.
- ◯ Bias error vary much with changing training data.
- ◯ Irreducible error can not be reduced with different algorithms.

https://forms.office.com/Pages/DesignPage.aspx#

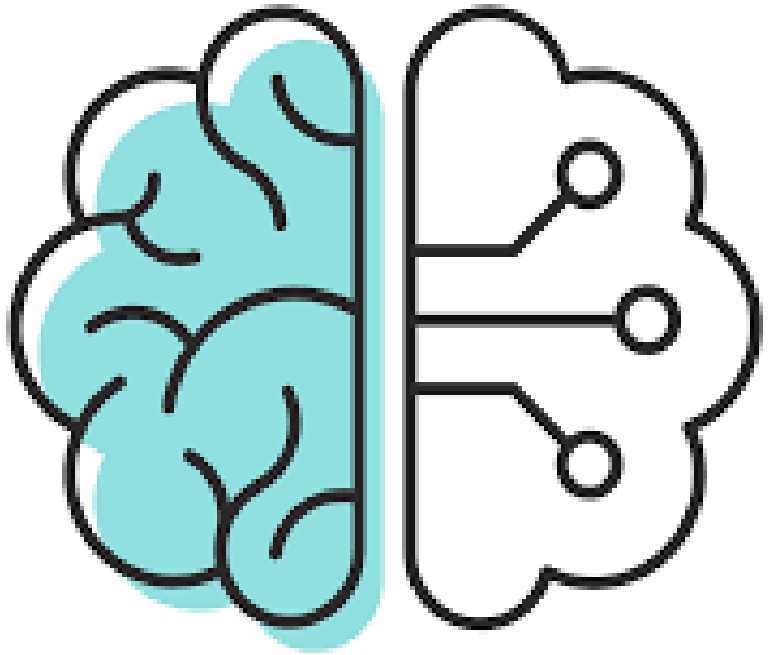
Overview: what we achieved so far

SIEMENS
Ingenuity for life

	Target values	Loss function	Error metric	Act function last layer	Act func hidden layers
Regression	Continuous	MSE, R^2, MAE,…	MSE, R^2, MAE,…		
Classification	Binary	Binary cross entropy	F1 score, precision, recall, ROC-curve		
	multiple classes	categorical cross entropy	F1 score, precision, recall, ROC-curve		

	Decision Trees	Artificial Neural Nets
Tabular data		
Text		
Images		

Agenda



- Review ML1
- Model Evaluation:
  - Regression Quality Metrics
  - Classification Quality Metrics
  - The Bias-Variance Trade-Off
- Model Types:
  - Decision Trees (Ensemble: Random Forrest)
  - Neural Networks
    - Basic Steps of Training a Neural Network
    - Neural Network Exercise
- Application of Machine Learning in Our Business

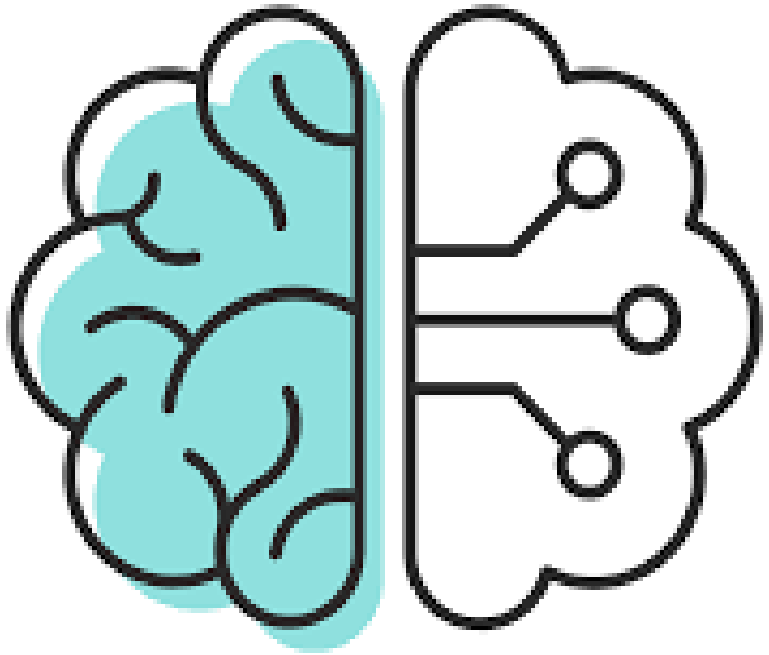
Overview: what comes next

SIEMENS
Ingenuity for life

	Target values	Loss function	Error metric	Act function last layer	Act func hidden layers
Regression	Continuous	MSE, R^2, MAE,…	MSE, R^2, MAE,…		
Classification	Binary	Binary cross entropy	F1 score, precision, recall, ROC-curve		
	multiple classes	categorical cross entropy	F1 score, precision, recall, ROC-curve		

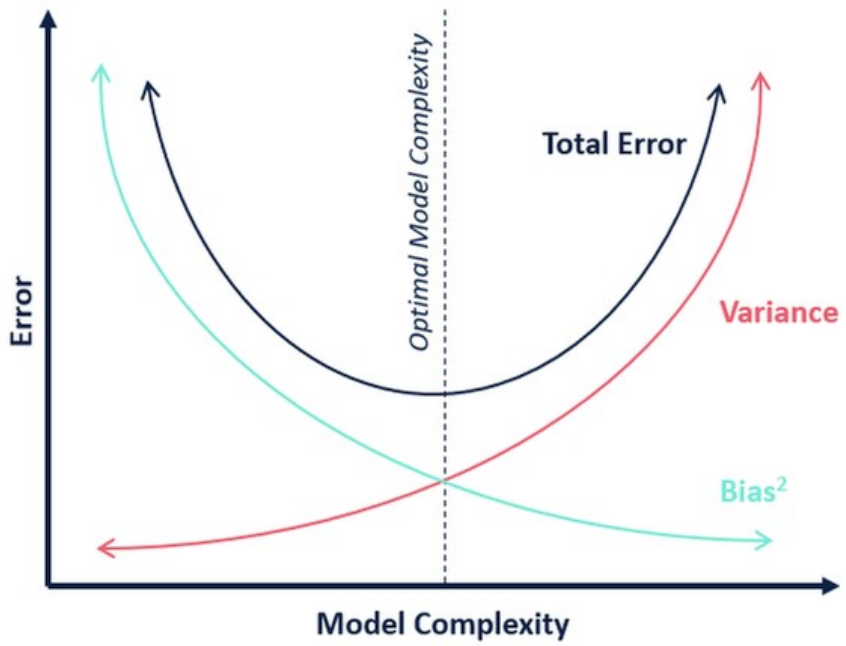
	Decision Trees	Artificial Neural Nets
Tabular data	Ensemble methods: Random Forest, Boosting	Starting from simple architectures
Text	-	Transformers (encoding, decoding, attention)
Images	-	CNNs

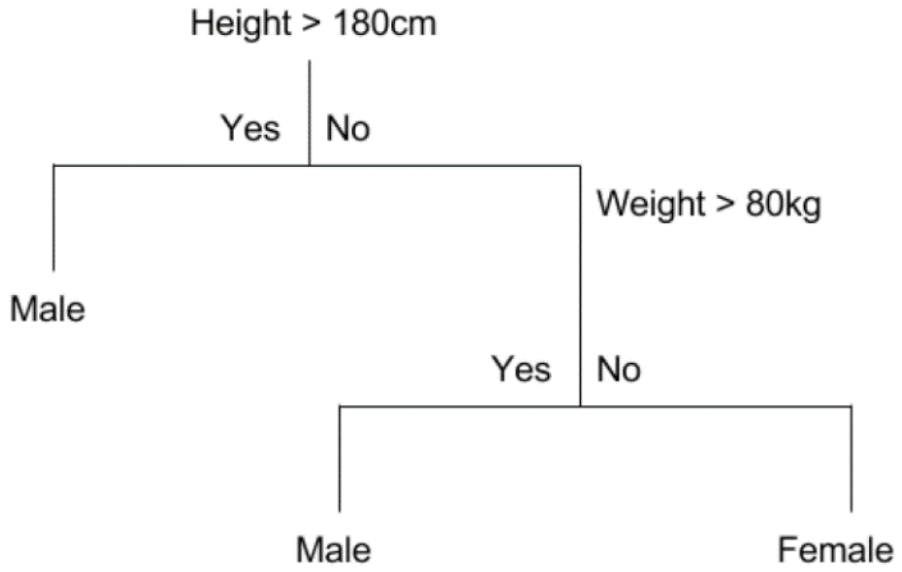
Agenda



- Review ML1
- Model Evaluation:
  - Regression Quality Metrics
  - Classification Quality Metrics
  - The Bias-Variance Trade-Off
- Model Types:
  - Decision Trees (Ensemble: Random Forrest)
  - Neural Networks
    - Basic Steps of Training a Neural Network
    - Neural Network Exercise
- Application of Machine Learning in Our Business

Recall: Bias-Variance Trade-Off



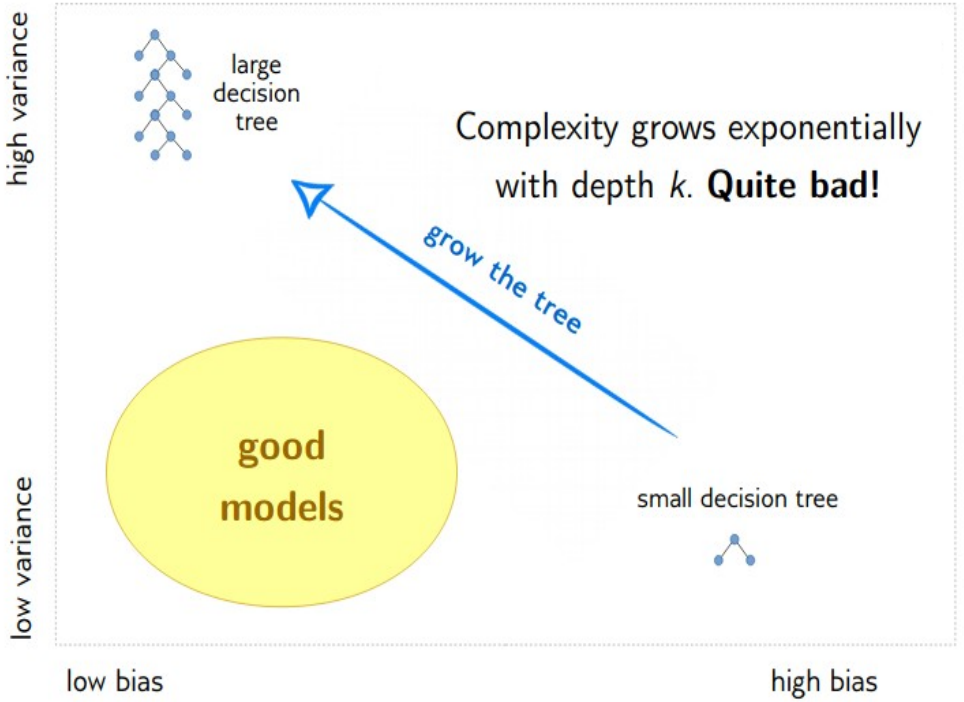


Occam‘s razor: The simplest model/most intuitive method is preferable

Classification and Regression Tree (CART): Pros and Cons

SIEMENS
Ingenuity for life

Pros	Cons
• explainable • little data preparation • implicit selection of features • can handle non-linearities • Computationally efficient	• without adaptions not very robust (=changes with diff. data) • overfitting • local optimal decisions per node → globally optimized decision tree not guaranteed



Decision Trees themselves are rather weak - **BUT**: it is easy to address their Bias and Variance problems

Ensemble Methods: Improve Decision Trees by Model Averaging

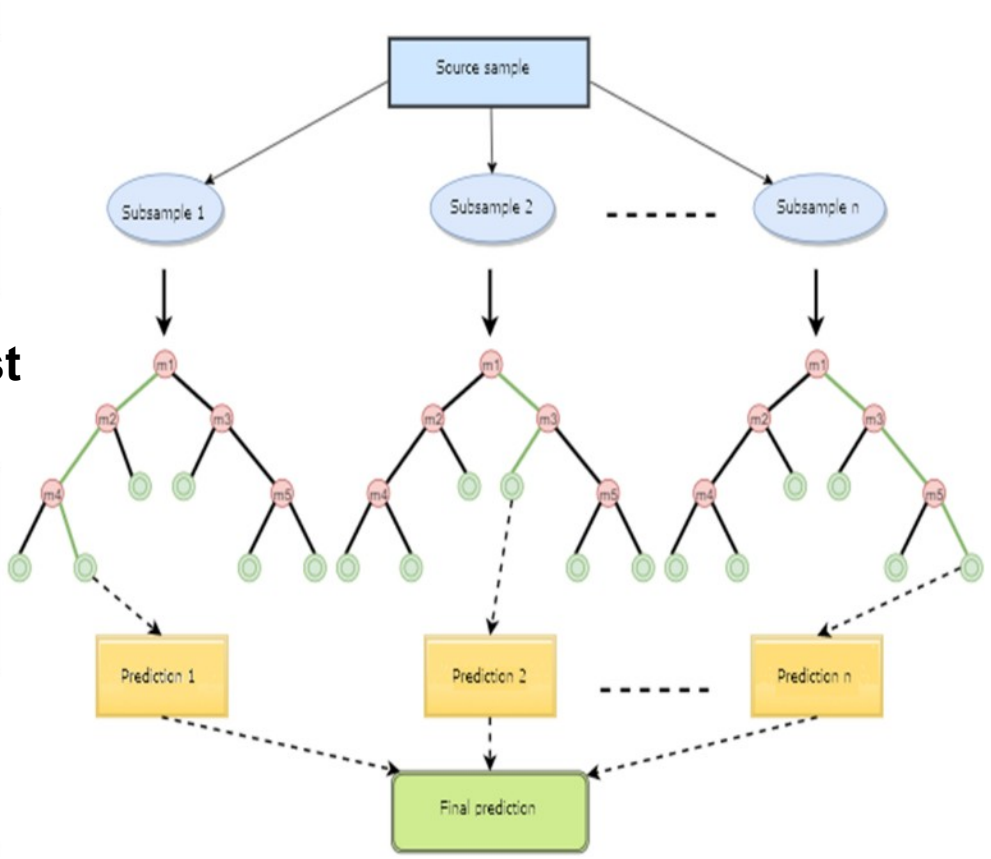
SIEMENS
Ingenuity for life

Construct more than one Decision Tree and average!

Address Variance by:

- Bootstrapping and aggregating
  (= Bagging: parallel:
  samples with replacement/
  average predictions)
- Random Feature Selection

Random Forest

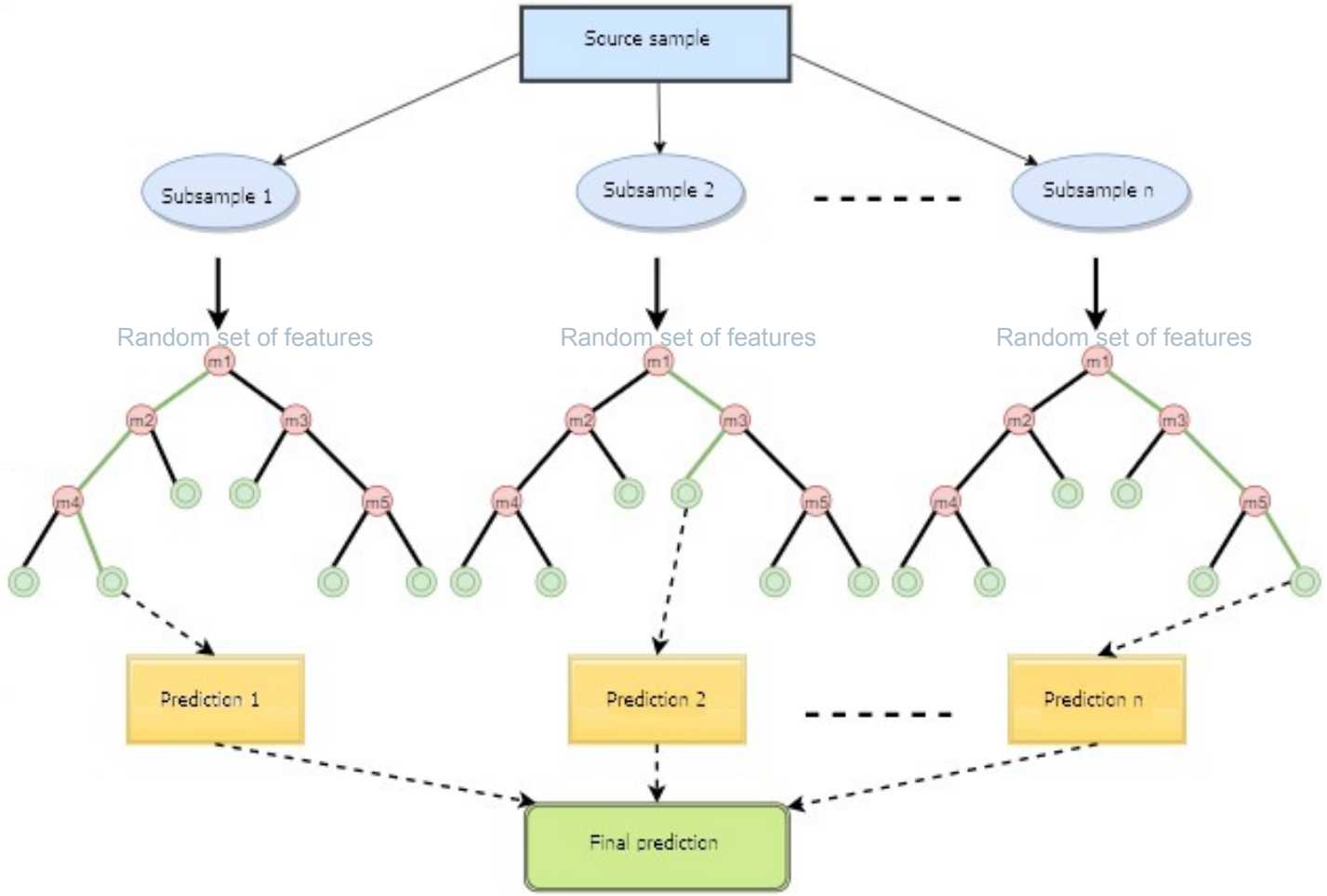


Address Bias by:

- Boosting (sequential: next tree only on errors of first tree)

  → not our focus, just and excursion

Random Forests: Bagging + Random Feature Selection



Bootstrap Aggregating (Bagging):

- **Number of Estimators n (**~100**):**
  - create n subsamples, build n models and aggregate results

(bootstrapping: random sampling with replacement)

Random Feature Selection:

- for each of the n subsamples: random set of m features (Xs) is used to construct Decision Tree
- m is typically or with M being the total number of features

Random Forests: Example for One Tree

draw subsample with replacement

	age	size	weight	health
1.	old	tall	heavy	diseased
2.	young	small	light	healthy
3.	old	small	heavy	healthy
4.	young	small	light	healthy
5.	old	tall	heavy	diseased
6.	young	tall	light	diseased
7.	young	tall	light	healthy

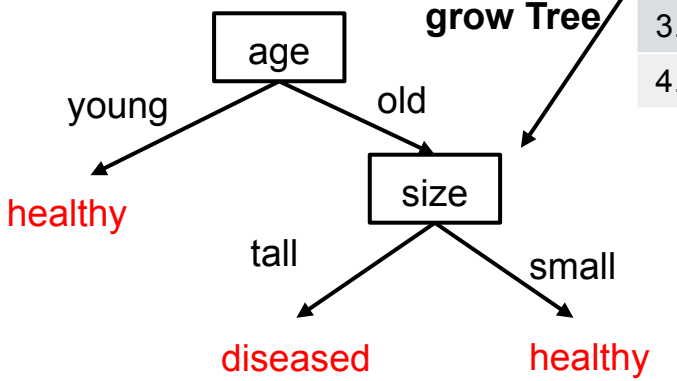
	age	size	weight	health
1.	old	tall	heavy	diseased
2.	young	small	light	healthy
3.	old	small	heavy	healthy
4.	old	small	heavy	healthy

select features

	age	size	health
1.	old	tall	diseased
2.	young	small	healthy
3.	old	small	healthy
4.	old	small	healthy

attributes: age, size, weight
Target classes: healthy/diseased

grow Tree



SIEMENS
Ingenuity for life

for each of n subsamples:

1) draw a random sample with replacement of training data
2) select m features at random
3) grow a Tree from subsample and selected features

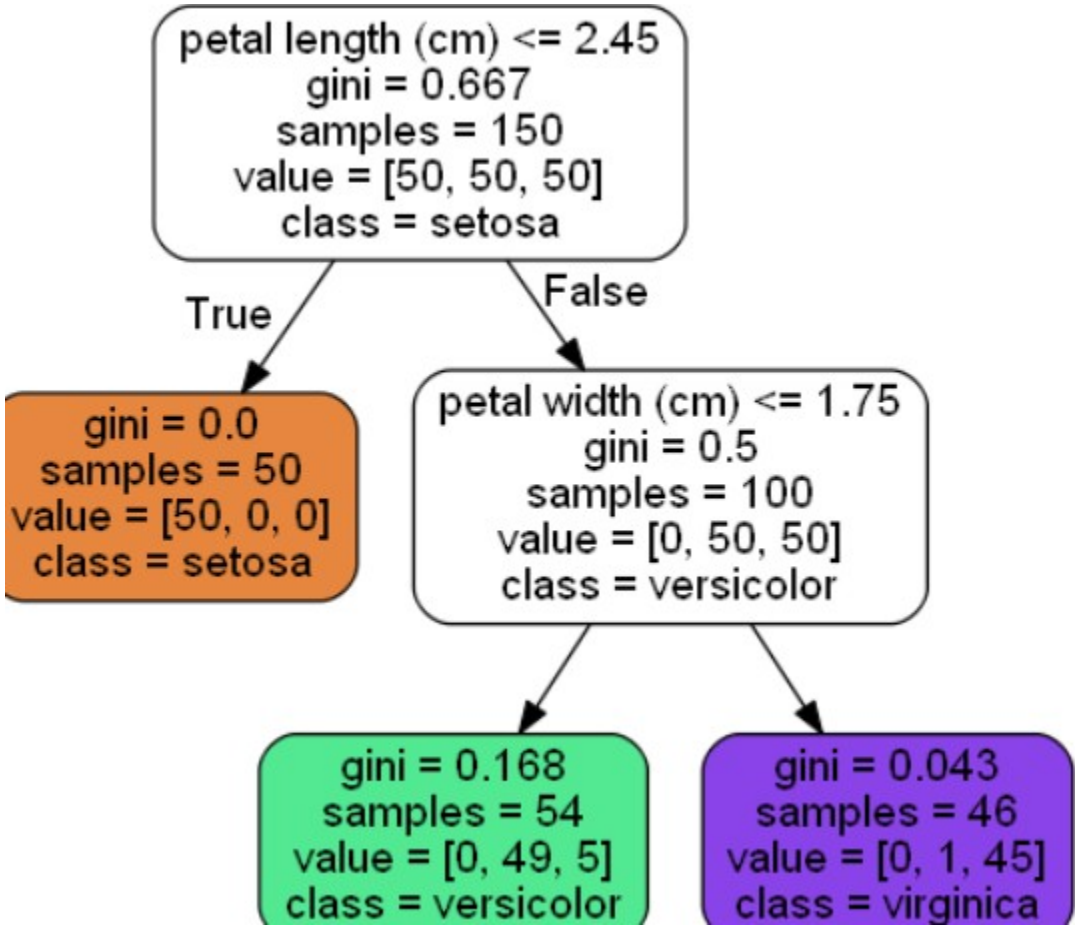
In the end:

4) aggregate results of all trees for overall result

- In Classification: each data point gets labeled according to class it was identified as in most Trees
- In Regression: e.g. calculate mean

In Comparison to a single Decision Tree: average has **smaller prediction Variance** which leads to a better performance, but therefore algorithm is a **„black box“**

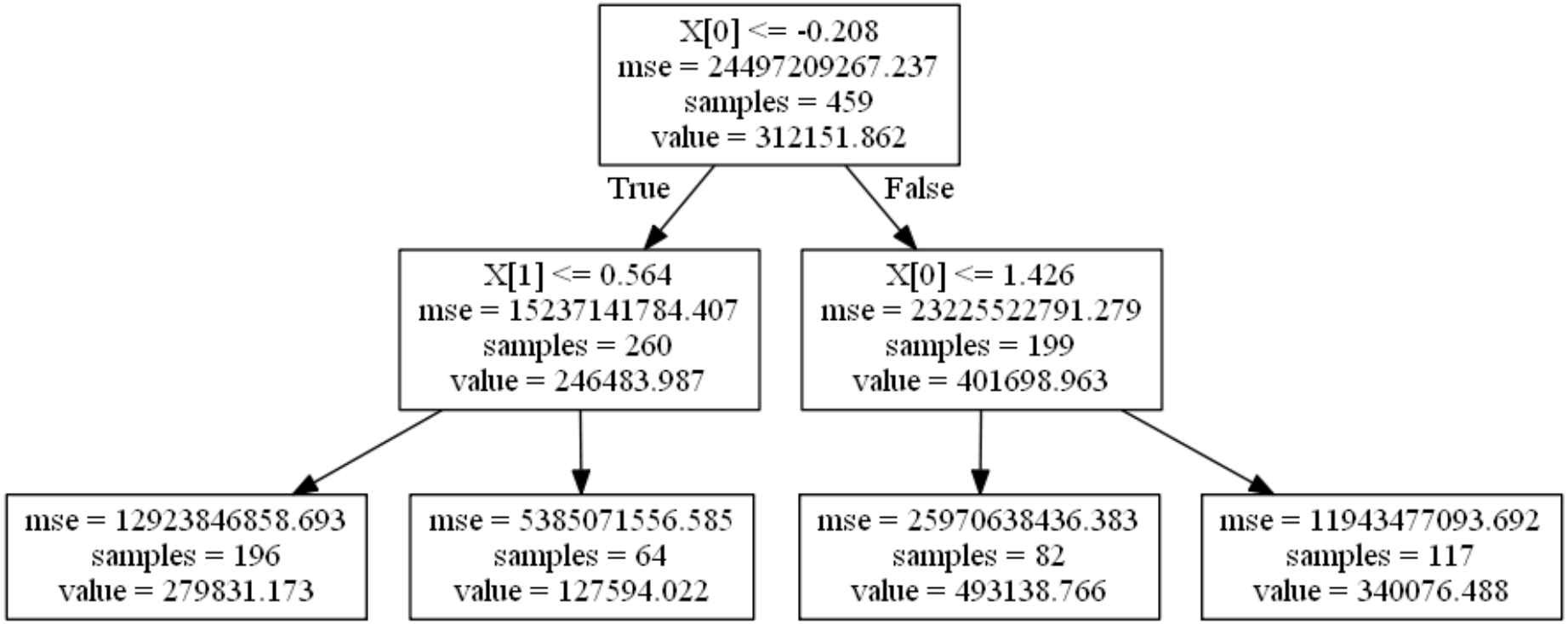
Random Forests: Classification



Random Forests: Regression

SIEMENS
Ingenuity for life

Example for Regression Trees: value = calculate something like a mean for each node or leaf



Recall Number Recognition Example from ML1

SIEMENS
Ingenuity for life

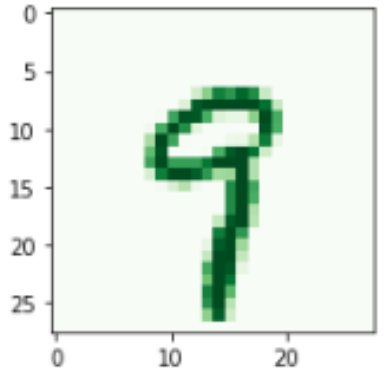
X-Variables: 28x28 = 784 variables with pixel data

X-Range: 0-255 for intensity of the pixel

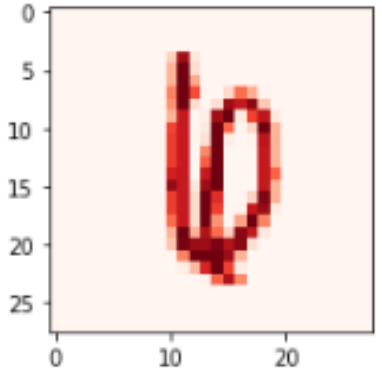
Y-Variable: correct number: 0, 1, …., 9

Numbers (700): 630 training data + 70 test data

Easy to identify:



Hard to identify:



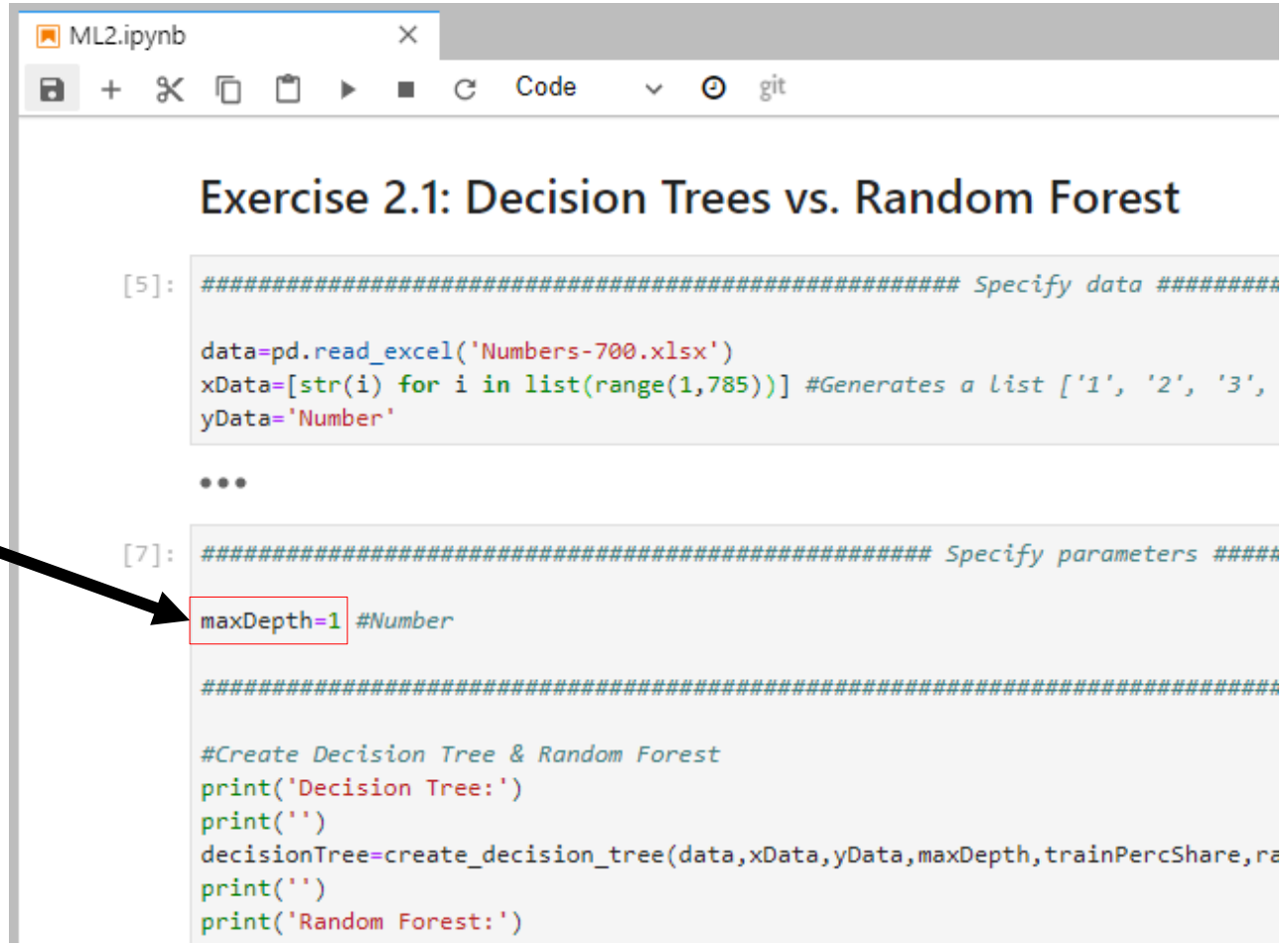
Exercise 2.1: Random Forest Vs. Decision Trees

SIEMENS
Ingenuity for life

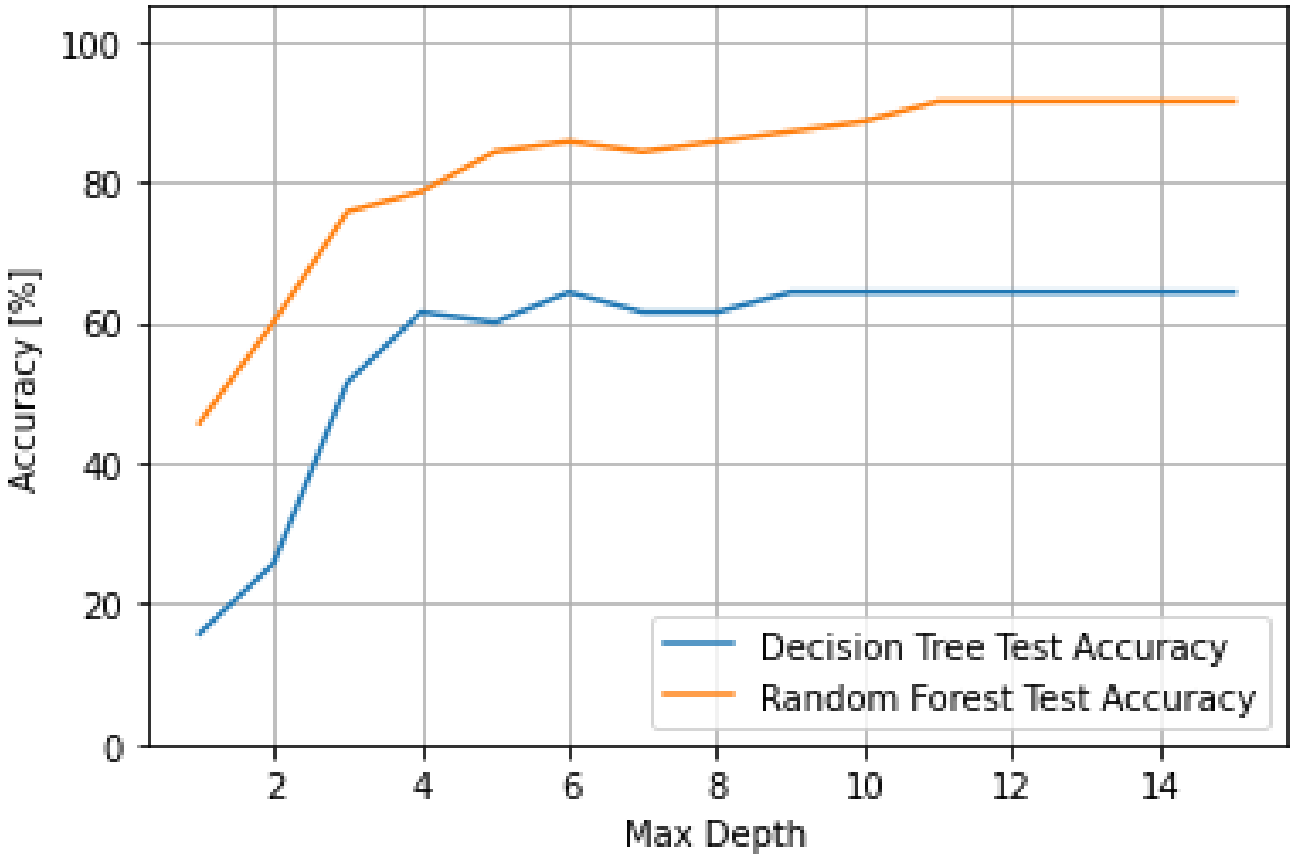
1. Open the ML2.ipynb Notebook
2. Run through all cells till Exercise 2.1: Decision Trees vs. Random Forest (shift+enter)
3. Repeat:

   Change the max depth (1-20) and create random forests and decision trees for each max depth you choose (shift+enter)

For random forest default number of estimators is 100



Exercise 2.1: Random Forest Vs. Decision Trees



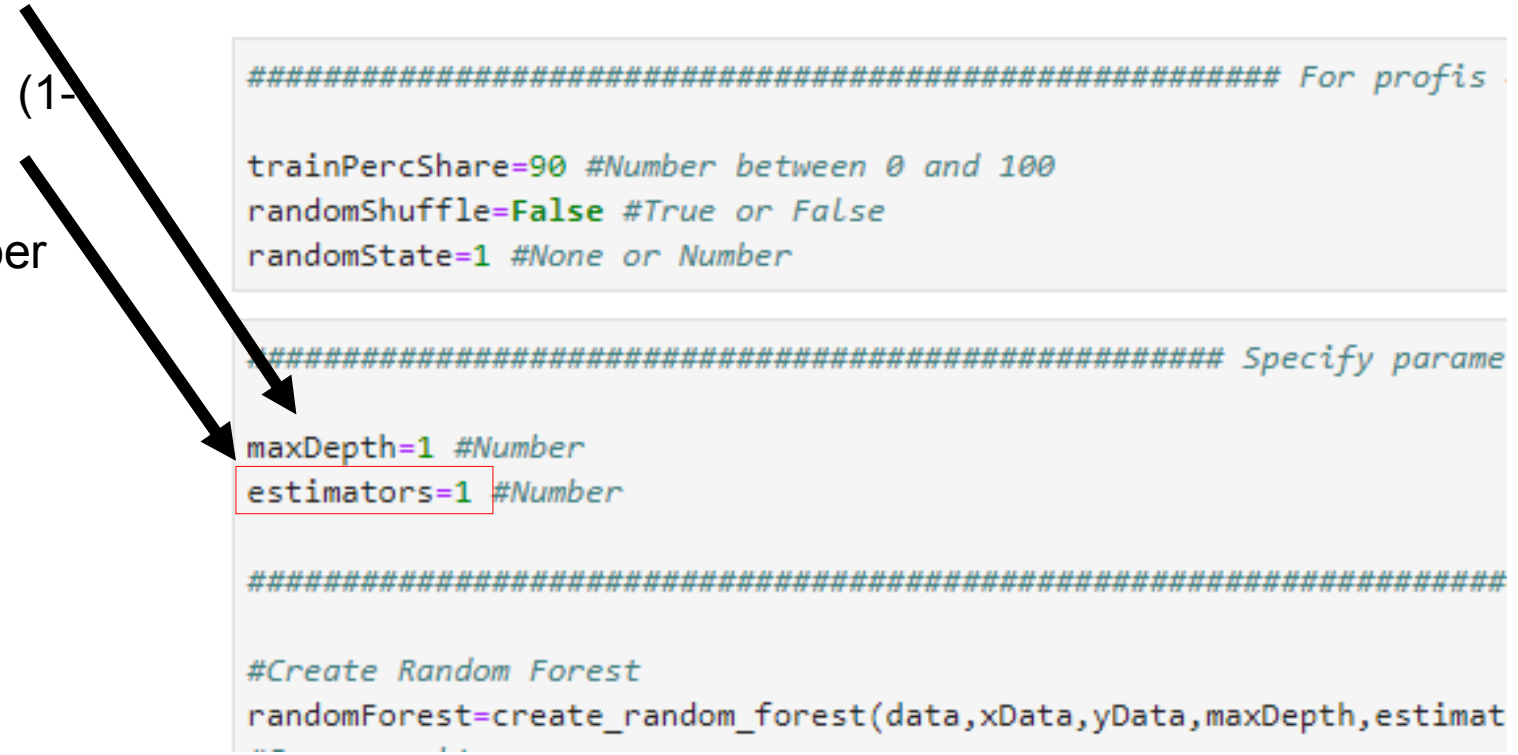
- We get better results from random forest!
- Similar trend for random forest and decision tree

Exercise 2.2: Explore the Influence of #Estimators (#Models/Subsamples) of Random Forests

SIEMENS
Ingenuity for life

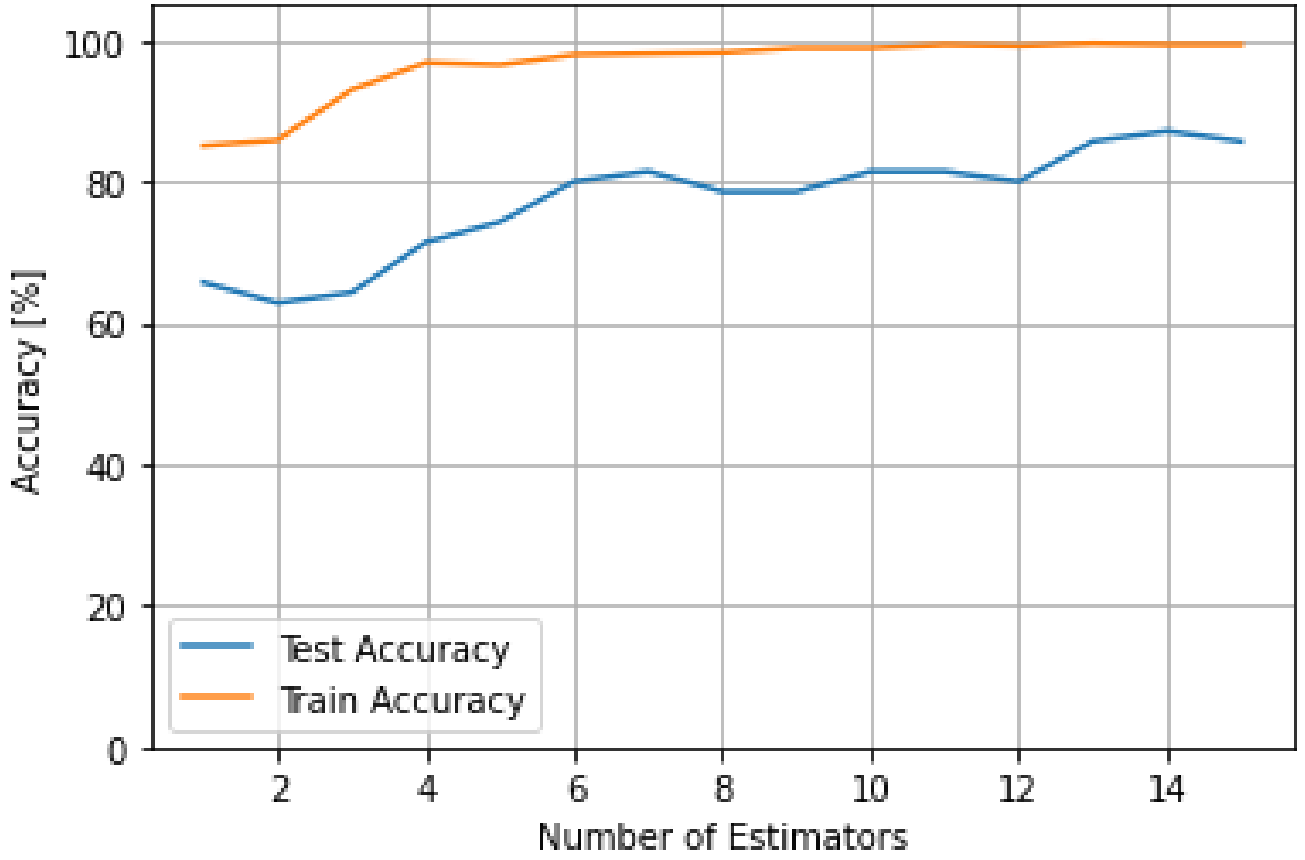
1. Go to Exercise 2.2: Random Forest
2. Enter the max depth you think is the best
3. Change the number of estimators (1-30) and create random forests
4. What do you think is a good number of estimators?

Default number of estimators is 100



Exercise 2.2: Explore the Influence of #Estimators of Random Forests

SIEMENS
Ingenuity for life



- We use Max Depth = 8
- In our example seems 6 - 12 to be a good number of estimators

Observed Parameters for Decision Tree and Random Forest in Python

SIEMENS
Ingenuity for life

```
from sklearn.tree import DecisionTreeClassifier

DecisionTreeClassifier(criterion='entropy',max_depth=1)
```

For more information go to:
https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html#sklearn.tree.DecisionTreeClassifier

Gini or Entropy – How the purity of the dataset should measured (default: 'gini')

Maximal depth of the tree (default: None)

```
from sklearn.ensemble import RandomForestClassifier

RandomForestClassifier(criterion='entropy',max_depth=1,n_estimators=1)
```

For more information go to:
https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

Number of build models / subsets (default: 100)

There are a lot more parameters you can variate to perfect your model. But therefore it's not always easy to find a very good combination of parameters.

Excursus: Boosting

SIEMENS
Ingenuity for life

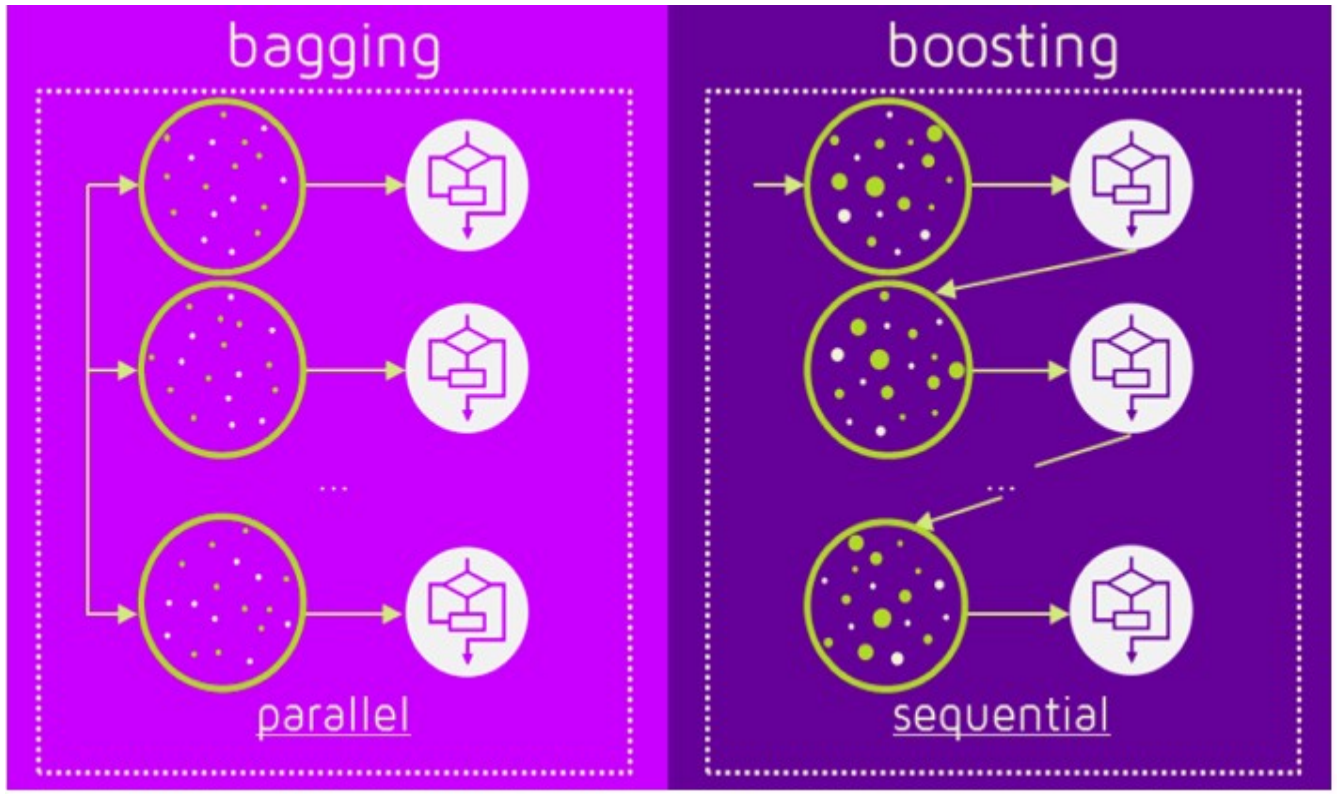
Improve a Decision Tree by running it multiple times on **weighted** training data.

Iteratively:

1) draw a sample from data with replacement. In first iteration each data point has equal likeliness to be chosen
2) grow the Tree from sample
3) evaluate which data points in training data were incorrectly labeled
   - ➢ these data points get weighted higher
   - ➢ thus more likely to be chosen in next sample drawing

In the end:

4) aggregate results of all trees for overall result



Decision Tree Algorithms: Sum up

SIEMENS
Ingenuity for life

Decision Trees itself are rather weak ML algorithms

However, with little tricks they can become VERY powerful and at the same time computationally efficient (compared to NNs):

- Random Forests (focus this training):
  - Bagging
  - Random feature selection
  - -> fully grown trees (high variance, low bias): average different overfitted trees
- Boosting (considered for future trainings)
  - -> shallow trees (high bias, low variance): sequentially reduce bias

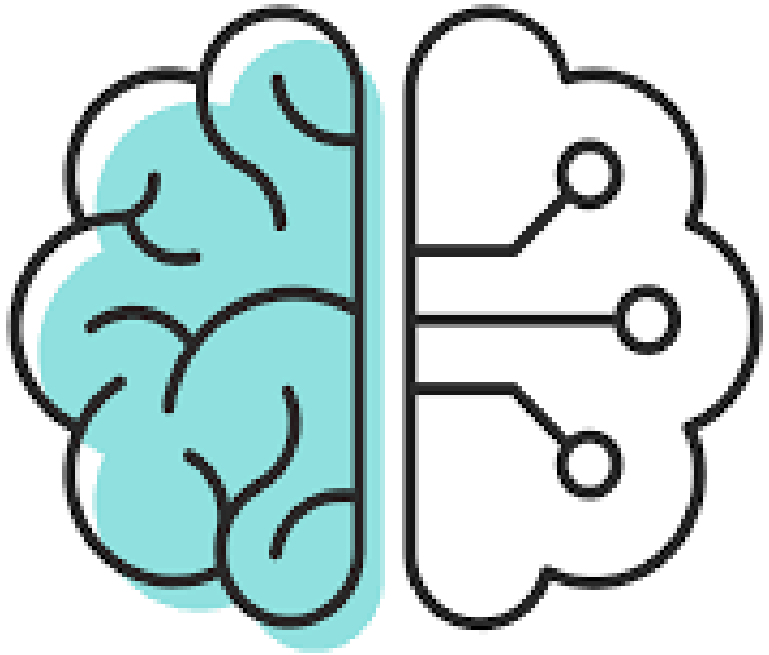
Overview: what we achieved so far

SIEMENS
Ingenuity for life

	Target values	Loss function	Error metric	Act function last layer	Act func hidden layers
Regression	Continous	MSE, R^2, MAE,…	MSE, R^2, MAE,…		
Classification	Binary	Binary cross entropy	F1 score, precision, recall, ROC-curve		
	multiple classes	categorical cross entropy	F1 score, precision, recall, ROC-curve		

	Decision Trees	Artificial Neural Nets
Tabular data	Ensemble methods: Random Forest, Boosting	Starting from simple architectures
Text	-	Transformers (encoding, decoding, attention)
Images	-	CNNs

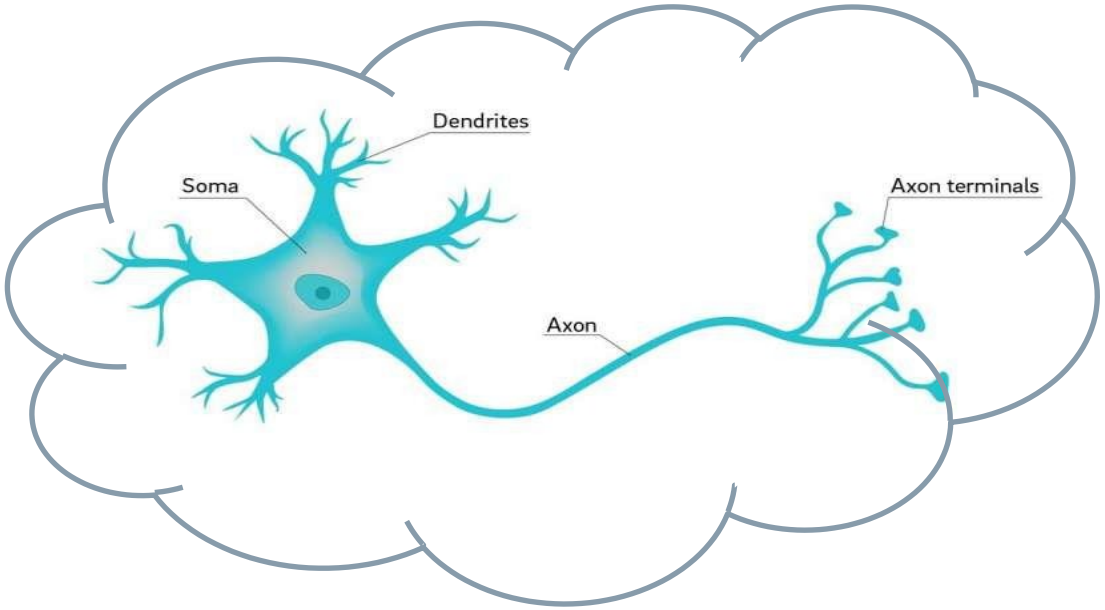
Agenda



- Review ML1
- Model Evaluation:
  - Regression Quality Metrics
  - Classification Quality Metrics
  - The Bias-Variance Trade-Off
- Model Types:
  - Decision Trees (Ensemble: Random Forrest)
  - Neural Networks
    - Basic Elements and Steps of Training a Neural Network
    - Neural Network Exercise
- Application of Machine Learning in Our Business

Review: Artificial Neural Nets (ANN) Are Highly Complex Models Inspired by the Biological Neural Networks that Constitute Brains

SIEMENS
Ingenuity for life



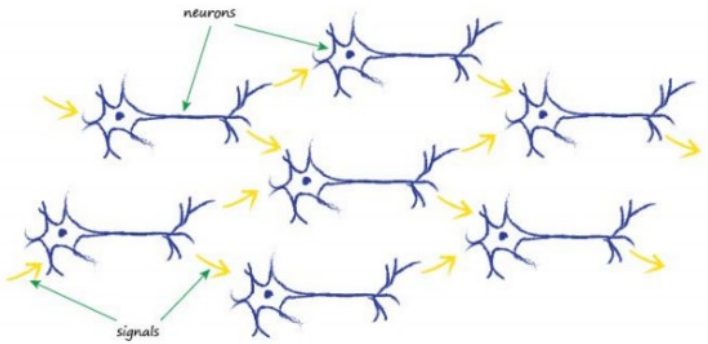
- With just ~100.000 neurons a fruit fly brain can:
  1. Fly
  2. Find food
  3. Realize and avoid danger
  4. …
- All our computers have much more calculation capacity than 100.000 neurons
- Why is it hard for computers to do what a fruit fly can?

Adapted from: Tariq Rashid " Make Your Own Neural Network"

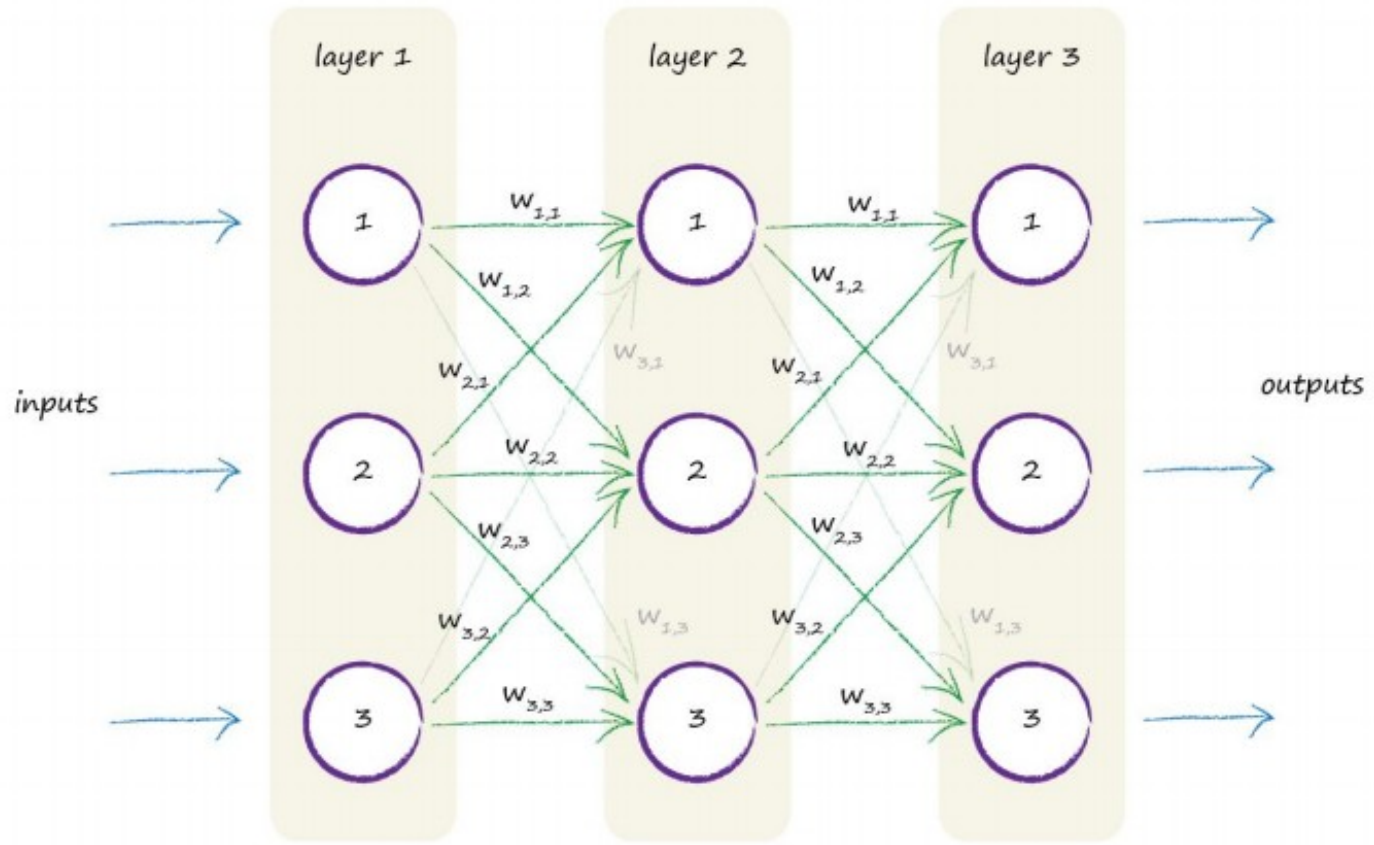
Neural Networks Take Key Inspirations from Biological Brains: 1. Create a Net of Many Input-Output Connections

SIEMENS
Ingenuity for life

Human brain → Neural Network



1. Build network of neurons in several layers
2. Connect a neuron with all neurons to previous and next layer (fully connected net)
3. Weights Wi,j adjust the strength of the connections between neurons
4. Sum weighted inputs up to next layer neuron



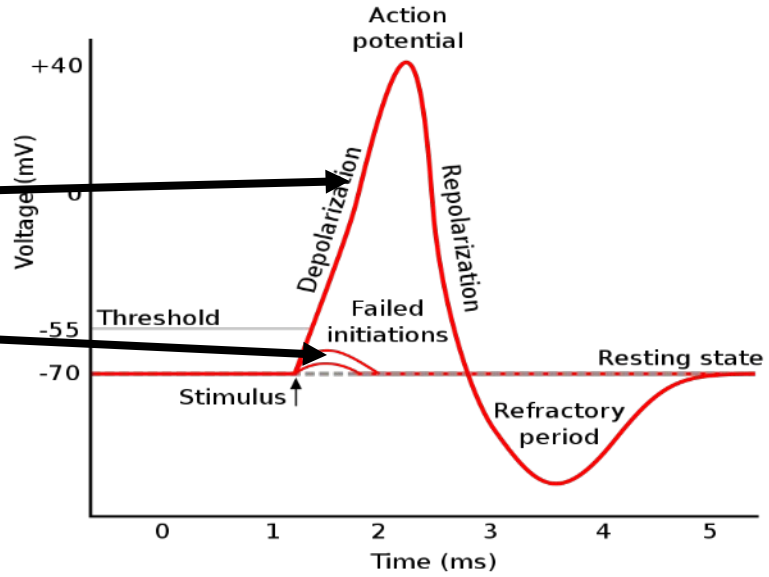
Adapted from: Tariq Rashid " Make Your Own Neural Network"

Neural Networks Take Key Inspirations from Biological Brains: 2. Use Activation Function to Differentiate btw. Important and Unimportant Inputs

SIEMENS
Ingenuity for life

Neurons work with a threshold:

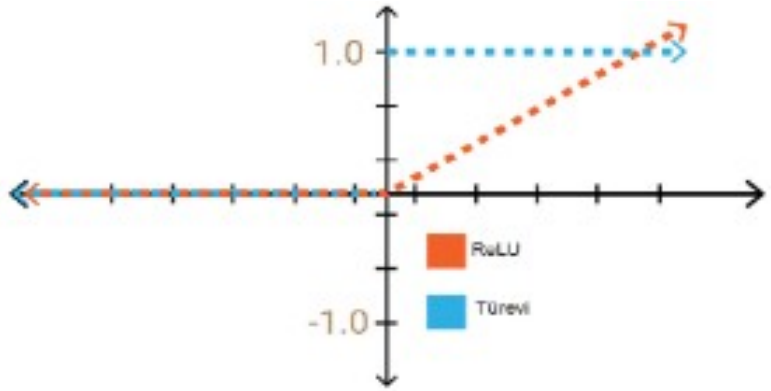
- Large inputs -> output
- Small input -> no output change



https://en.wikipedia.org/wiki/Action_potential

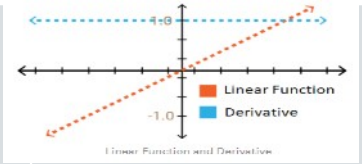
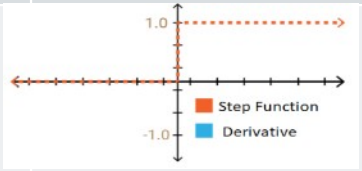
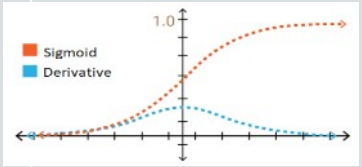
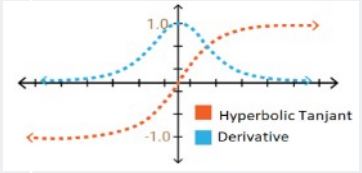
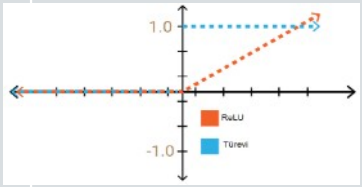
Computer uses an “Activation Function” instead

- Not dealing with time, but weighting input:
- Often used in hidden layers:
  - ReLU function:
    - Efficient as linear, but can capture more complex relations (deactivates less important parts of network to zero)
    - Normalize data before to 0-1 (otherwise weights too big) and regularize loss function
    - No vanishing gradients problem, but dying ReLu (leaky ReLu)



List of theoretically possible Activation Functions

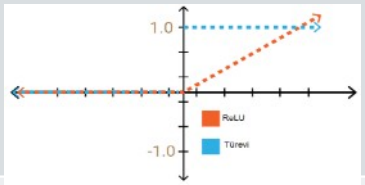
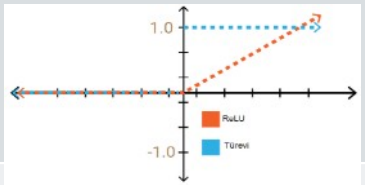
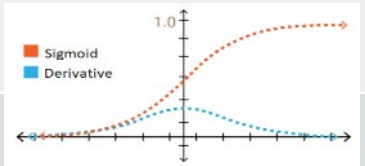
SIEMENS
Ingenuity for life

	Description	Figure (red)	Pros and Cons
Linear Function (Identity)	y = x		- Non binary output - Problem with non-linearities
Step Function (Heavy side)	y = θ(x) x<0 ☾ y=0 x>0 ☾ y=1		- Good to be used for output layer - Represent no derivative learning values
Sigmoid Function (Logistic)	y = sig(x) 0 < y < 1		- Activation value never gets zero (input not lost) - Good for predicting probabilities (0 ≤ y ≤ 1) - Lost gradient problem
Hyperbolic-Tangent Function (Tanh)	y = tanh(x) -1 < y < 1		- Activation value never gets -1 (input not lost) - More efficient than sigmoid + learning for negative values - Good for classification between two classes - Lost gradient problem
Rectified Linear Unit Function (ReLU)	y = max(0,x) x<0 ☾ y=0 x>0 ☾ y=x		- Very efficient ☾ Good for multi-layer networks - Solves lost gradient problem - No learning for negative values n/ dying ReLu (-> Leaky ReLu)

Our Focus

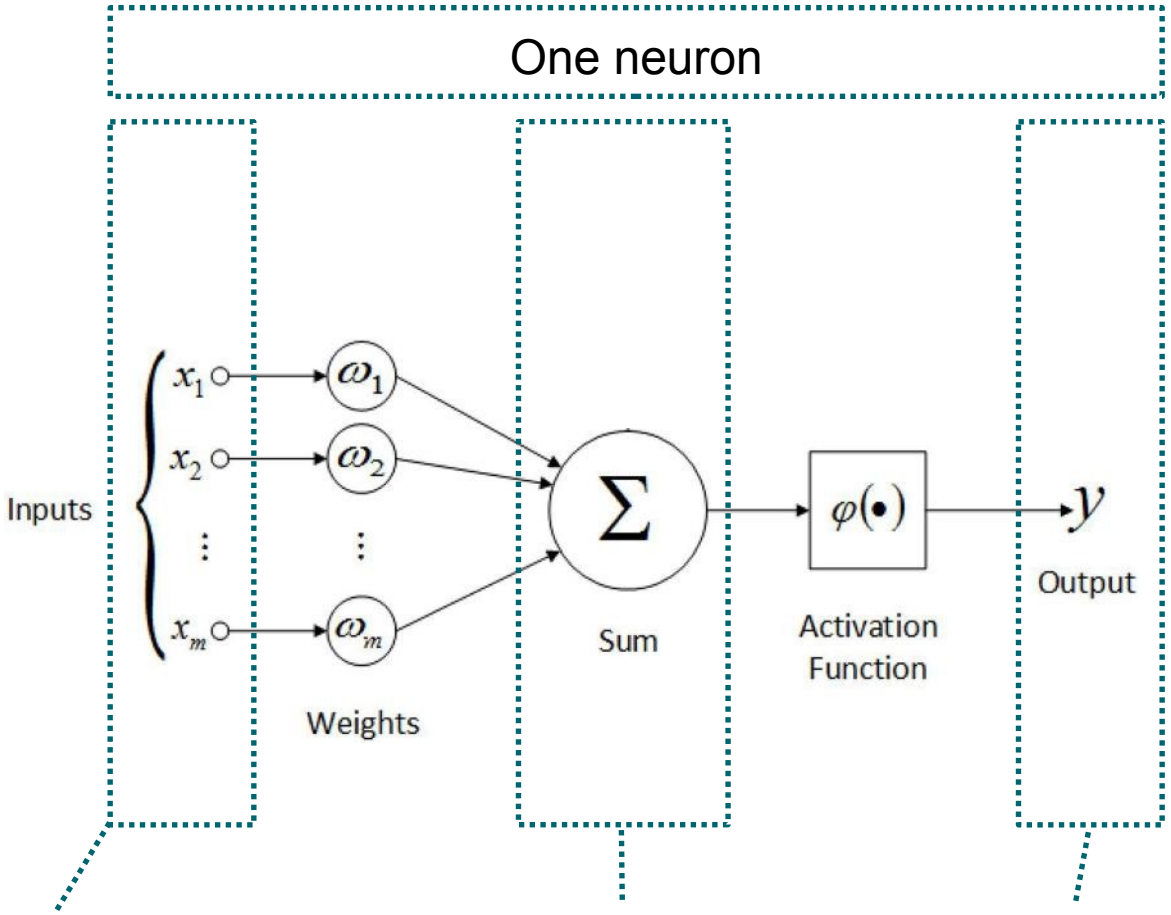
Mostly used Activation Functions: hidden and output layer

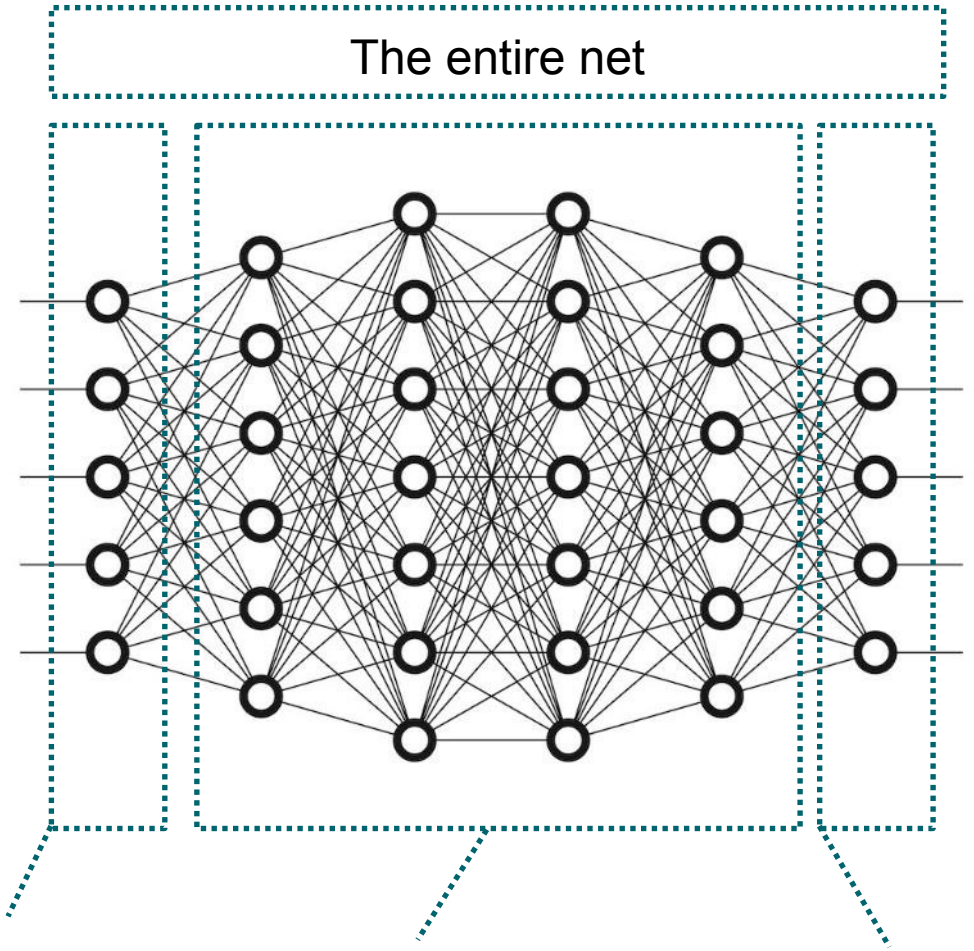
SIEMENS
Ingenuity for life

HIDDEN LAYER	Description	Figure (red)	Pros and Cons
Rectified Linear Unit Function (ReLU)	y = max(0,x) x<0 ☾ y=0 x>0 ☾ y=x		- Very efficient ☾ Good for multi-layer networks - Solves lost gradient problem - No learning for negative values n/ dying ReLu (-> Leaky ReLu)
OUTPUT LAYER			
Rectified Linear Unit Function (ReLU)	y = max(0,x) x<0 ☾ y=0 x>0 ☾ y=x		- Predict continuous value: regression (linear activation functions works as well)
Sigmoid Function (Logistic)	y = sig(x) 0 < y < 1		- Predict probabilities (0 ≤ y ≤ 1) for binary classification
Softmax Function			- Predict probability distribution for multi class classification (sums up to 1)

Review: Analogies between Human and Artificial Neural Nets

SIEMENS
Ingenuity for life





‚Dendrites': weighted inputs from many other neurons

‚Neurons'

‚Axon'

Input layer

Net of many input-output-connections created by Hidden layers (example: 4 layers)

Output layer

The Learning Rate (LR) (Review)

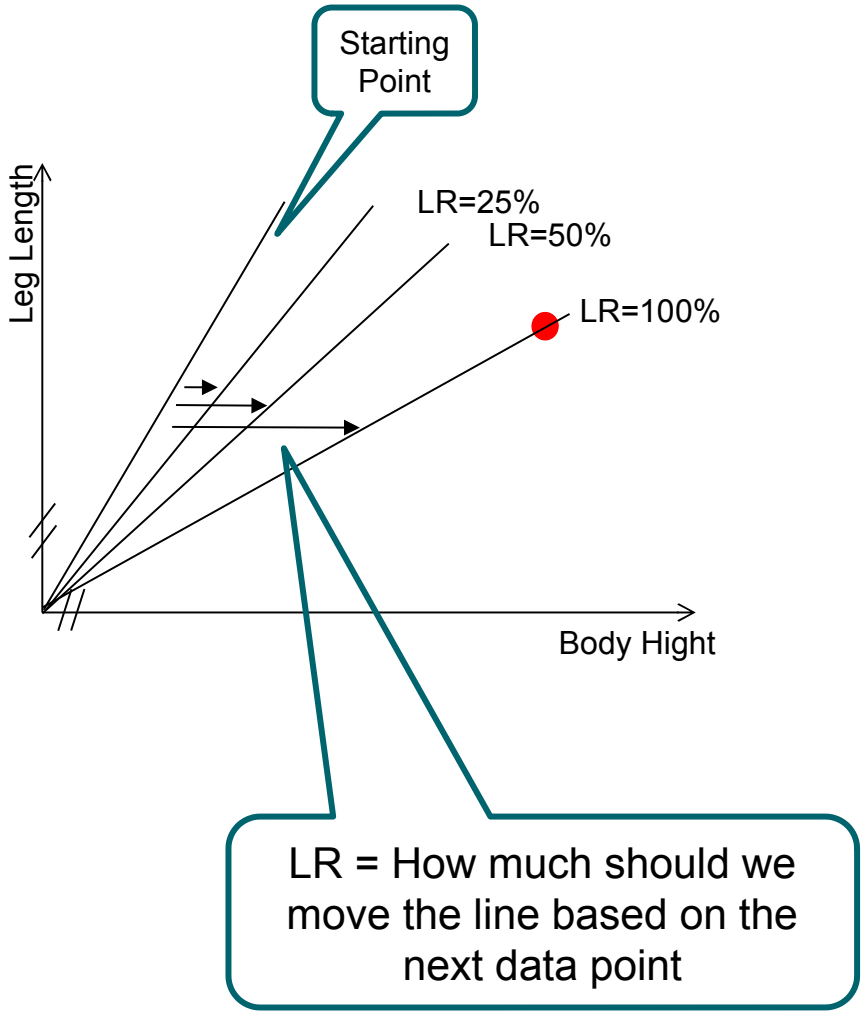
SIEMENS
Ingenuity for life

Makes NN to adapt for every additional data point

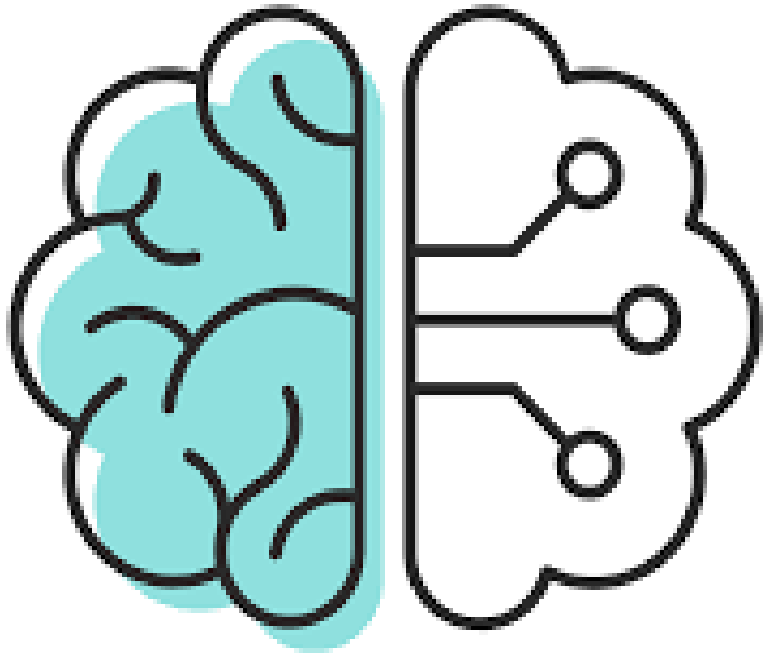
- Factor between 0 and 100%
- Determines how much the line moves to the next data point
- Dampens effect of the last data, errors and noise

Formula: Move = Learning Rate x Distance / 100%

- For big data (> Mill) often: LR< 0.01%



Agenda



- Review ML1
- Model Evaluation:
  - Regression Quality Metrics
  - Classification Quality Metrics
  - The Bias-Variance Trade-Off
- Model Types:
  - Decision Trees (Ensemble: Random Forrest)
  - Neural Networks
    - Basic Elements and Steps of Training a Neural Network
    - Neural Network Exercise
- Application of Machine Learning in Our Business

Basic Steps of Training a Neural Network with Sigmoid Activation Function Using Statistical Gradient Descent (SGD) Solver

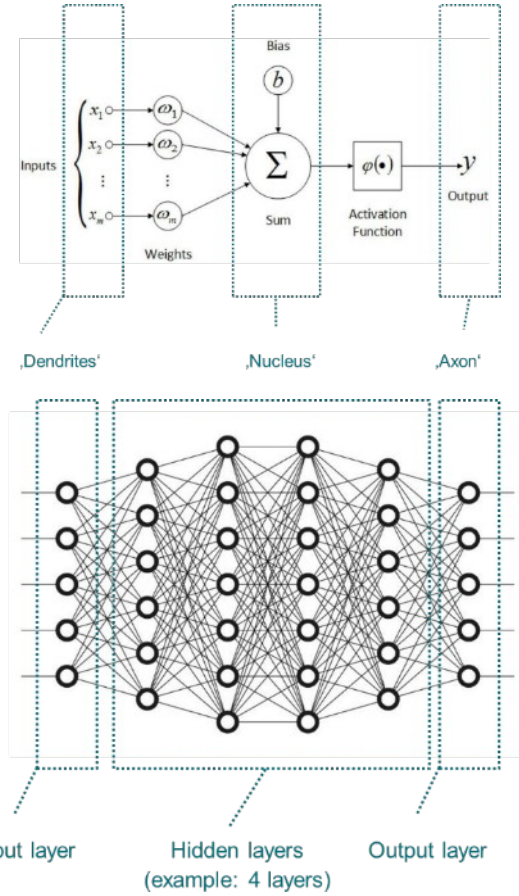
SIEMENS
Ingenuity for life

a) Select first element from the training data set: X_1 (Inputs) and Y_1 (Outputs)

   1. Start with random weights (some restrictions apply) -> NN_1
   2. Calculate predicted output O_1 using NN_1 and first element
   3. Calculate Error $E_1 = Y_1 - O_1$ and back propagate to weights
   4. Recalculate weights based on error -> NN_2

b) Repeat step 2-4 with all elements from the training data set

c) Iterate through all data several times (steps a) and b)) until weights converge to one value



Traditional Regression Analysis would calculate all weights based on all sample data at once. (Not good for general solution.)

1. Start with Random Weights -> NN_1

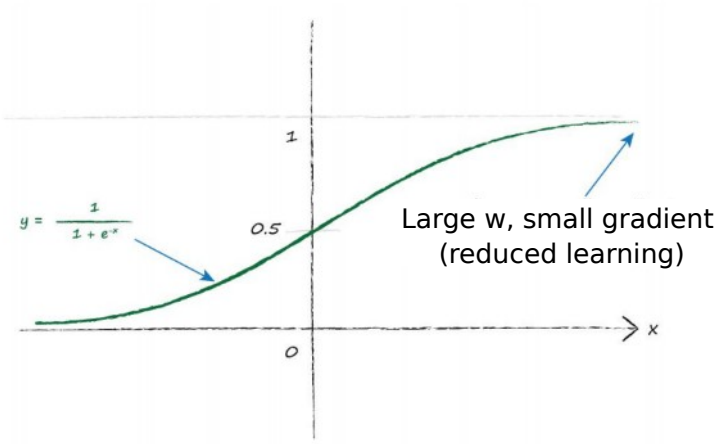
SIEMENS
Ingenuity for life

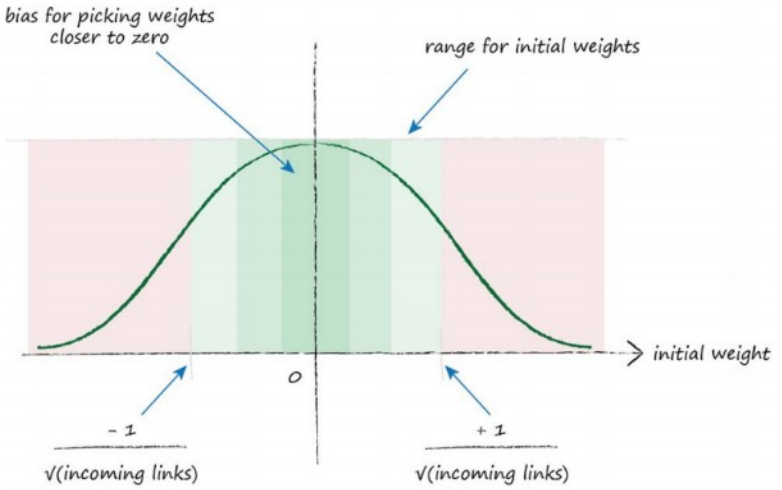
Limitations on randomness:

1. Avoid large values (normalize/ standardize)
   -> otherwise Y will explode (ReLu)

2. If many nodes use smaller weights to avoid large outputs
   i.e. normal distribution with:
   - Mean = 0
   - Standard deviation = 1/square root (incoming links)

3. No Zeros!
   - Factor zero kills input signal, no learning from input





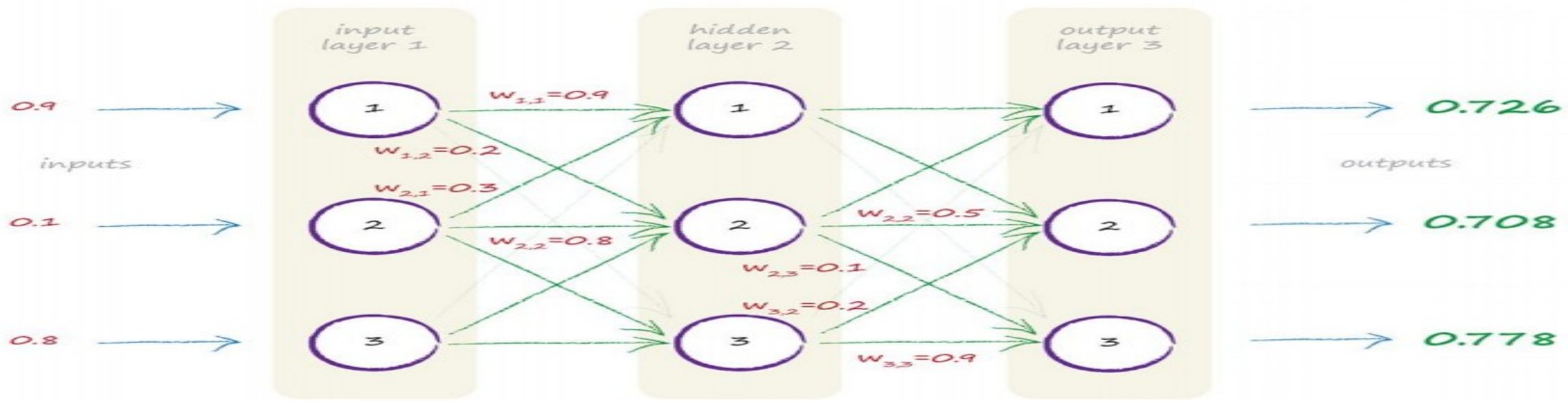
Adapted from: Tariq Rashid " Make Your Own Neural Network"

2. Calculate predicted output O_1 using NN_1

SIEMENS
Ingenuity for life

See following slides for details

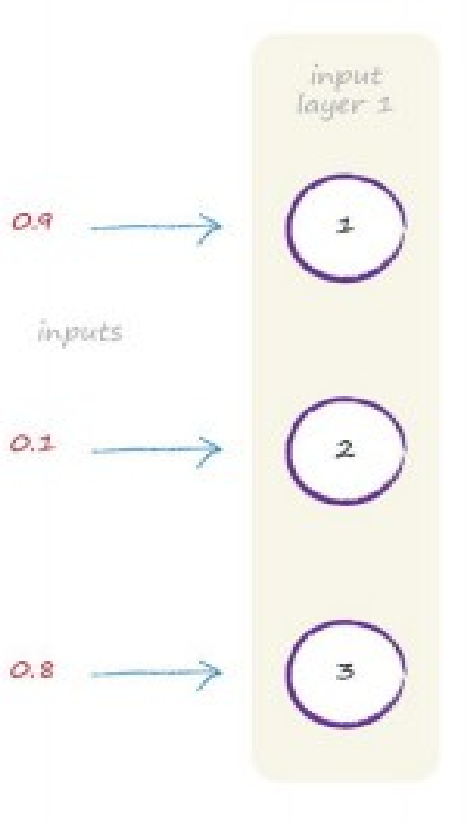
Matrix/ Vector connotation: I_{1_input} $W_{1\_input\_hidden}$ X_{1_hidden}->O_{1_hidden} $W_{1\_hidden\_output}$ X_{1_output} ->O_{1_output}



Four calculation steps in matrix formulas:	$X_{1\_hidden} = W_{1\_hinput\_hidden} \times I_{1\_input}$ O_{1_hidden} = sigmoid (X_{1_hidden}) (usually ReLu) $X_{1\_output} = W_{1\_hidden\_output} \times O_{1\_hidden}$ O_{1_output} = sigmoid (X_{1_output}) (for binary classification)

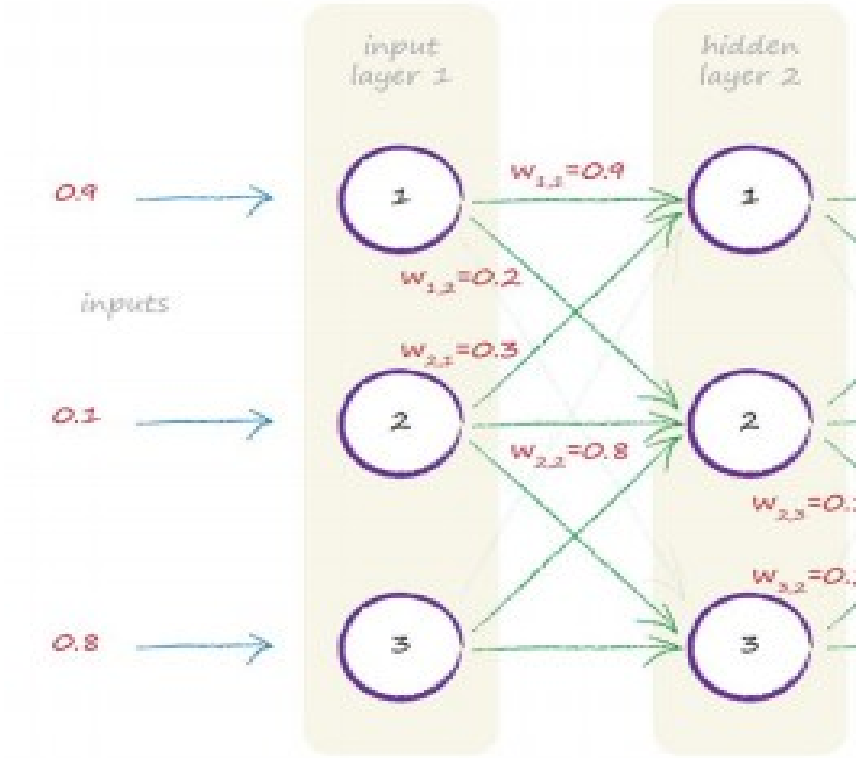
Adapted from: Tariq Rashid " Make Your Own Neural Network"

Passing Input Values to Input Layer



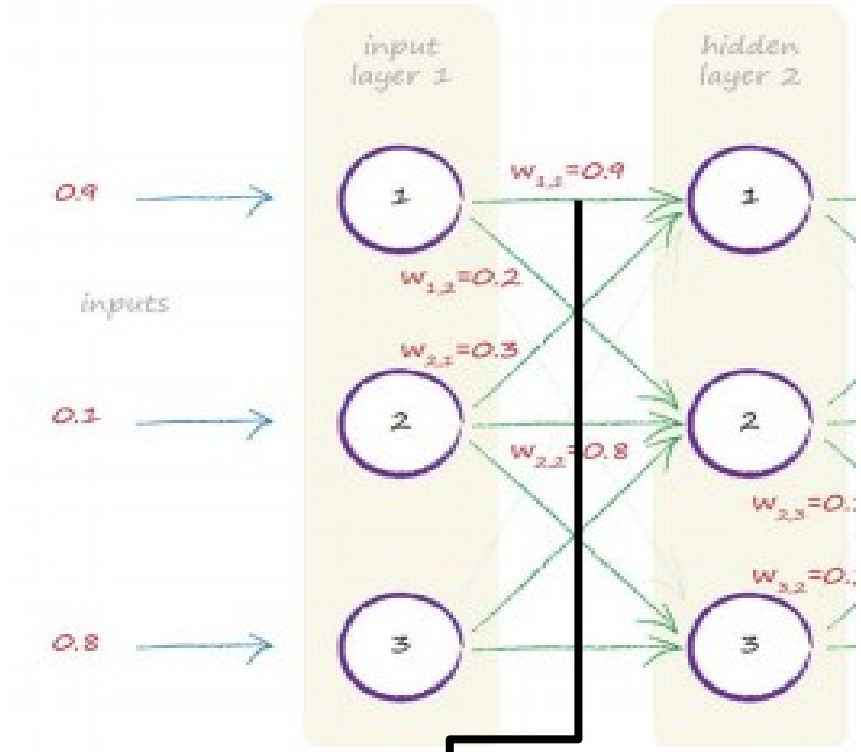
Adapted from: Tariq Rashid " Make Your Own Neural Network"

The Node Connections between Input Layer and Hidden Layer Are Weighted



Adapted from: Tariq Rashid " Make Your Own Neural Network"

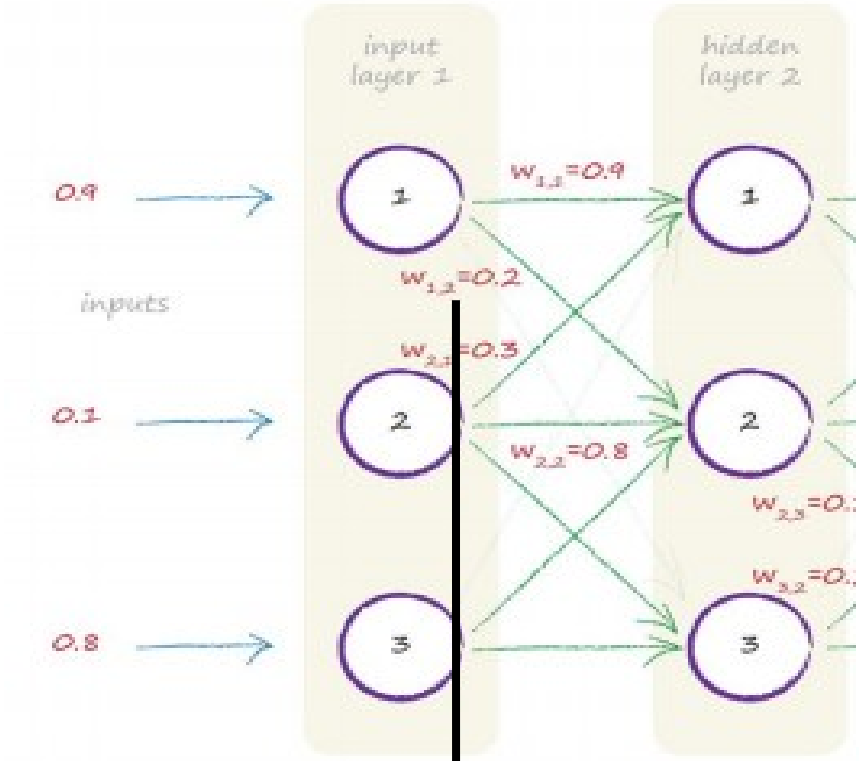
The Weightings are Used to Determine the Input for the Hidden Layer



$$\begin{pmatrix} 0.9 & 0.3 & 0.4 \\ 0.2 & 0.8 & 0.2 \\ 0.1 & 0.5 & 0.6 \end{pmatrix} \cdot \begin{pmatrix} 0.9 \\ 0.1 \\ 0.8 \end{pmatrix} = \begin{pmatrix} 1.16 \\ 0.42 \\ 0.62 \end{pmatrix} = X_{hidden}$$

Adapted from: Tariq Rashid " Make Your Own Neural Network"

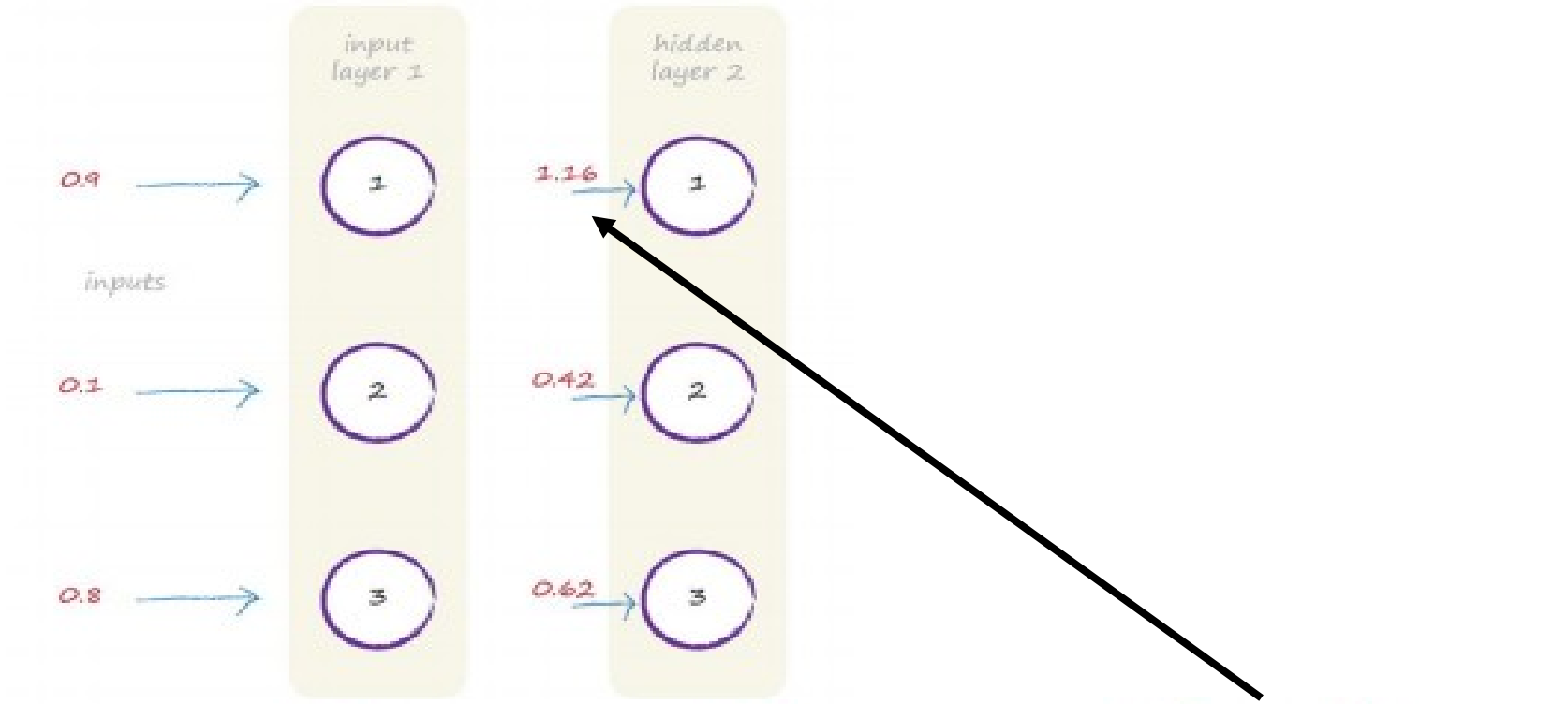
The Weightings Can Be Used to Determine the Input for the Hidden Layer



$$\begin{pmatrix} 0.9 & 0.3 & 0.4 \\ 0.2 & 0.8 & 0.2 \\ 0.1 & 0.5 & 0.6 \end{pmatrix} \cdot \begin{pmatrix} 0.9 \\ 0.1 \\ 0.8 \end{pmatrix} = \begin{pmatrix} 1.16 \\ 0.42 \\ 0.62 \end{pmatrix} = X_{hidden}$$

Adapted from: Tariq Rashid " Make Your Own Neural Network"

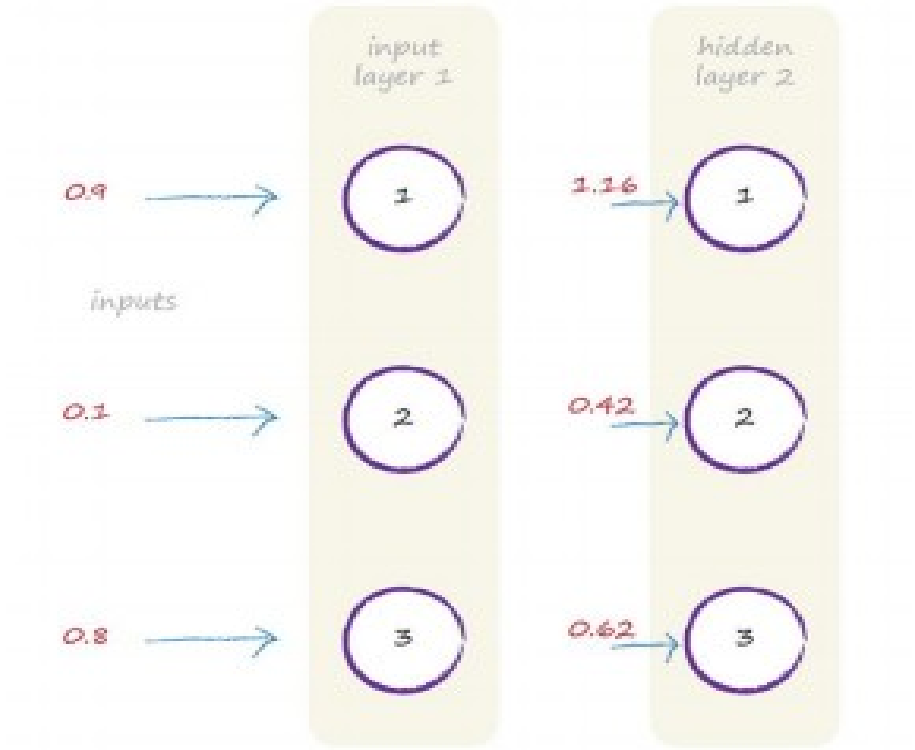
The Weightings Can Be Used to Determine the Input for the Hidden Layer



$$\begin{pmatrix} 0.9 & 0.3 & 0.4 \\ 0.2 & 0.8 & 0.2 \\ 0.1 & 0.5 & 0.6 \end{pmatrix} \cdot \begin{pmatrix} 0.9 \\ 0.1 \\ 0.8 \end{pmatrix} = \begin{pmatrix} 1.16 \\ 0.42 \\ 0.62 \end{pmatrix} = X_{hidden}$$

Adapted from: Tariq Rashid " Make Your Own Neural Network"

The Output of the Hidden Layers Is Calculated with the Activation Function

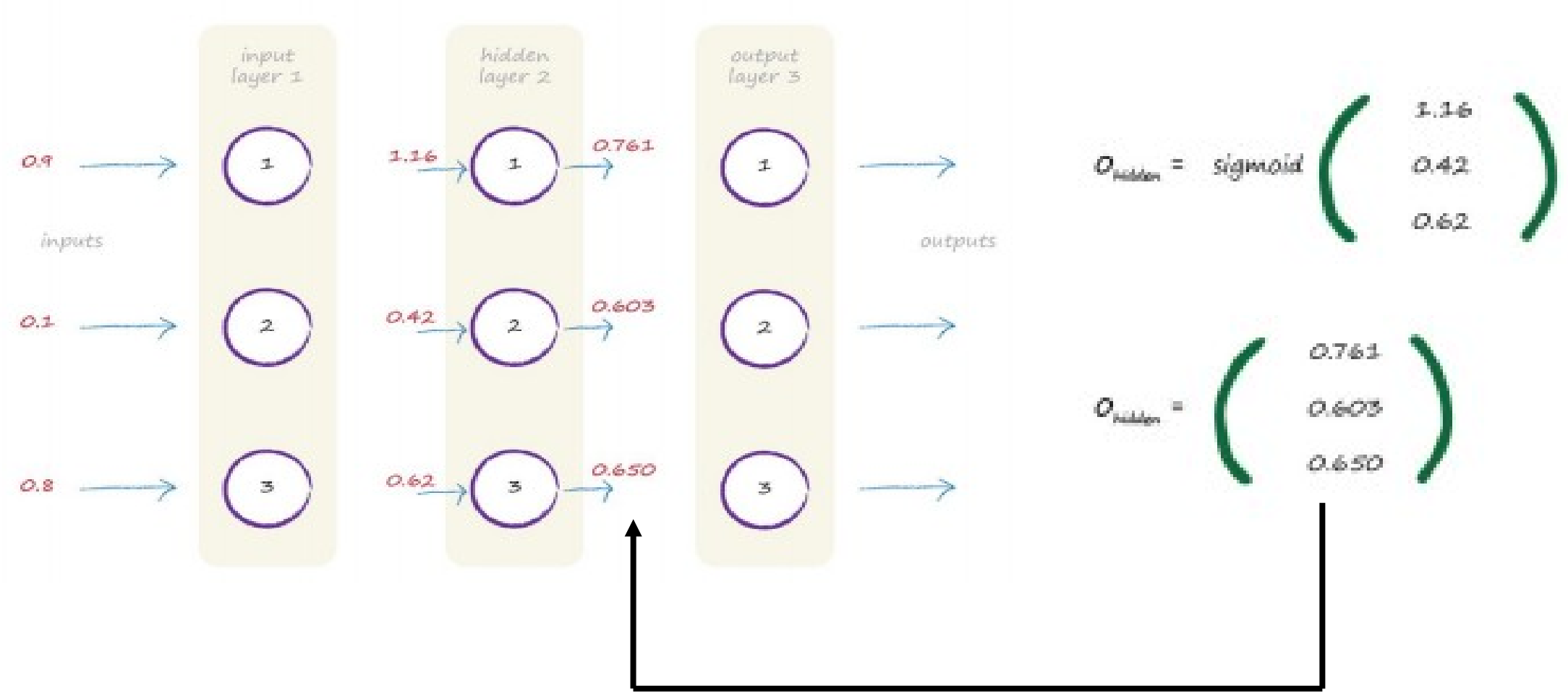


$$O_{hidden} = \text{sigmoid}\begin{pmatrix} 1.16 \\ 0.42 \\ 0.62 \end{pmatrix}$$

$$O_{hidden} = \begin{pmatrix} 0.761 \\ 0.603 \\ 0.650 \end{pmatrix}$$

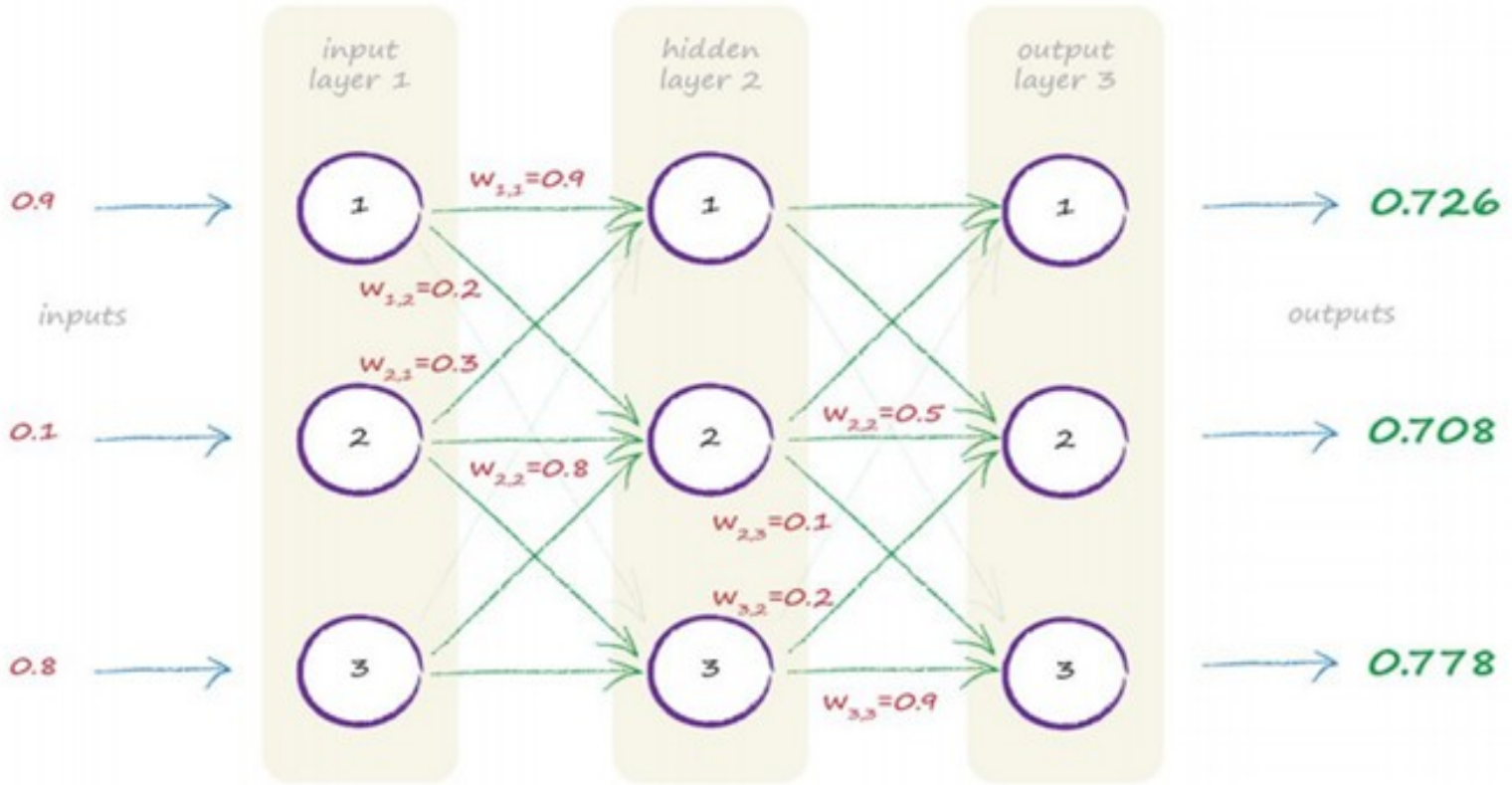
Adapted from: Tariq Rashid " Make Your Own Neural Network"

The Output of the Hidden Layers Is Calculated with the Activation Function



Adapted from: Tariq Rashid " Make Your Own Neural Network"

Same Procedure between Hidden and Output Layer Result is the Entire Neural Network



Adapted from: Tariq Rashid " Make Your Own Neural Network"

3. Calculate Error and Back Propagate to Weights: How to Get from Attributive to Numerical Data Output Data

SIEMENS
Ingenuity for life

- For 10 numbers, we got 10 output layer nodes: multiclass
- Each number is converted to a list for the actual output

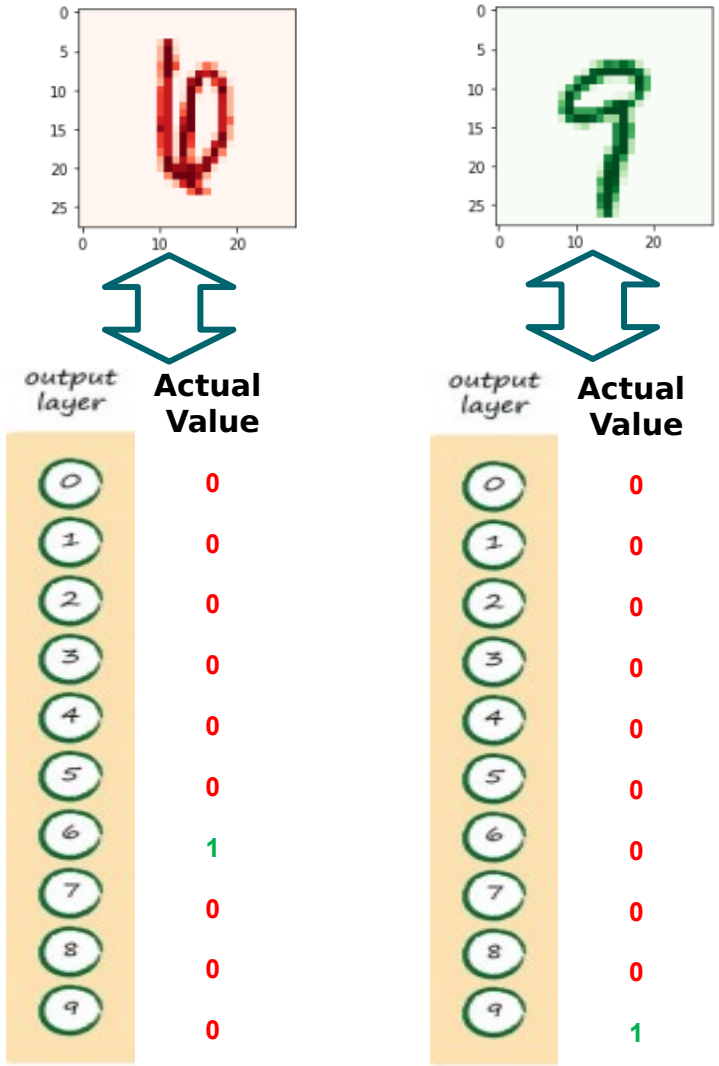
  0 ☾ [1,0,0,0,0,0,0,0,0,0]

  1 ☾ [0,1,0,0,0,0,0,0,0,0]

  2 ☾ [0,0,1,0,0,0,0,0,0,0]

  …

  9 ☾ [0,0,0,0,0,0,0,0,0,1]



3. Calculate Error and Back Propagate to Weights
How to calculate Prediction and Error in the Output Layer

SIEMENS
Ingenuity for life

Numerical predictions -> attribute predictions:

- Highest numerical output defines the predicted number

For each output layer:

- Error = Prediction – Actual

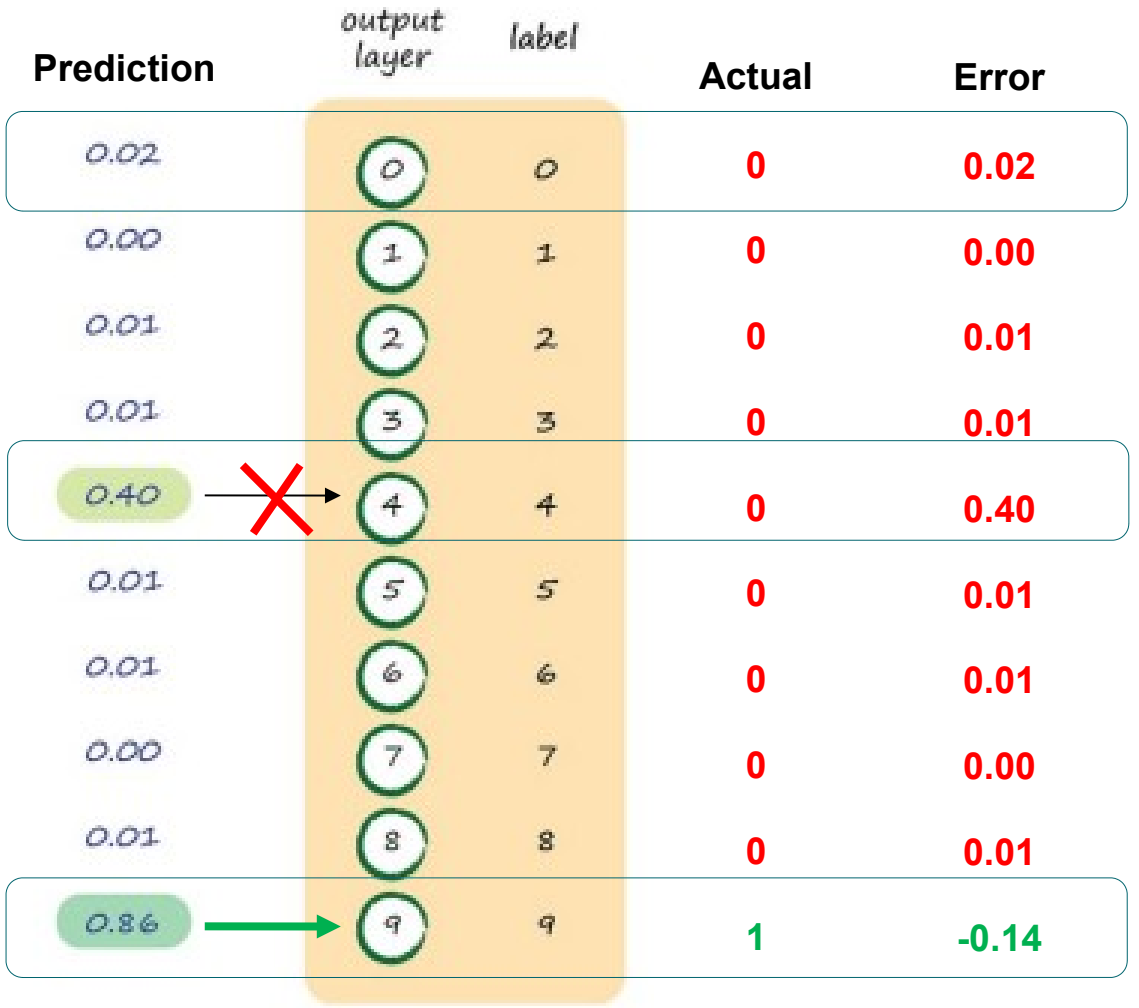
1: 0.02 - 0 = 0.02

…

4: 0.40 – 0 = 0.40

…

9: 0.86 -1 = -0.14



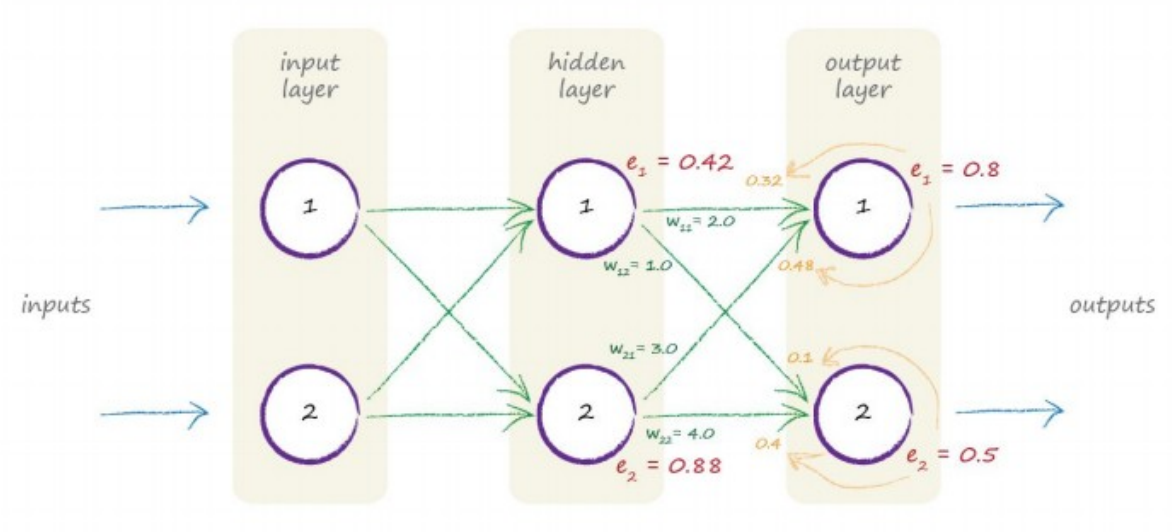
Adapted from: Tariq Rashid " Make Your Own Neural Network"

3. Calculate Error: Back Propagate Error

SIEMENS
Ingenuity for life

Error of hidden layer: Redistribute output error to

hidden layer based on weights



Calculation in matrix formulas:	$Error_{1\_output} = O_1 - Y_1$ $Error_{1\_hidden} = W^{-1}{}_{1\_hidden\_output} \times Error_{1\_output}$

Adapted from: Tariq Rashid " Make Your Own Neural Network"

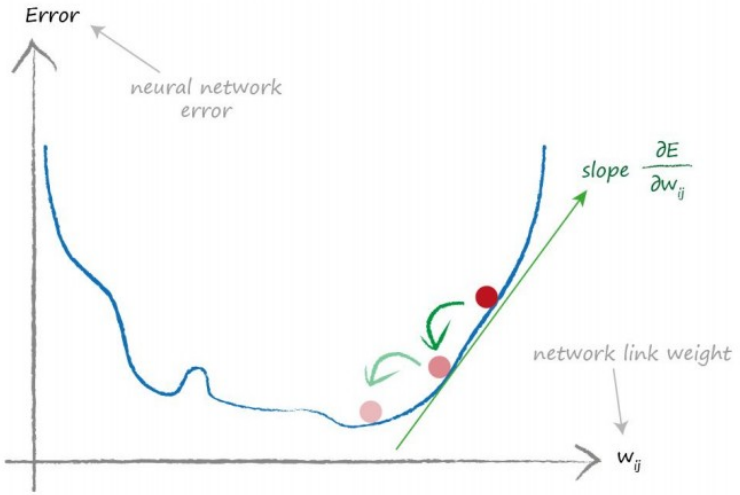
4. Recalculate Weights Based on Error -> NN_2 Gradient Descent Method:

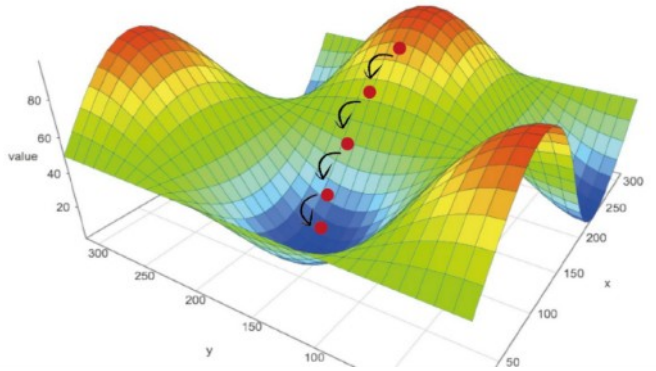
SIEMENS
Ingenuity for life

For each weight:

1. Calculate “Gradient” (derivative) of loss function (Error)=f(W_{IJ})
2. Principals:
   - We move weight in opposite direction than error/ loss function
   - We multiply with learning rate α
   - We multiply by derivative
     - For high slope error functions, we move weight a lot
     - For low slope we move it little
   - New Weight = Old Weight – LR * partial derivative of loss function with respect to weight

α = Learning Rate





Adapted from: Tariq Rashid “ Make Your Own Neural Network”

Review: Basic Steps of Training a Neural Network with Sigmoid Activation Function Using Statistical Gradient Descent (SGD) Solver

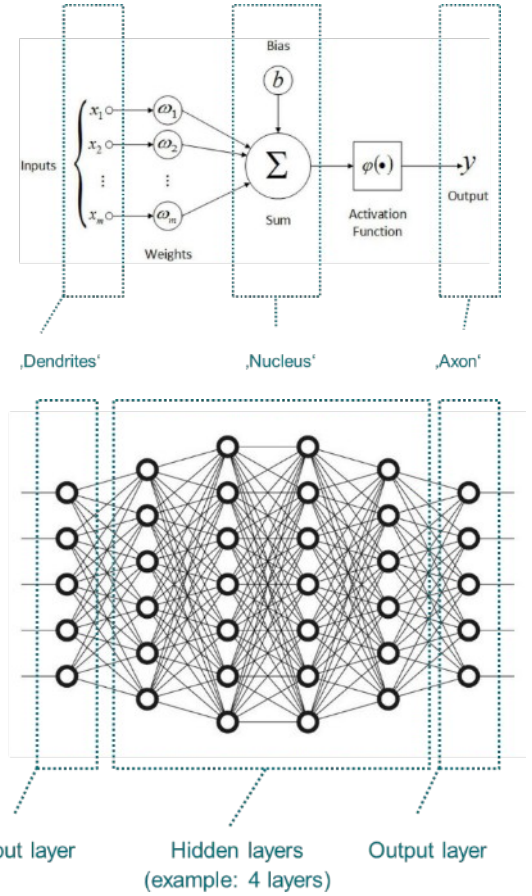
SIEMENS
Ingenuity for life

a) Select first element from the training data set: X_1 (Inputs) and Y_1 (Outputs)

   1. Start with random weights (some restrictions apply) -> NN_1
   2. Calculate predicted output O_1 using NN_1 and first element
   3. Calculate loss function (Error) $E_1 = O_1 - Y_1$ and back propagate to weights
   4. Recalculate weights based on error -> NN_2

b) Repeat step 2-4 with all elements from the training data set

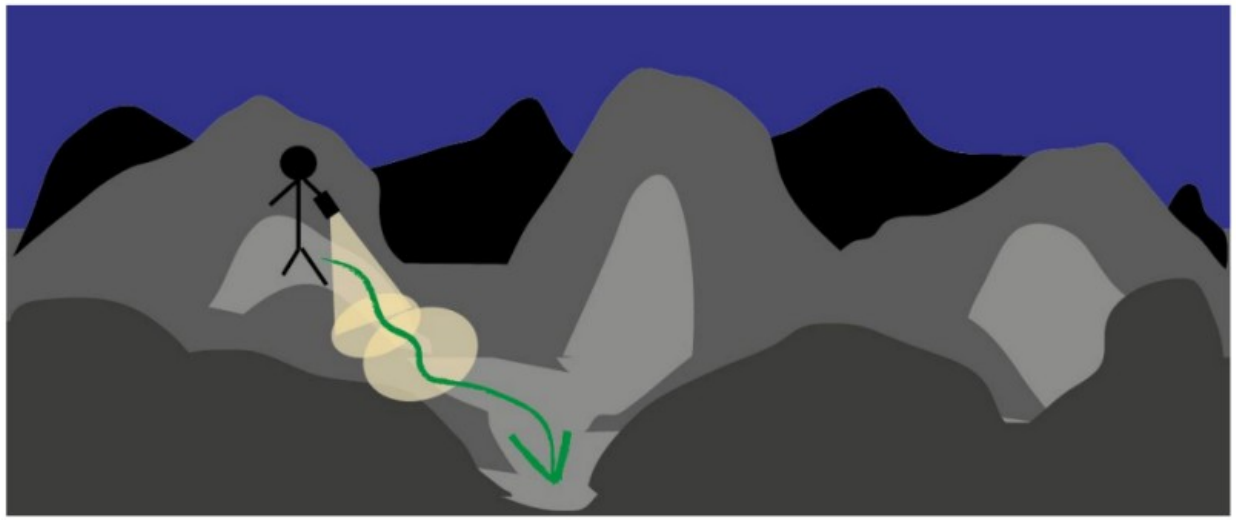
c) Iterate through all data several times (steps a) and b)) until weights converge to one value



Optimized Weight Updating Methods

SIEMENS
Ingenuity for life

1. Stochastic Gradient Descent (**SGD**):
   - replaces the actual gradient (calculated from the entire data set) by an estimate thereof (calculated from a randomly selected subset of the data)



2. **Adam**:
   - SGD based (with adapted LR, momentum: remembers last derivatives…)

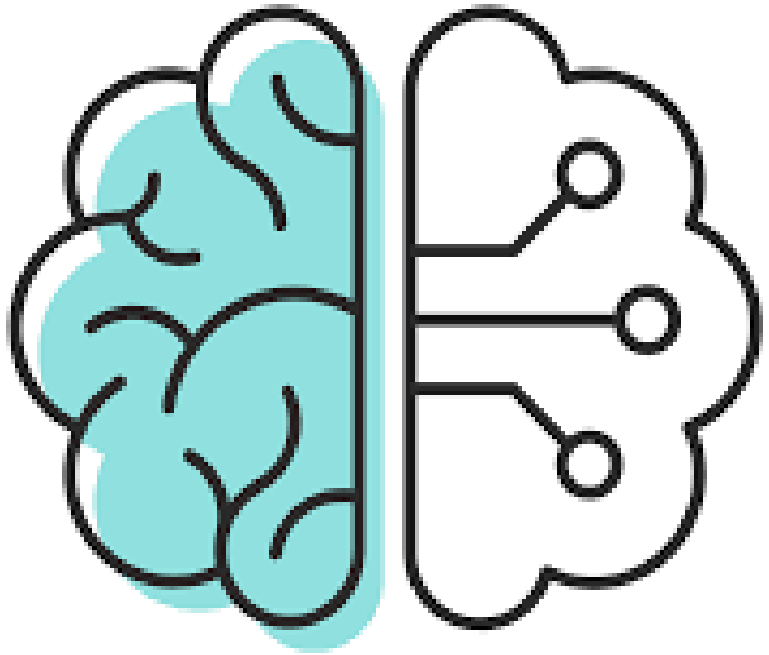
Overview: what we achieved so far

SIEMENS
Ingenuity for life

	Target values	Loss function	Error metric	Act function last layer	Act func hidden layers
Regression	Continous	MSE, R^2, MAE,…	MSE, R^2, MAE,…	Linear/ ReLu	ReLu
Classification	Binary	Binary cross entropy	F1 score, precision, recall, ROC-curve	Sigmoid	ReLu
	multiple classes	categorical cross entropy	F1 score, precision, recall, ROC-curve	softmax	ReLu

	Decision Trees	Artificial Neural Nets
Tabular data	Ensemble methods: Random Forest, Boosting	Starting from simple architectures
Text	-	Transformers (encoding, decoding, attention)
Images	-	CNNs

Agenda



- Review ML1
- Model Evaluation:
  - Regression Quality Metrics
  - Classification Quality Metrics
  - The Bias-Variance Trade-Off
- Model Types:
  - Decision Trees (Ensemble: Random Forrest)
  - Neural Networks
    - Basic Elements and Steps of Training a Neural Network
    - Neural Network Exercise
- Application of Machine Learning in Our Business

Exercise 3: Neural Network

SIEMENS
Ingenuity for life

1. Go to Exercise 3: Neural Network
2. Build different neural networks by changing #hiddenLayers, #neurons and learning rate. You can also change the activation function **(Only logistic or ReLU)**
3. To get a look at your results run through **result for logistic** and **result for relu**

Blue/Purple – Low test accuracy

Green – Mid test accuracy

Yellow – High test accuracy

Exercise 3: Neural Networks

Explore the Parameters **Learning Rate, Hi**
Recognition Dataset

...

```
########################################################## S

learningRate=0.0001 #Number
hiddenLayers=1 #Number
neurons=10 #Number
activationFkt='relu' #'identity', 'logistic', 'tanh' c

##########################################################
```

Observed Parameters for Neural Network

SIEMENS
Ingenuity for life

```
from sklearn.neural_network import MLPClassifier

MLPClassifier(hidden_layer_sizes=(10, ),activation='logistic',solver='sgd',learning_rate_init=0.0001)
```

Structure of hidden layers (default: (100,))
Determines the number of hidden Layers and the number of neurons in each layer

For 1 hidden Layer with 10 neurons (10,)
For 3 hidden layers with each 5 neurons (5,5,5)

Activation function (default 'relu')

Solver (default: 'adam')

Learning rate (default: 0.001)

For more information go to: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html#sklearn.neural_network.MLPClassifier

There are a lot more parameters you can variate to perfect your model. But therefore it's not always easy to find a very good combination of parameters.

Machine Learning in Python (Back Up)

SIEMENS
Ingenuity for life

- In the MLinPython.ipynb script you have examples for the code of different Machine Learning algorithms:

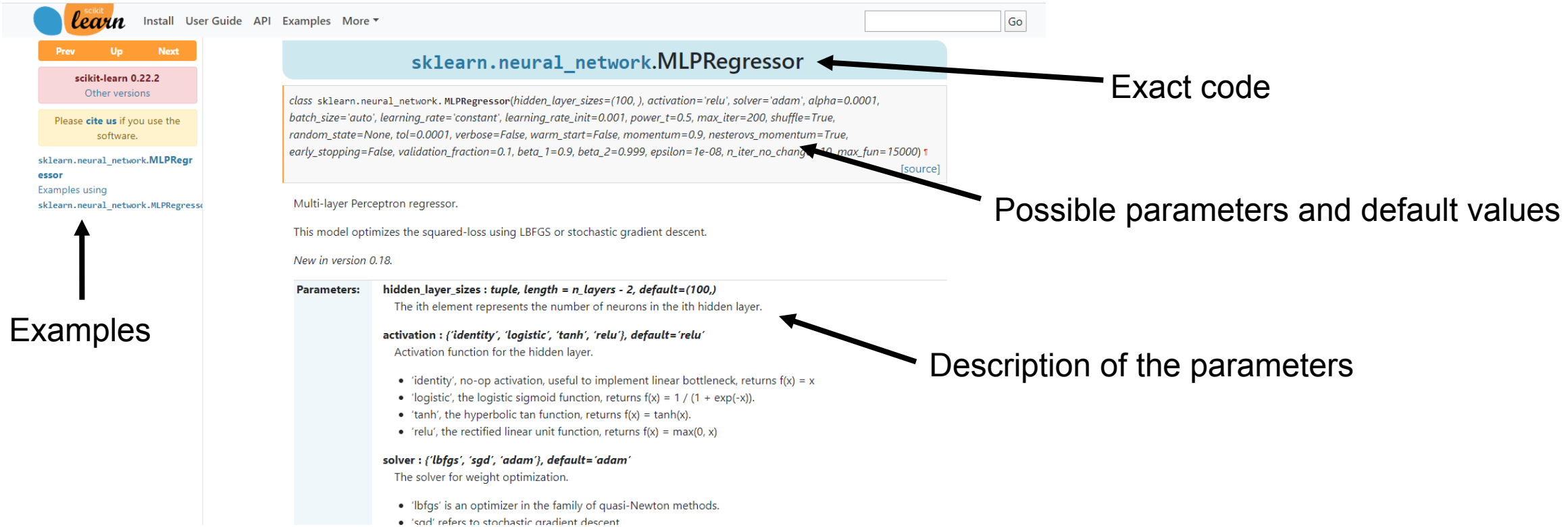
  - Decision Tree – Classification (using Iris): **DecisionTreeClassifier**
  - Random Forest – Classification (using Iris): **RandomForestClassifier**
  - Neural Network – Classification (using Iris): **MLPClassifier**

  - Decision Tree – Regression (using Bike Sharing): **DecisionTreeRegressor**
  - Random Forest – Regression (using Bike Sharing): **RandomForestRegressor**
  - Neural Network – Regression (using Bike Sharing): **MLPRegressor**

Machine Learning in Python (back up)

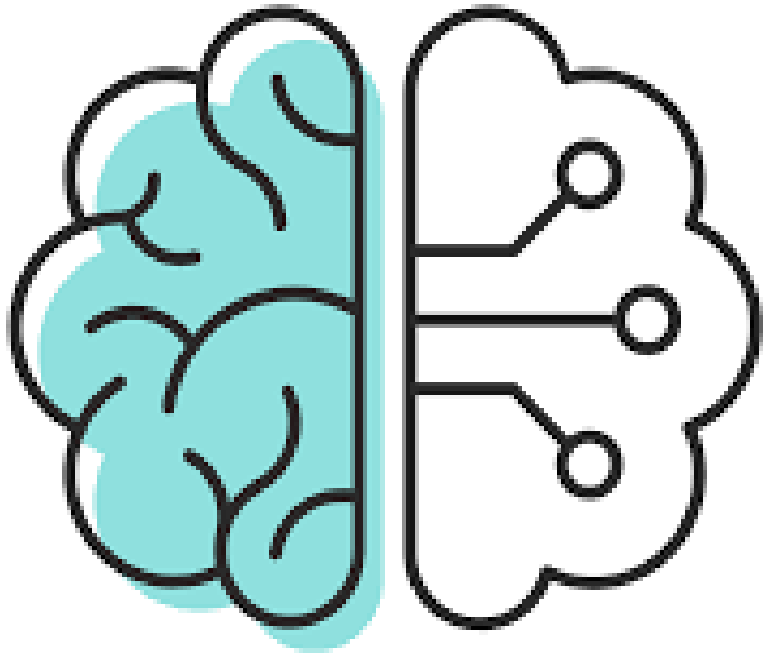
SIEMENS
Ingenuity for life

- For more details go to: https://scikit-learn.org/stable/
- Here you get an overview about the different tools and detailed descriptions of the different tools



https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html

Agenda



- Review ML1
- Model Evaluation:
  - Regression Quality Metrics
  - Classification Quality Metrics
  - The Bias-Variance Trade-Off
- Model Types:
  - Decision Trees (Ensemble: Random Forrest)
  - Neural Networks
    - Basic Elements and Steps of Training a Neural Network
    - Neural Network Exercise
- Application of Machine Learning in Our Business

What Could Be an Application in Your Business?

SIEMENS
Ingenuity for life

Judge on CMM data

Predict if a product will be defective or not based on some input variables.

Supply Chain Material Forecasting

Forecasting problems in engine using measurements from test engine and conditions in new engine

Optimization of production process and predictive maintenance

Generators are operating more and more in an flexible operating regime

process monitoring based on camera images

Coating booth input parameters

fleet analysis:(GT & ST) detect abnormal behavior

Predict number of spare needed per year based on number gas turbine fleet, operation hours, type etc.

Helping prediction of upcoming unfortunate weather using a lot of past data set.

Data quality check of test data

Determine the quality of product based on manufacturing process parameters thus replacing destructive testing.

Manufacturing non-conformance prediction based on machine, part, and operator data

Comparison of protoype test data and fleet data. Feasibility checks. Digital Twin.

Predict likehood of a purchase order comming to late for production

Predict Contract Cost (Basket costs) based on existing Contracts, the scope, the use. Even for unknow or non existing scenarios

Anticipation of project risks based on previously detected risks

Gt combustion data evaluation, root cause analysis

Predict the non-conformance cost of defects in the manufacturing process (e.g. for reworking) in order to prioritize the corrective actions.

Automated Timetables (terminpläne)

Based on the historical performance data of the site, forecast its operational performance for next year.

Challenges of Machine Learning Application in Turbine Eng.

SIEMENS
Ingenuity for life

Consumer Data Analysis

- Little knowledge about consumer.
- Users can be tracked very closely.

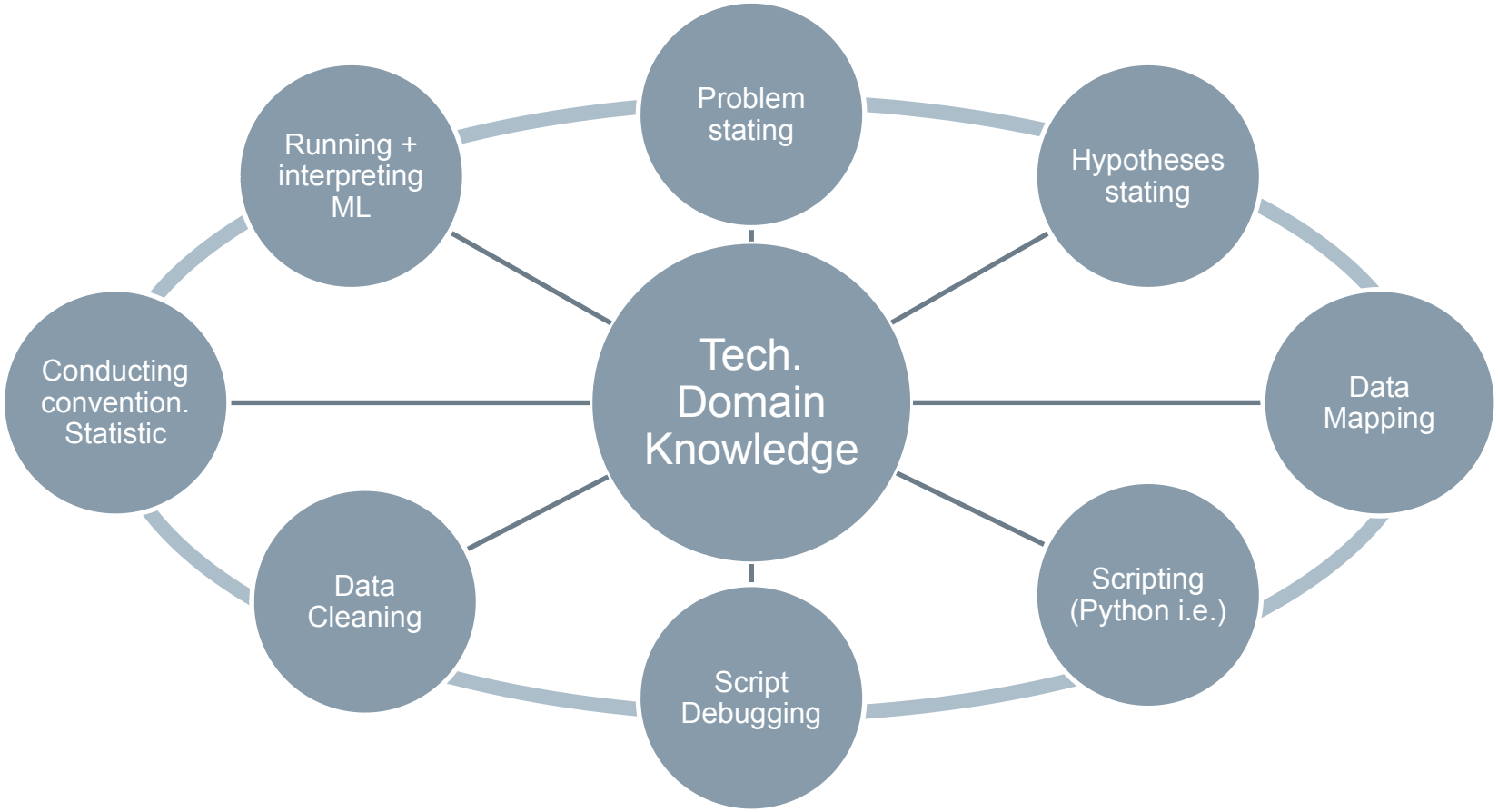
Turbine Data Analysis

- Starting from deep understanding -> very difficult to add value on top.
- Limited number of engines + change in time, operating conditions, ambient conditions.
- ML tools work best when “output data” exists (“supervised” vs. “unsupervised” learning).

Machine Learning is best applied if an output variable is measured (“labeled data”).

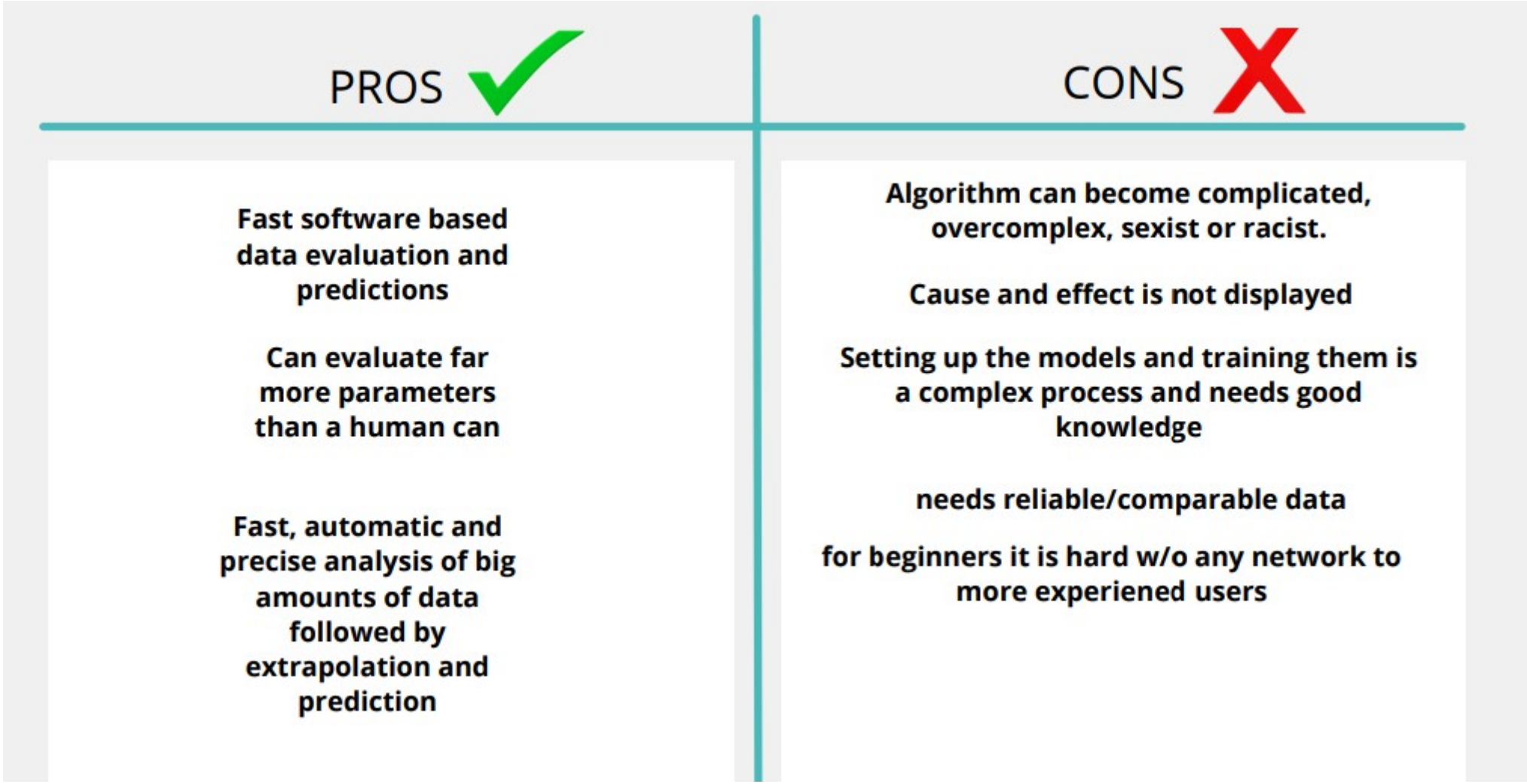
Data Science Is an Iterative Process.

SIEMENS
Ingenuity for life

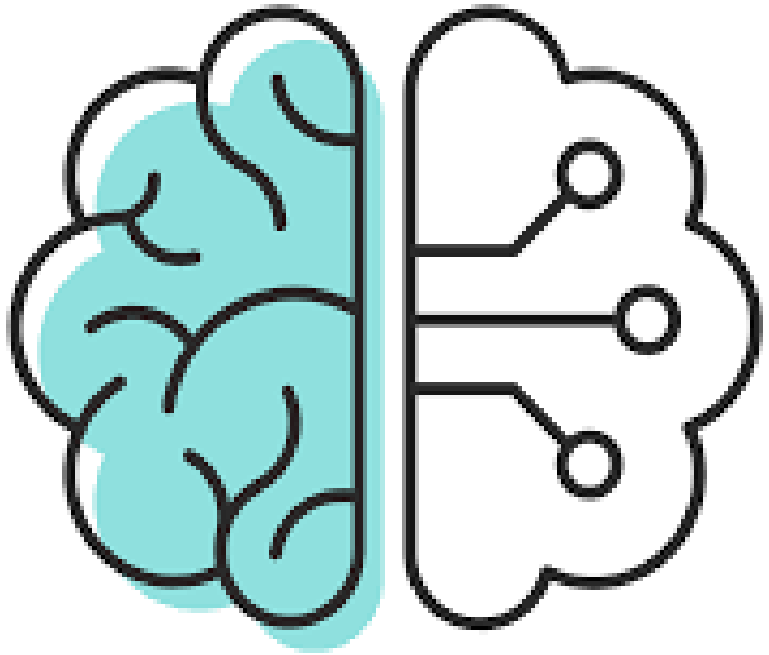


Exercise 2:
Brainstorming on Pros and Cons of Machine Learning

SIEMENS
Ingenuity for life



Agenda



- Review ML1
- Model Evaluation:
  - Regression Quality Metrics
  - Classification Quality Metrics
  - The Bias-Variance Trade-Off
- Model Types:
  - Decision Trees (Ensemble: Random Forrest)
  - Neural Networks
    - Basic Elements and Steps of Training a Neural Network
    - Neural Network Exercise
- Application of Machine Learning in Our Business

Overview

SIEMENS
Ingenuity for life

	Target values	Loss function	Error metric	Act function last layer	Act func hidden layers
Regression	Continous	MSE, R^2, MAE,…	MSE, R^2, MAE,…	Linear/ ReLu	ReLu
Classification	Binary	Binary cross entropy	F1 score, precision, recall, ROC-curve	Sigmoid	ReLu
	multiple classes	categorical cross entropy	F1 score, precision, recall, ROC-curve	softmax	ReLu

	Decision Trees	Artificial Neural Nets
Tabular data	Ensemble methods: Random Forest, Boosting	Starting from simple architectures
Text	-	Transformers (encoding, decoding, attention)
Images	-	CNNs

Please Give Us Feedback!

SIEMENS
Ingenuity for life

- **Feedback is very important for us. This course is free, but we need to know if this is beneficial for you.**
- **Based on your feedback, we will continue to offer this, develop additional courses – or not.**

- Please visit at the end of this course the following website: https://forms.office.com/Pages/ResponsePage.aspx?id=PqlLJW8f80iQ5uJ2ZmS0d-5LdjYRShZEhN3YmsD7St9UMFZZTFFVQjFSUUxTMzhQNTM3S1hSM0ExVS4u